Agenda

Approach → Identify the problems

Approach → Find ways to solve them

Why RAG is imp?

What is RAG?

How RAG is implemented?

Nov 2022 → Launch of ChatGPT → intelligent, powerfull, fast

Domain → tech, coding, finance, Healthcare

GPT → Transformer Architecture (DL Model)

↓ Self Attention Mech

Text → Language Patterns

Training

LLMs → large language Models → AI Models

Parameters (Weights / Biases)

NLU

Natural language Understanding

Language Patterns → Generative capability

Internet → Articles, books, webpages, blogs, reviews, coding repos

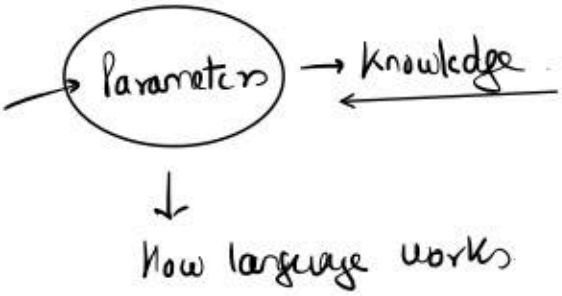
{ compute Energy }

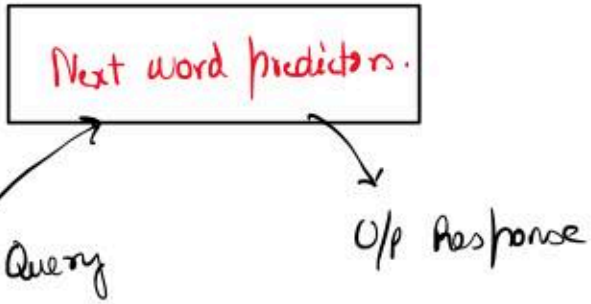
weeks / few months

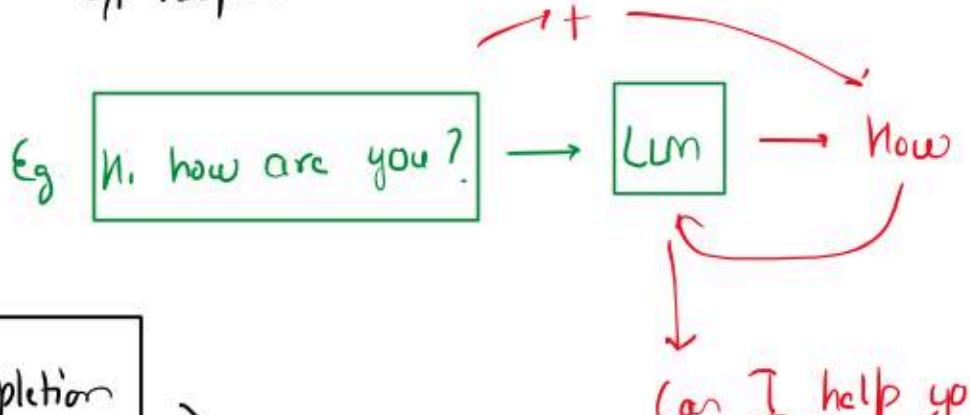
Compute
Manpower
Storage
Infrastructure
} Energy

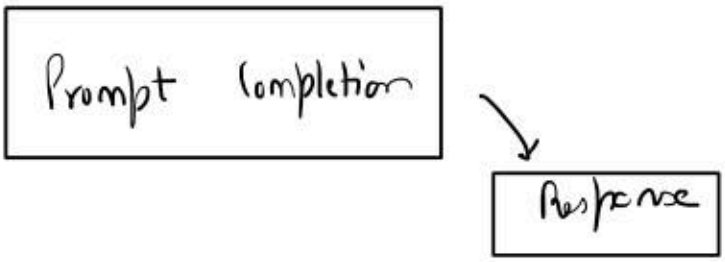


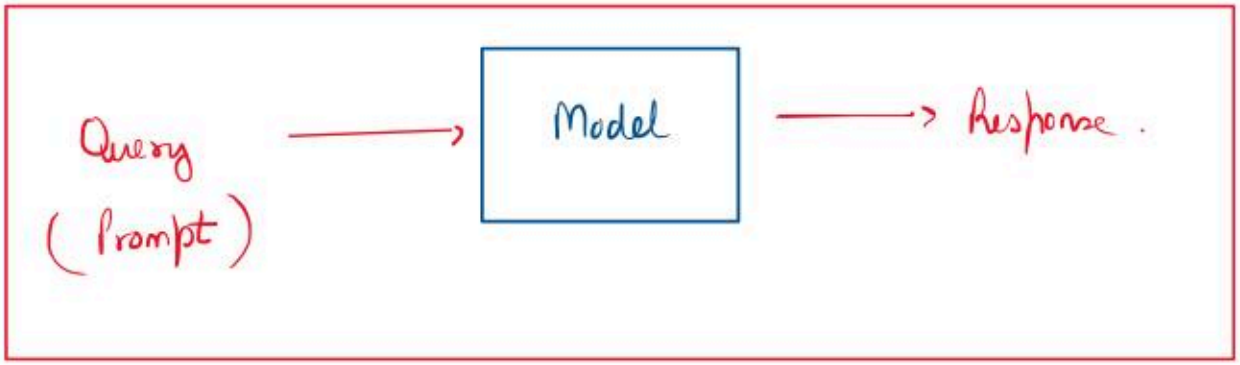








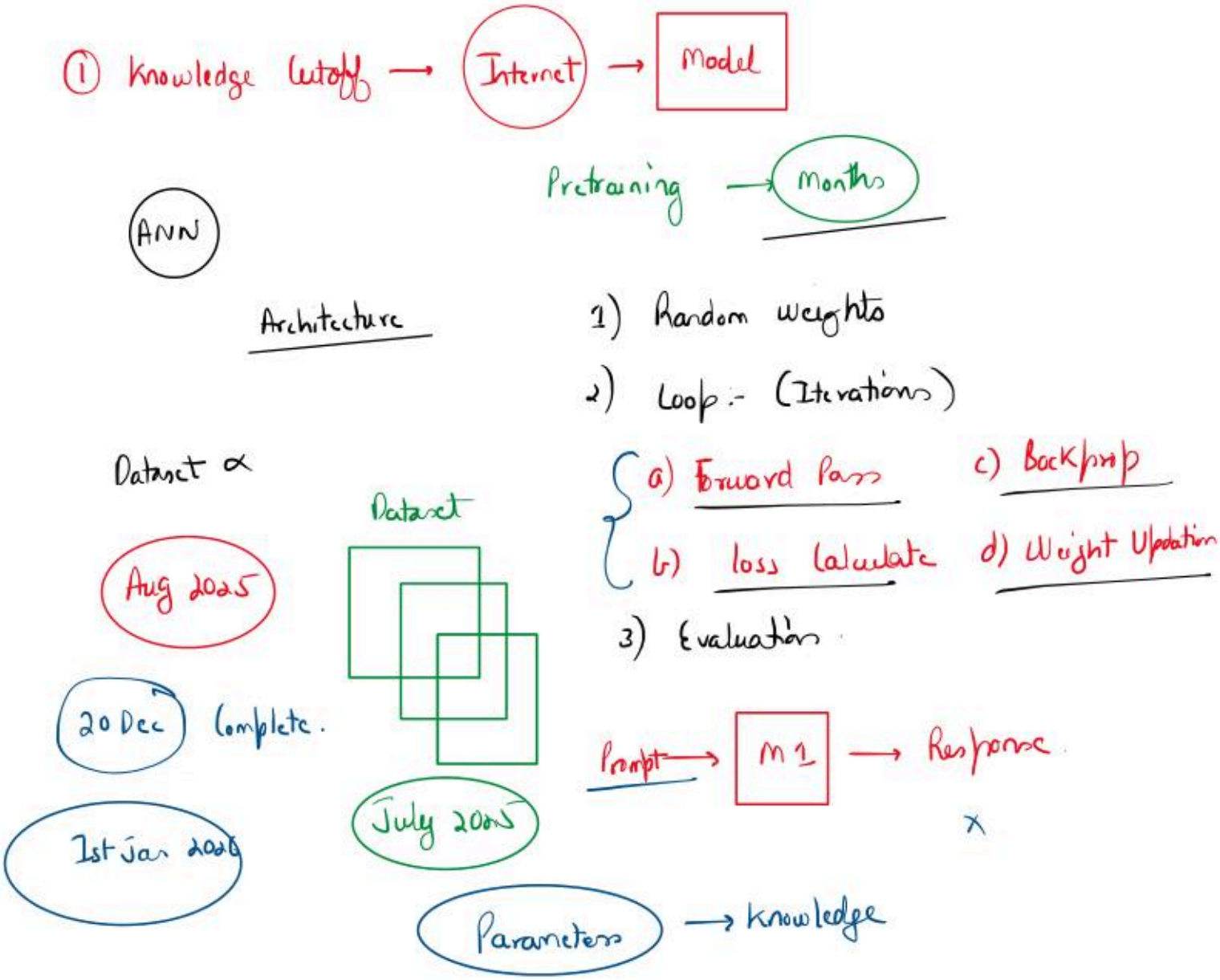




Inference Phase

Four Problems in LLMs

1. **Knowledge Cutoff** - Models frozen in time, can't access recent information.
2. **Hallucinations** - Models generate plausible but false information confidently.
3. **No Source Attribution** - Can't verify or trace where information came from.
4. **No Private Data Access** - Can't access company/proprietary documents.



② Training time

② Hallucinations → False facts with full confidence.

Eg. Agentic AI → Paper

Prompt

NLU → Generative

Factual ✗

False facts → Real

3) No Source Attribution

What is linear Regression?

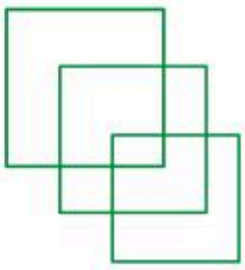
query → response

↓

verifiable source ✗

Training

Source → Knowledge Parameters



④ Cannot Access Private Data → Public Data.

Parameters

Parametrised Knowledge.

knowledge.

1) Knowledge cut off
2) Hallucinations
3) No Source Attribution.
4) No Access to Private Data.

(1) Knowledge cut off → a) Large training times } Pretraining
b) Static dataset

LLM → Complexity

Parameters + Additional knowledge

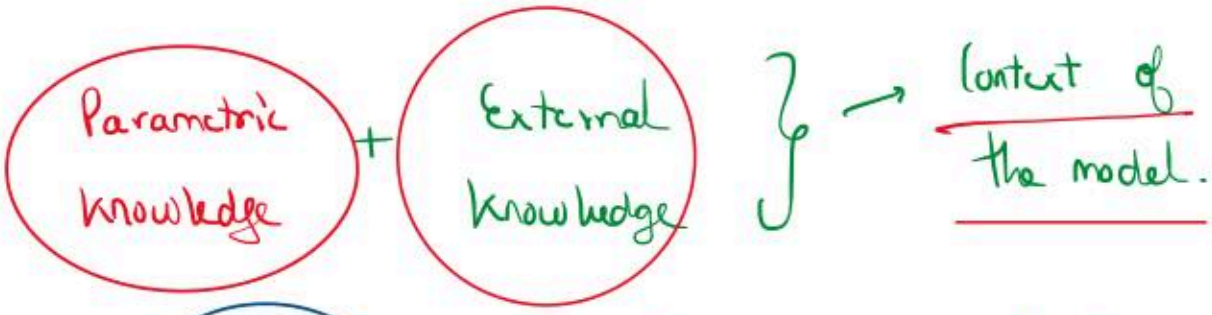
(2) Hallucinations → Knowledge ✓ → Correct
Knowledge ✗ → hallucinate

Parameters + Additional knowledge

(3) No Source Attribution → Parameters Source ✗
+ Source information.

4) No Access to Private data → Parameters

+ External knowledge



Context → What the model currently knows

Knowledge the model has.

ICL → 1000 page

X

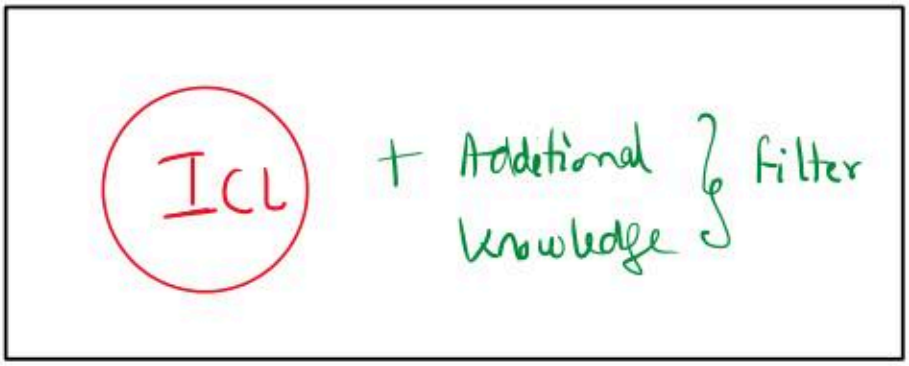
Transformer

Self Attention

Context Window

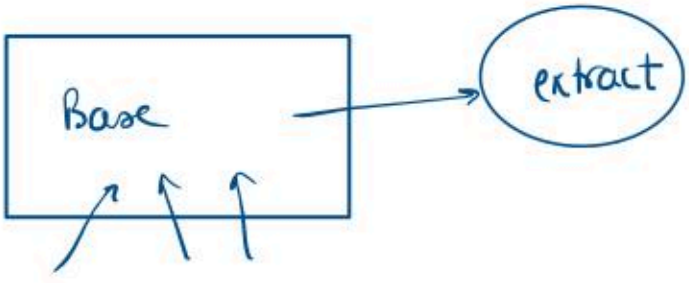
200k, 1M

lost in the middle



query + Additional context

∝





→ Retrieval Augmented Generation.

1) Prompt Eng

2) Fine tuning

Context

Parametric knowledge.

M1

Input

Context → Additional knowledge

Hi my name is Hinanshu

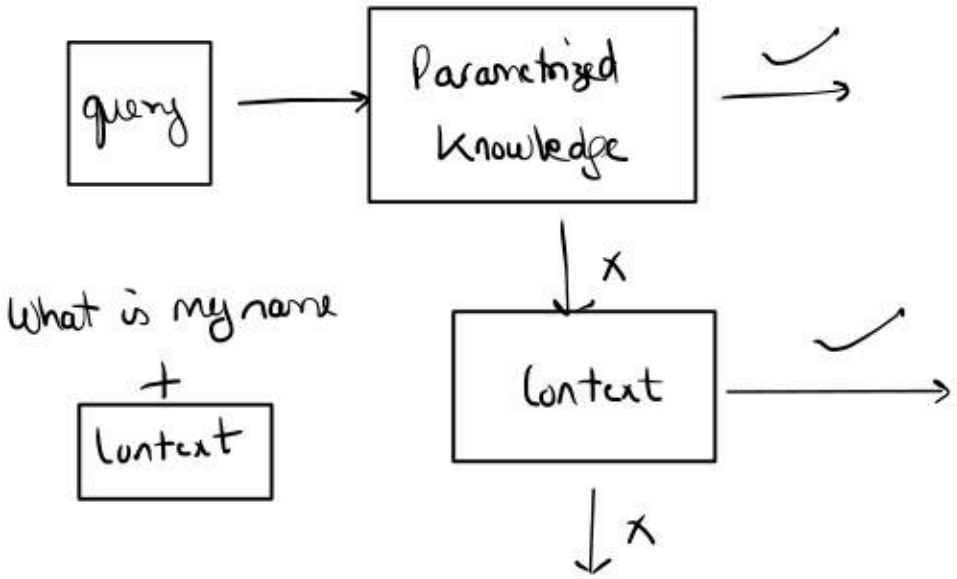
Hi Hinanshu How r u ?

What is my name?

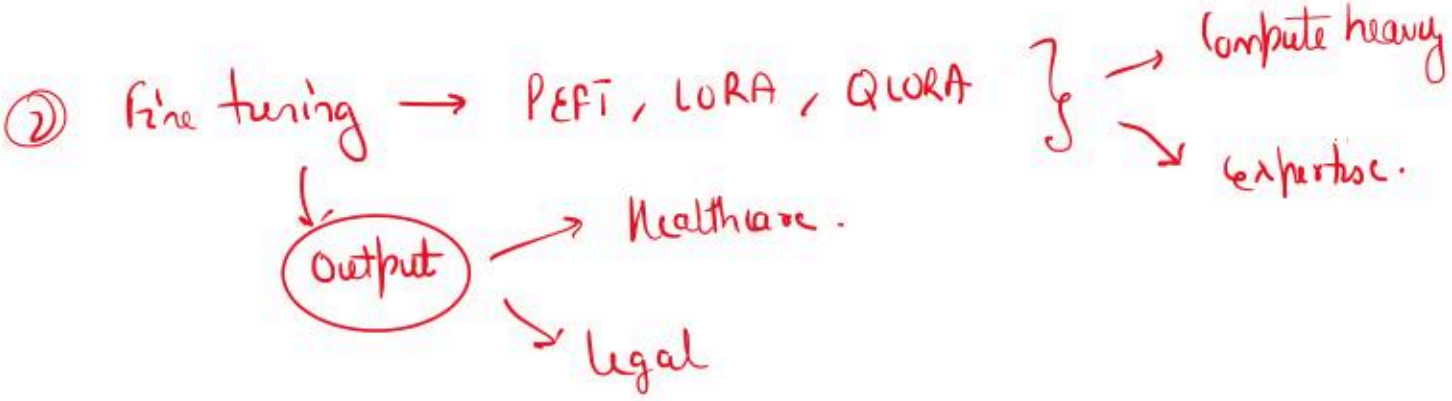
Your name is hinanshu.

In context learning (ICL)

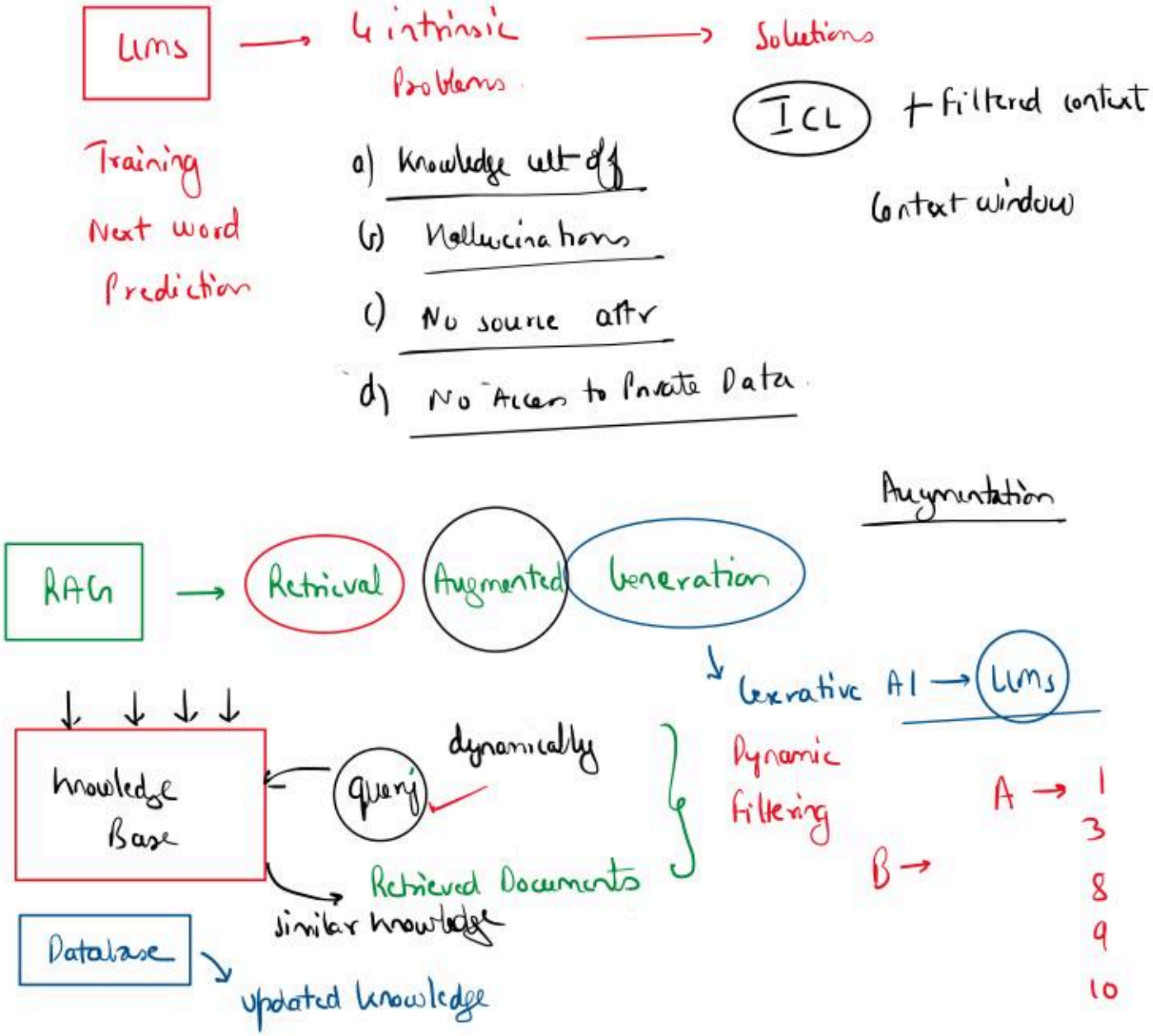
We can add additional / external knowledge to our model through prompt.

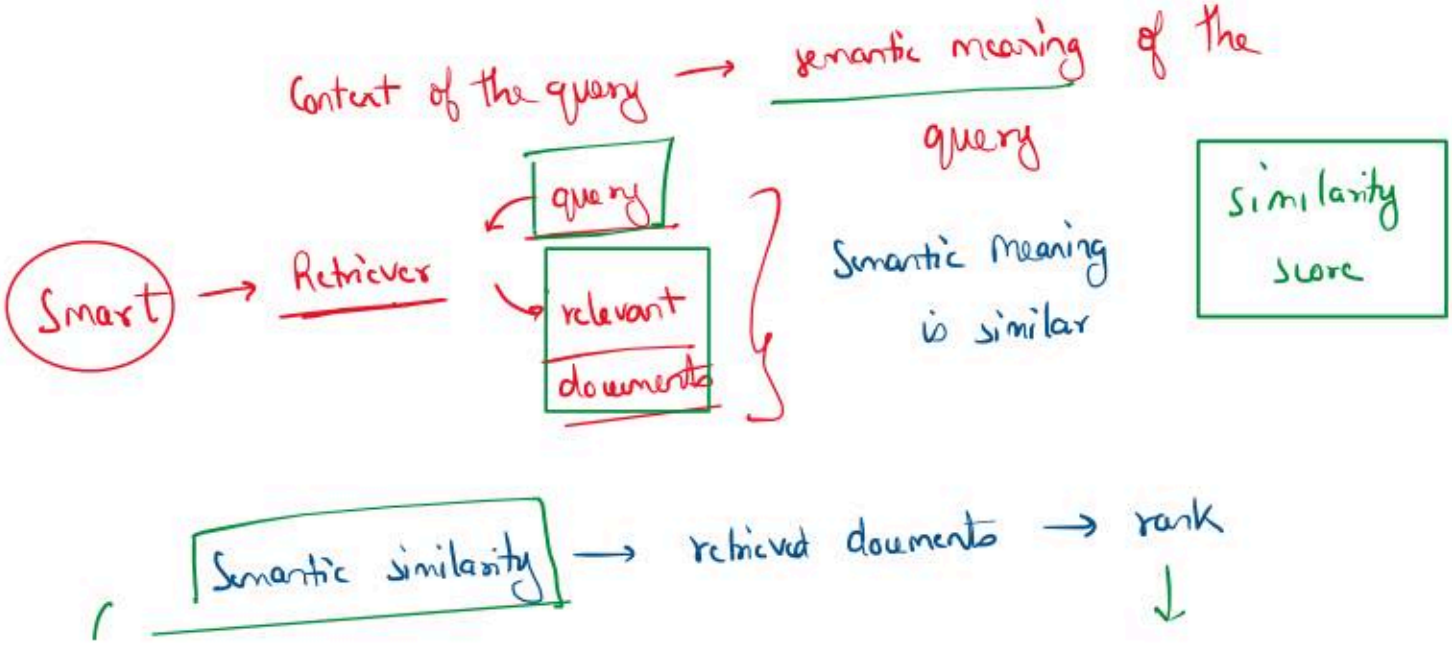


→ Compute heavy


② Fine tuning → PEFT, LORA, QLORA
Compute heavy
Expertise.
Output
Healthcare.
Legal

What is RAG?


LLMs
4 intrinsic Problems
Solutions
ICL + filtered context
Context window
Training
Next word Prediction
a) Knowledge cut-off
b) Hallucinations
c) No source attr
d) No Access to Private Data
Augmentation
RAG
Retrival
Augmented
Generation
Generative AI → LLMs
Knowledge Base
query
dynamically
Dynamic filtering
Retrieved Documents
similar knowledge
Database
updated knowledge
A → 1 3 8 9 10
B →


Context of the query → semantic meaning of the query
query
Smart
Retriever
relevant documents
Smantic meaning is similar
similarity score
Semantic similarity → retrieved documents → rank

Semantic similarity → retrieved

↓

Similarity store

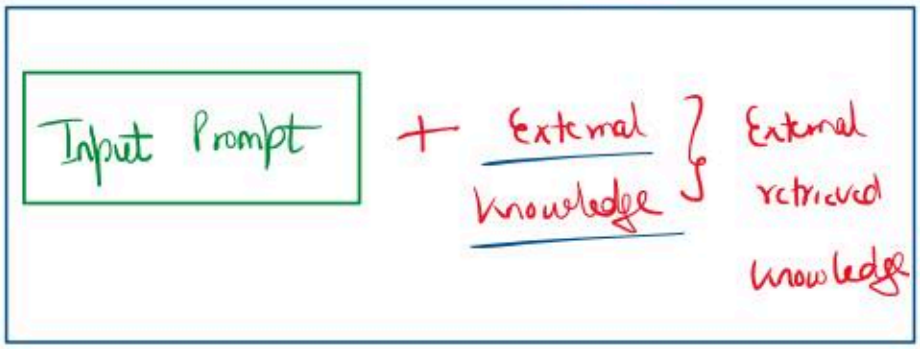
Further filtration

15 → 4 → Sort Descending order

Top 4 most similar documents

Augmentation → To enhance something

Retriever → top most similar meaning documents



ICL + filtered context

Input query

Context Assembly

Prompt → Augment

ICL

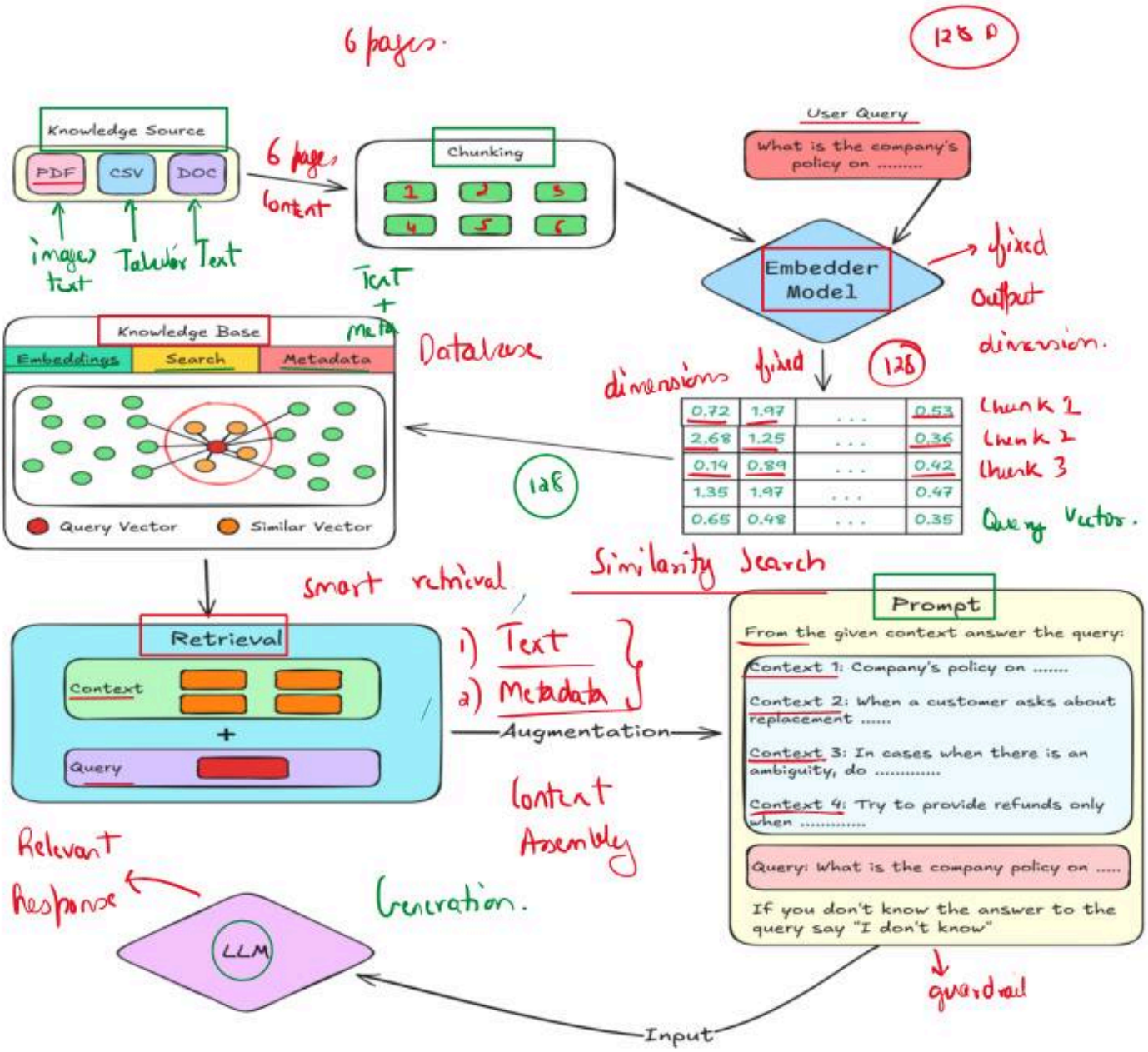
Relevant Answers

Context window ∝ Lost in the middled

Added context → limited

Dynamic

Relevant → Most similar to your input query.



Recap :-

Why RAG is imp ? → RAG

What is RAG?

Context Assembly

How RAG works ? How RAG Pipeline works?

→ Input for your RAG Pipeline. Private Data

→ Input for your RAG Pipeline.

① Knowledge Source : Source / Input of your external knowledge

External files

pdf, txt

pdf html
csv mp4
doc .mkv
txt .mp3
md

google drive, S3 bucket AWS

Document loaders. → Parse

Load → Memory

Parse → Content

Structure of the document

Document

metadata.
{ type: ,
name: ,
date of creation,
size }

pdf

text

table

image

① loading

② Parsing the document

② Chunking → 1 pdf → 100 pages

To break a large document into parts

Page wise chunking

100 Pages
Pdf

Chunking

100 documents

metadata

{ page no: ,
document name ,

{ page no. /
document name,
file-type }

NLP
↓
Bow, Tfidf.

③ Embedding → Text → Numbers.

↓ DL Models

↘ Semantic Meaning
↓
Numbers.

128 128

↓ fixed output dimensions.

500 words.
5000 words

pdf → 100 Pages

Query → Parameters.
↘ Response

RAG

100 documents
↓
100 vectors → store.

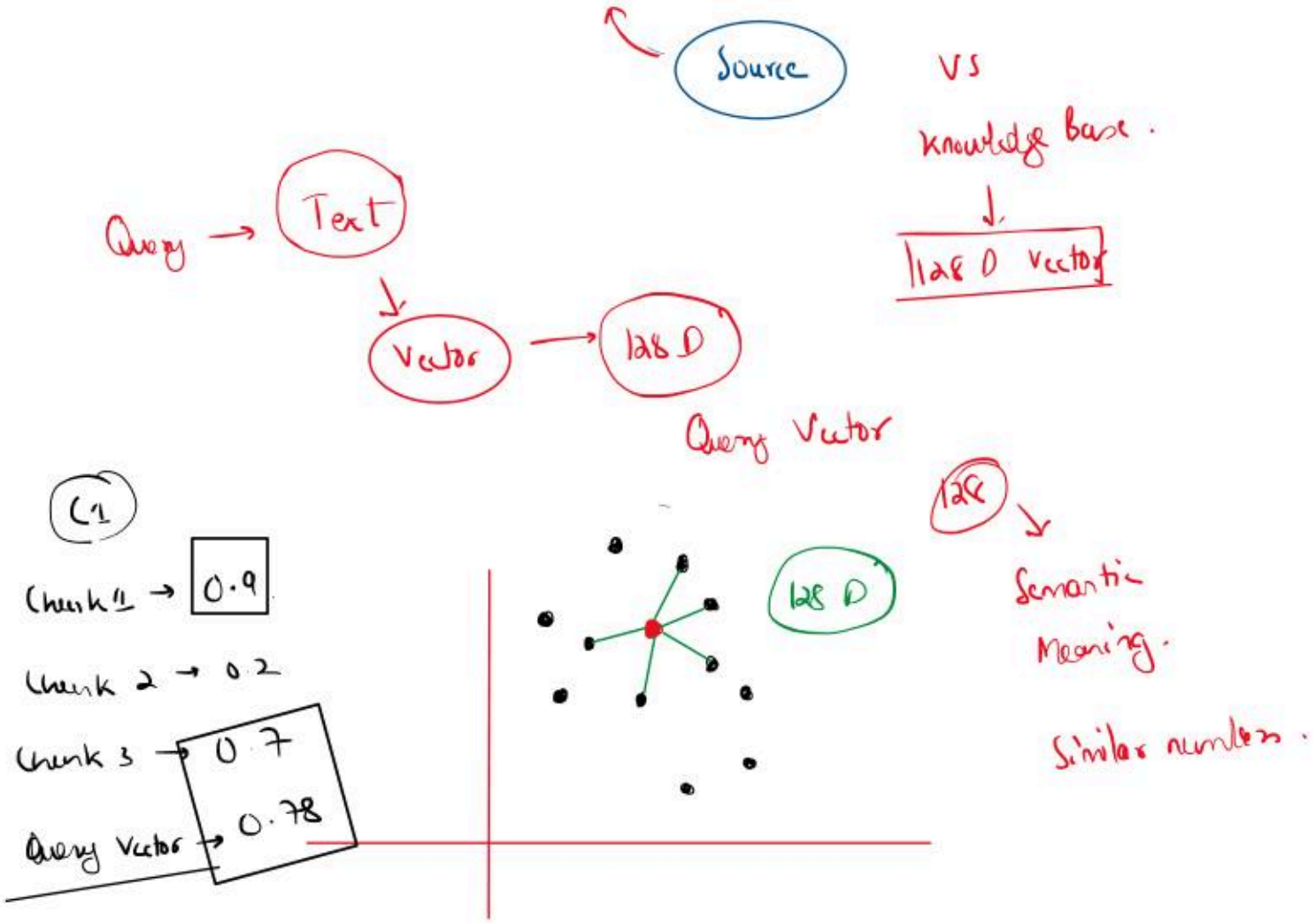
✓ Document loading
✓ Chunking
✓ Embedding
} Time taking steps

④ Database → Vector store → Knowledge Base.

CRUD

Vector + { Text + metadata }

Augment
Text



5) Retrieval → Text + Metadata

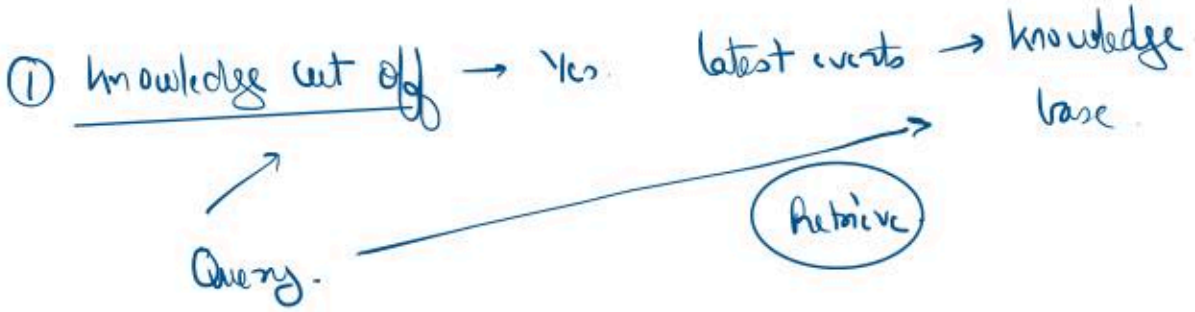
Similarity search

6) Augmentation → Final Prompt

Retrieved context + Query.

LLM

Response.



② Hallucinations → Knowledge
Yes. Context

Ground in facts → Source

③ No Source Attribution → Yes.

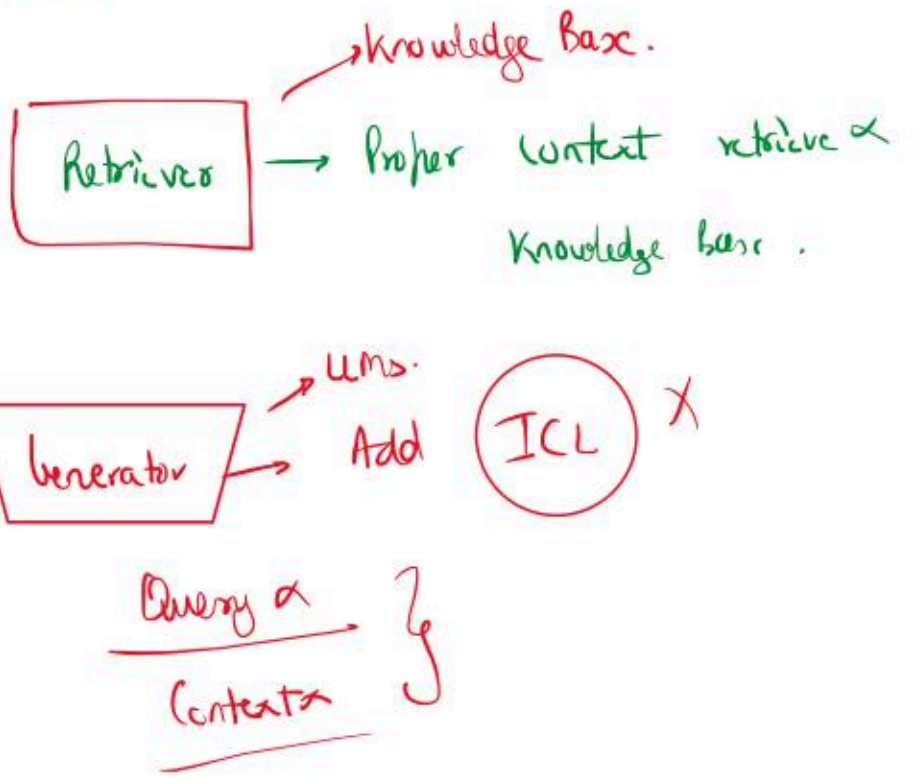
Text + Metadata

④ No Access to Private Data → Yes → Query

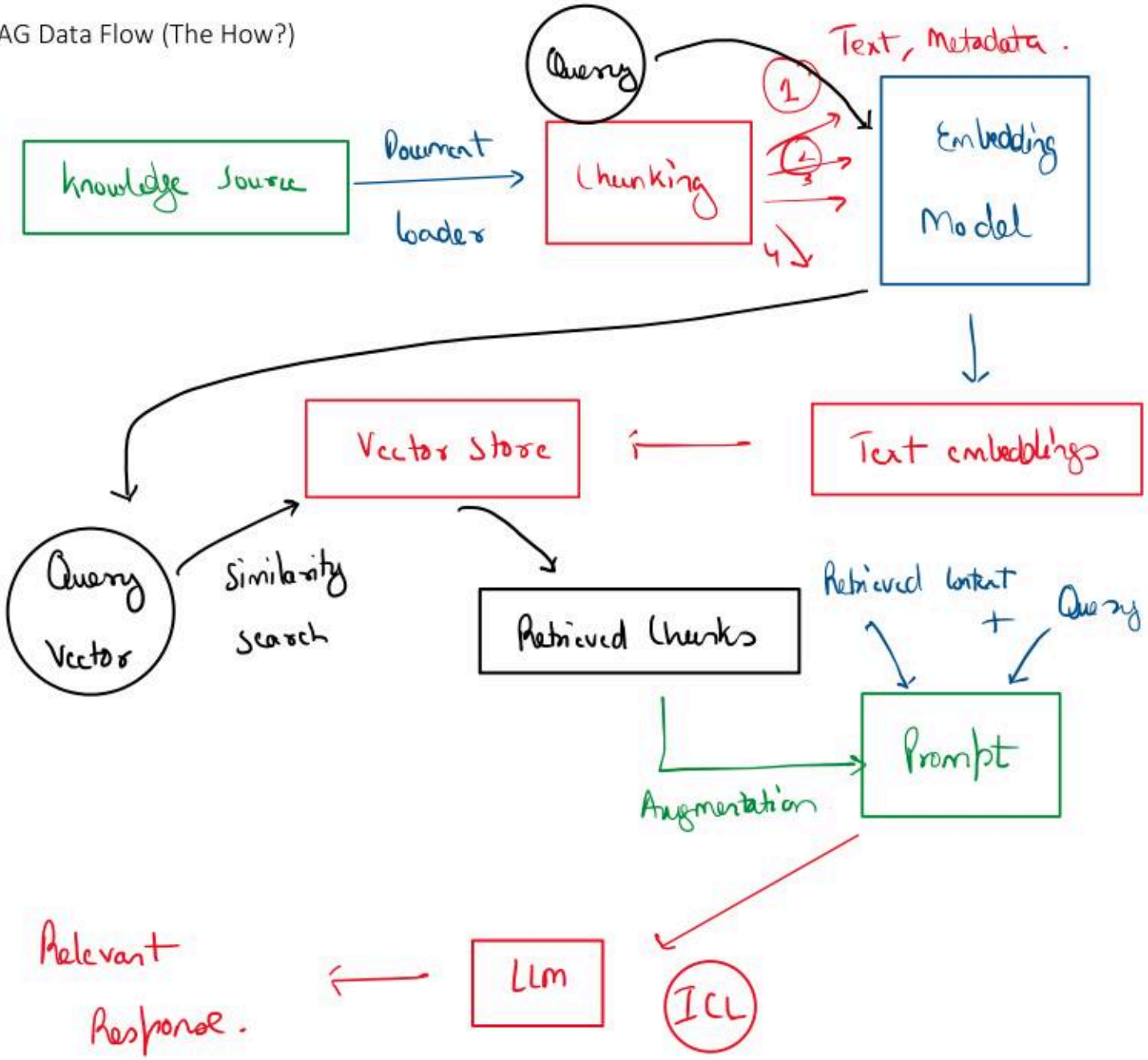
① Retriever Failure.

② Generator failure.

Garbage



RAG Data Flow (The How?)



Why RAG is required?

What is RAG?

How RAG is implemented?

LLM → Train

1) Knowledge cut off
2) Hallucinations
3) No Source Attr.
4) No Access to Private Data.

Additional knowledge → Externally

ICL

Prompt + Context → Answers.

① Context window
② Lost in the middle

Filter → Relevant → RAG

Retrieval → Filtered Relevant

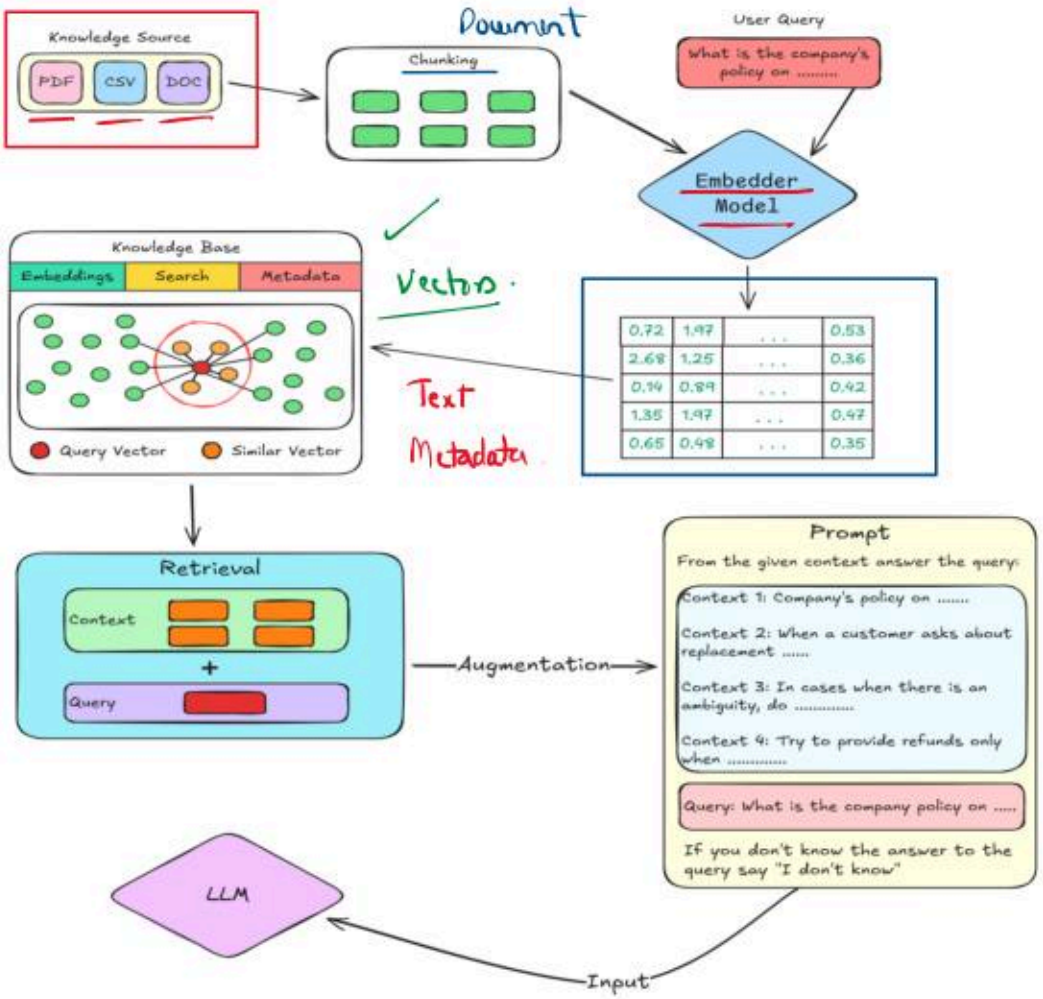
smart → Highly Relevant + dynamic → Query

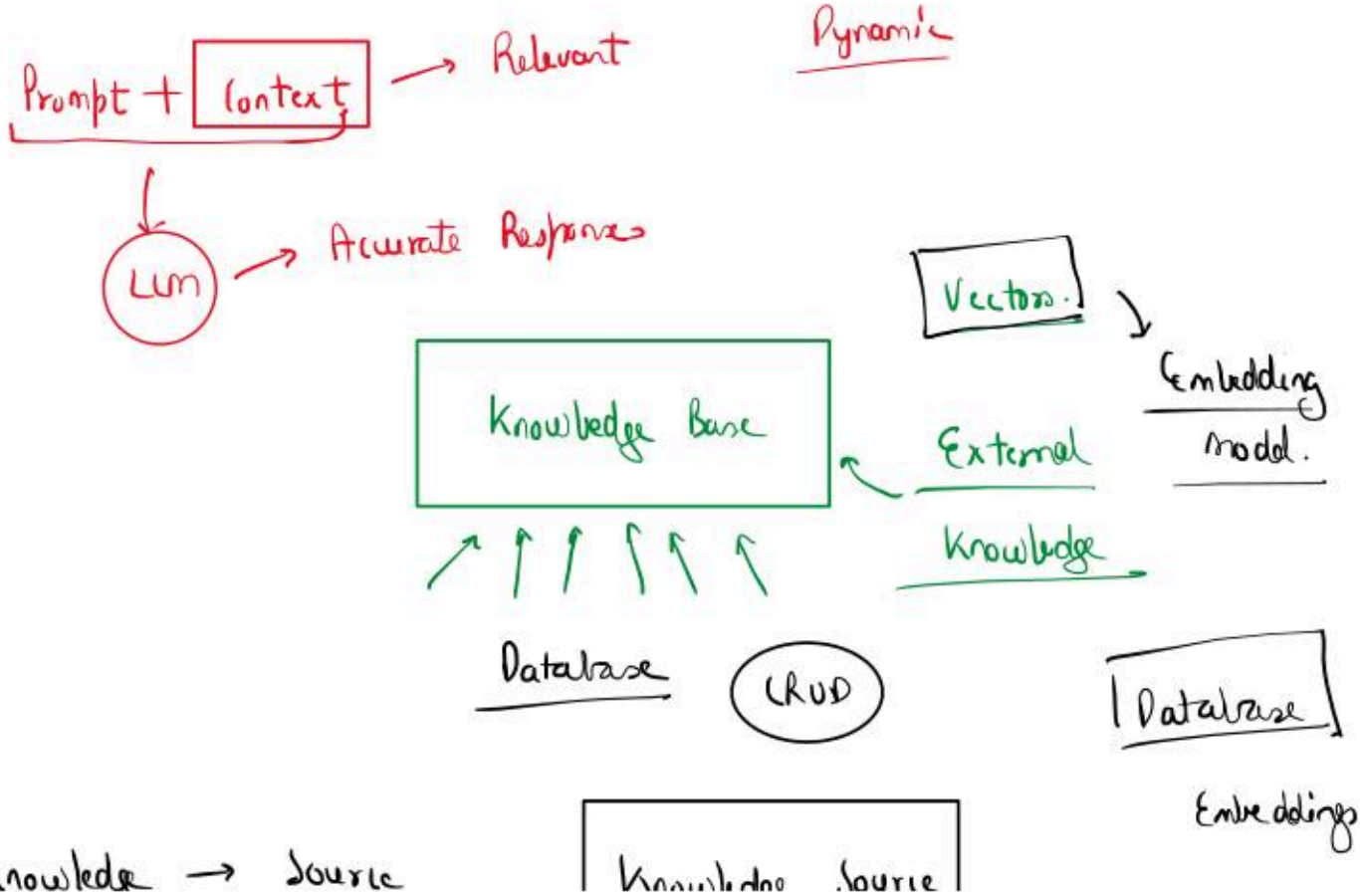
Prompt + Context } Augmentation

↓ response

RAG

Why are they required





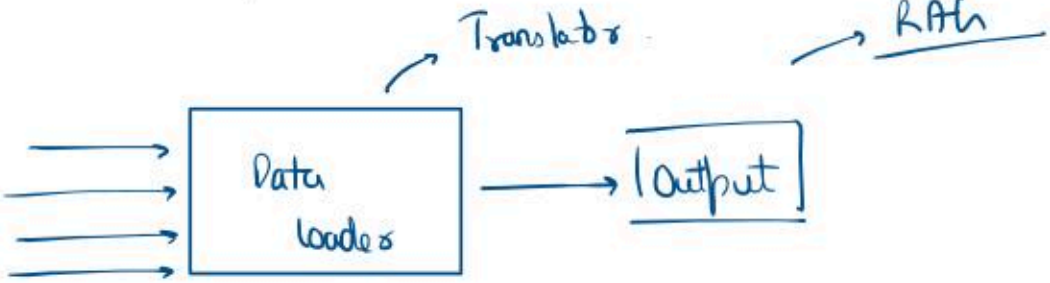
Embeddings

Knowledge → Source → Knowledge Source

.txt , .pdf , .md , .html , .csv

Knowledge Source → Directory → Multiple files (text) → Different formats

Data Ingestion



HDD

Components — After pipeline.



③ Key task

a) Load the document

b) Parse the document

c) Extract and Output

markdown

Title

- - - -

- - - - -

1.
2.
3.

##

html.

 . . .

<id> . . . .

CSV

id, name, age, gender

1, Rahul, 20, M.

2, Neha, 28, F

Specialized Parser → Document structure.

Extract the text.

Output → Unified format

formats

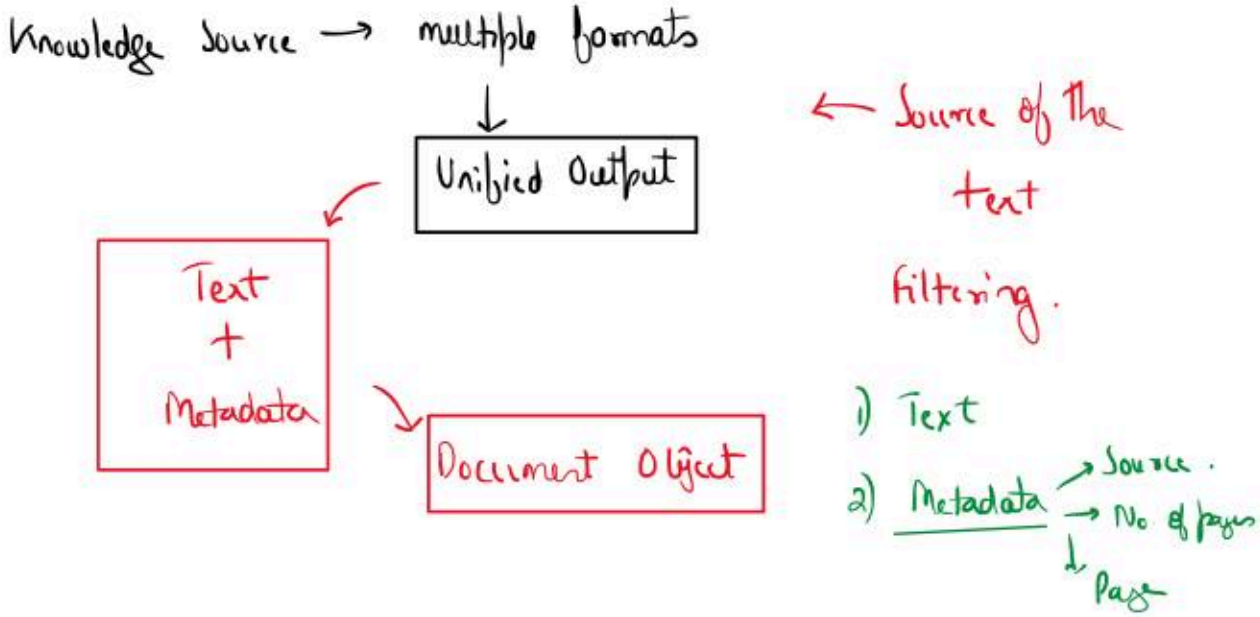
single format

Downstream

Document

Variation ∝

Problem they solve




```
Document {
  page_content: str,   # The actual text extracted
  metadata: dict       # Information about the source
}
```

→ Text

Document loader → Document → Text, Metadata.

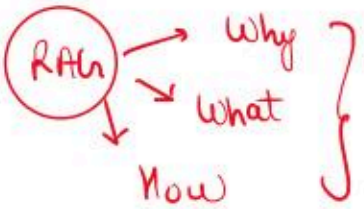
Document loaders (Base loader) →

Input → Document type: separate Document loader — Specialized

Document loaders.

Code

Chunking strategies → Text splitters



Components of RAG :-

① Document loaders

② Text splitters

Document loaders → Data ingestion

External knowledge into context



multiple files

Diff file formats

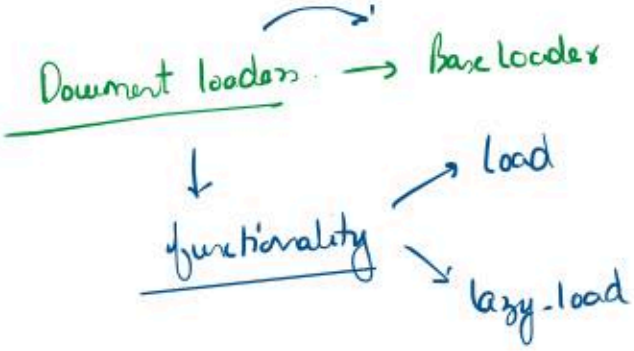
Specialized →

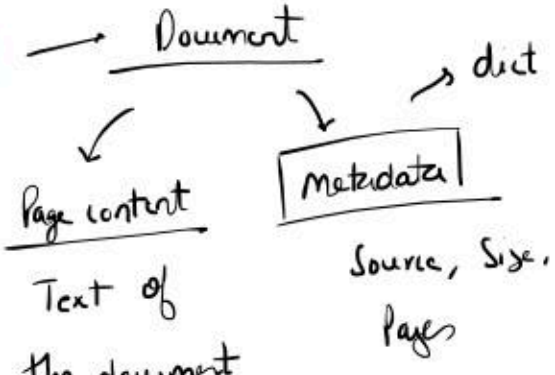
3 tasks

① load document into memory

② Parse the document → Parser

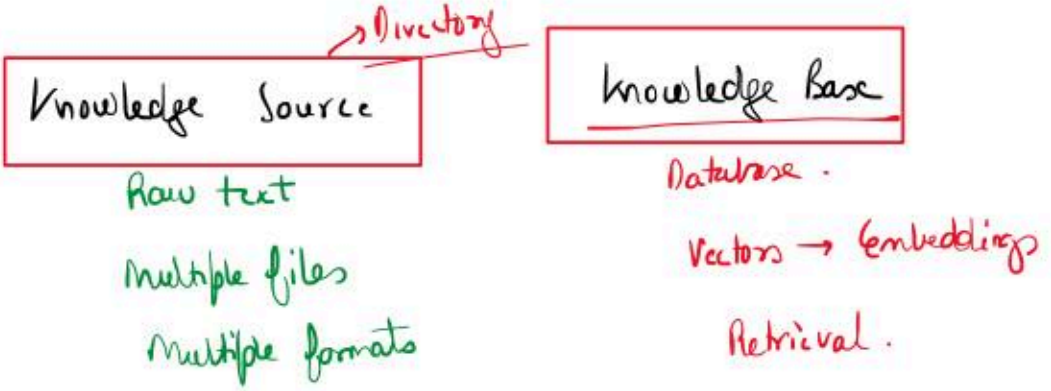
③ Return output in a unified format → Document



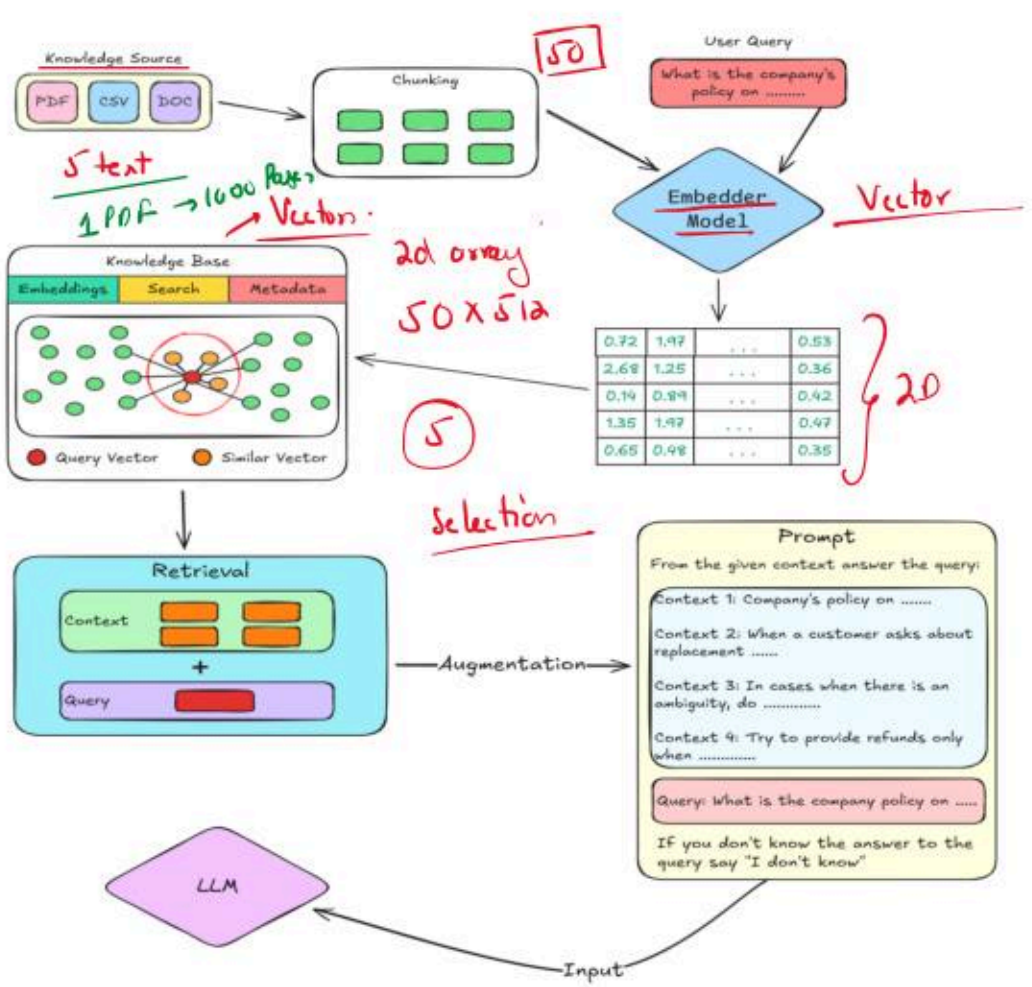


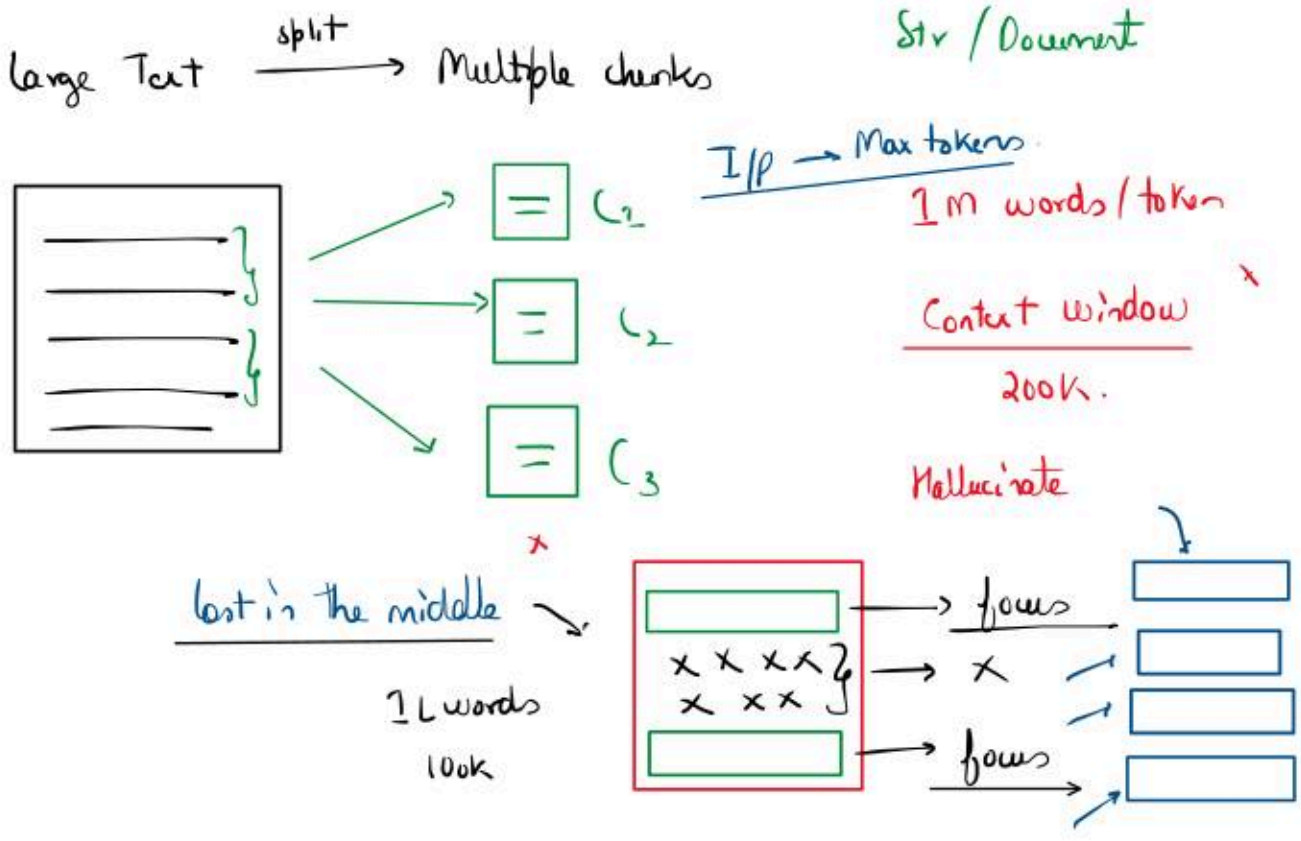
Text splitter → Base splitter




Directory
Knowledge Source
Raw text
Multiple files
Multiple formats
Knowledge Base
Database.
Vectors → Embeddings
Retrieval.

Each document → 10 parts Nd → Embedding model





Splitting → ① Never breach the context limit

→ ② Avoid the lost in the middle problem

Downstream Process

1000 lines → 50 → 100 lines

5000 lines → 500 → 10x

Knowledge source → 5 text files (knowledge)

↓

Text loader → 5 Document Objects (text document)

Context

D1
:
D5

5

Order

Return

Not specific to query

PDF document → PyPDF → 1000 documents → Embedding → 1000 x 512

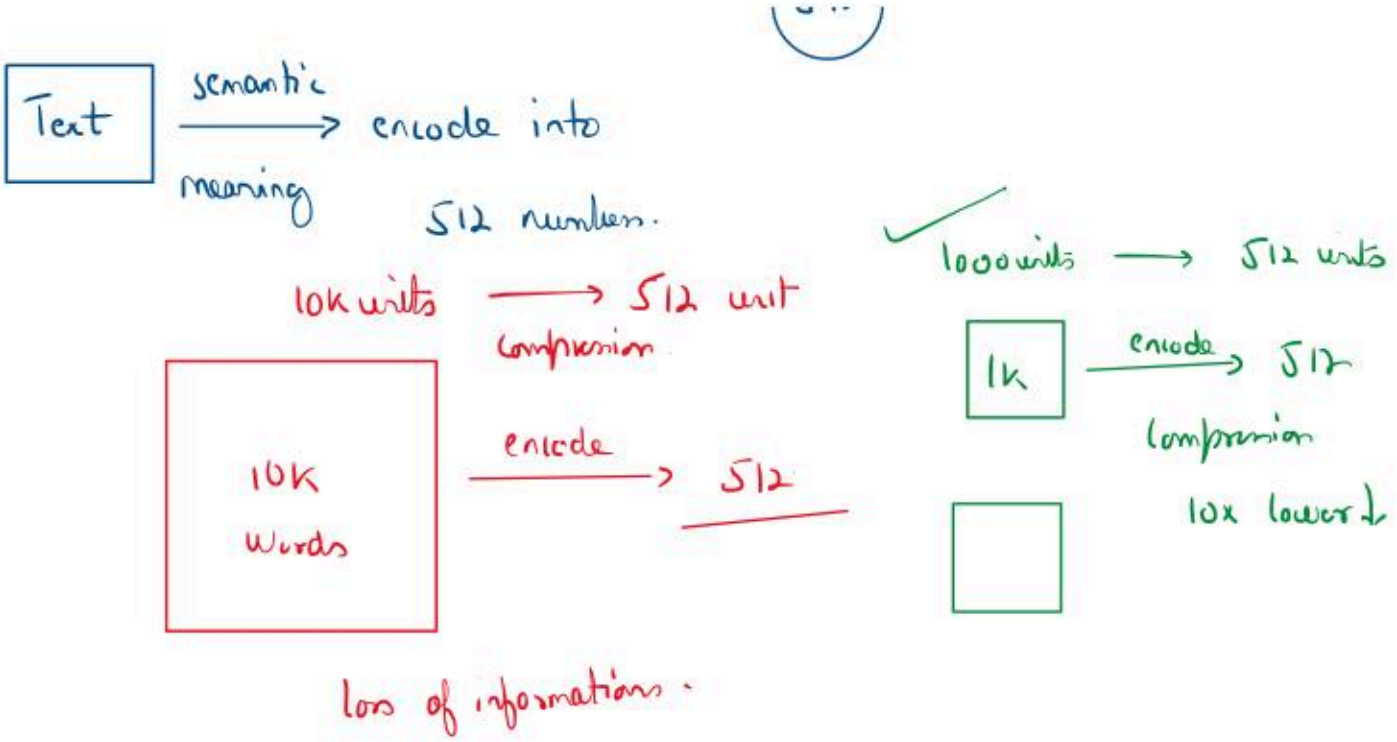
Page → Same meaning

Semantic meaning chunking ● Semantic Meaning

Embedding models → fixed Output dimensionality

512

semantic



(3) embedding models → 1 vector

20 parts → parallely

Batches

Text Splitters.

- split text
- split documents
- create documents

Split the given text

I/P → O/P

split text — I/P: str — O/P: list[str]

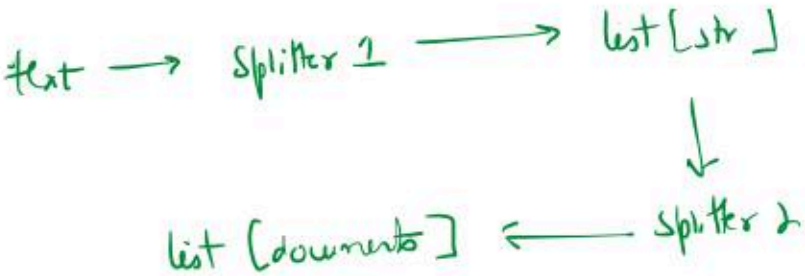
split documents — I/P: list[Doc] — O/P: list[Documents]

create documents — I/P: list(str) — O/P: list[documents]

1000 words → 10 chunks

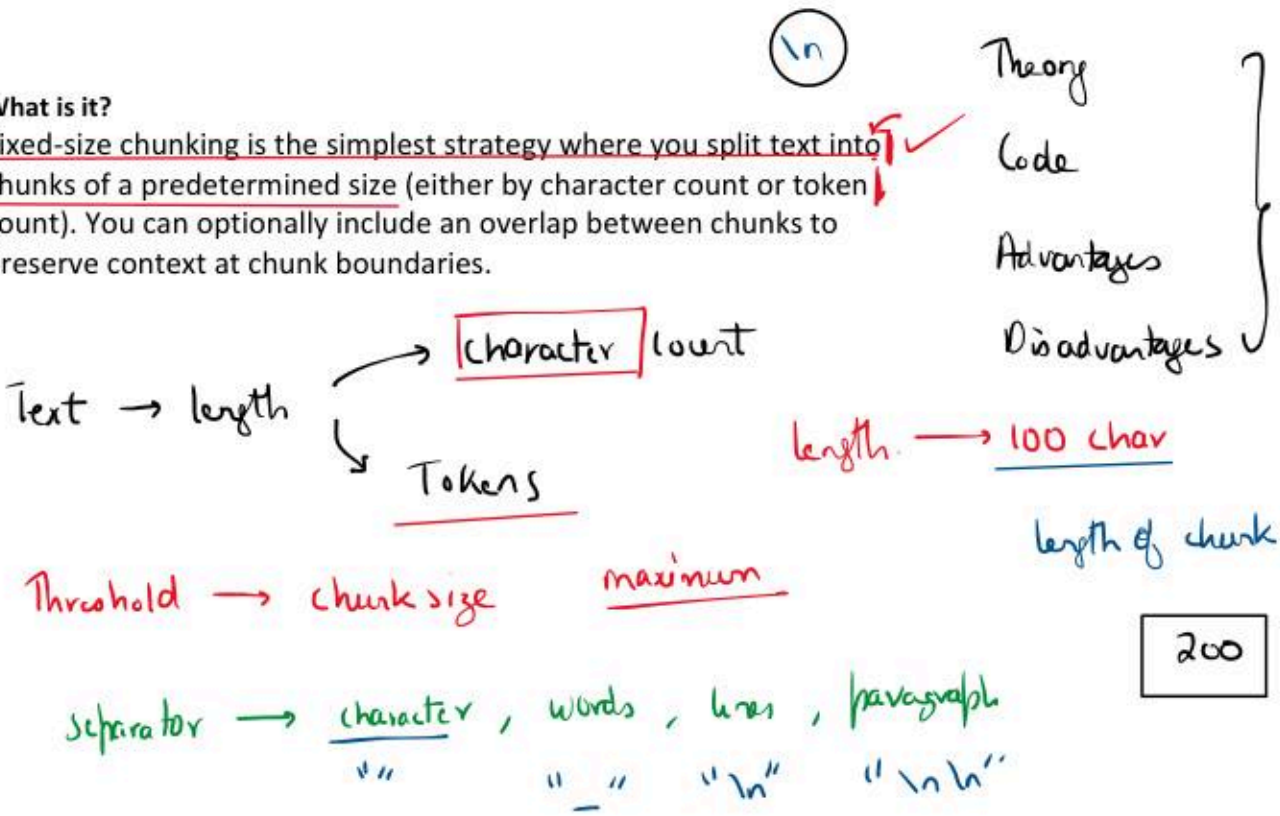
100 pages → 100 docs → 10 parts → 500 doc

text → Splitter 1 → list[str]


text
Splitter 1
list [str]
Splitter 2
list [documents]

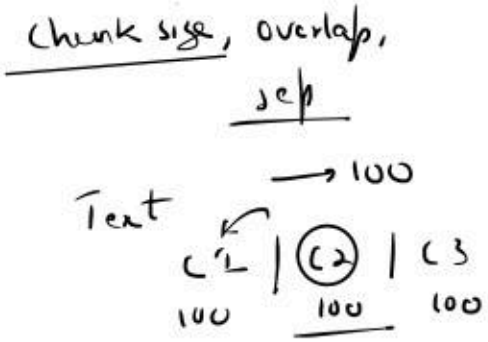
What is it?

Fixed-size chunking is the simplest strategy where you split text into chunks of a predetermined size (either by character count or token count). You can optionally include an overlap between chunks to preserve context at chunk boundaries.



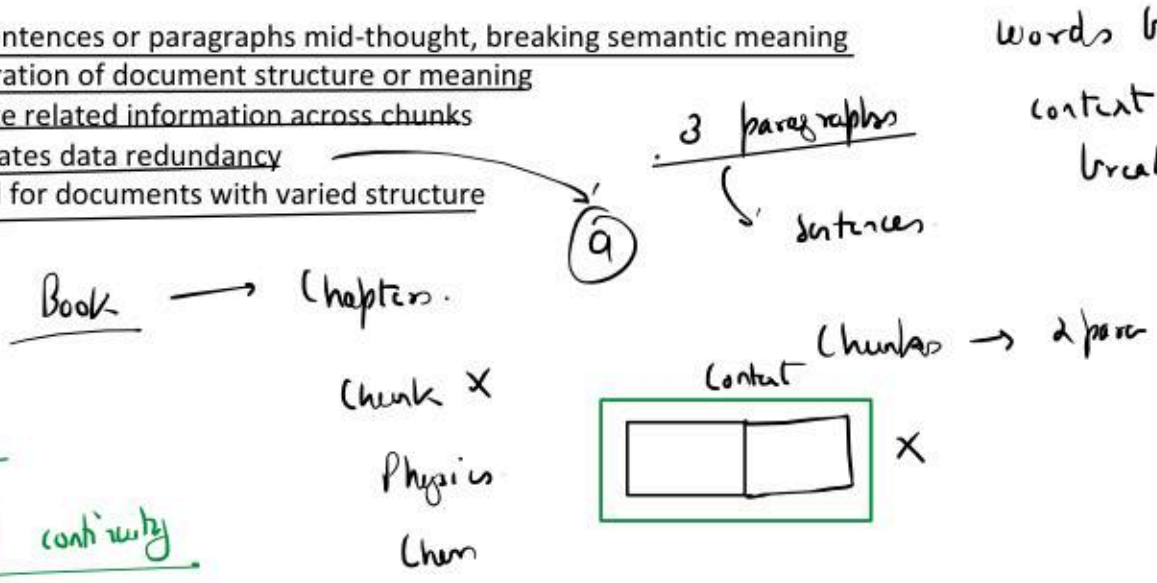
Advantages:

- Simple and straightforward to implement
- Predictable chunk sizes fit model context windows
- Fast processing speed
- Consistent memory usage
- Easy to parallelize



Disadvantages:

- May split sentences or paragraphs mid-thought, breaking semantic meaning
- No consideration of document structure or meaning
- Can separate related information across chunks
- Overlap creates data redundancy
- Not optimal for documents with varied structure



Text Structure Based Splitting

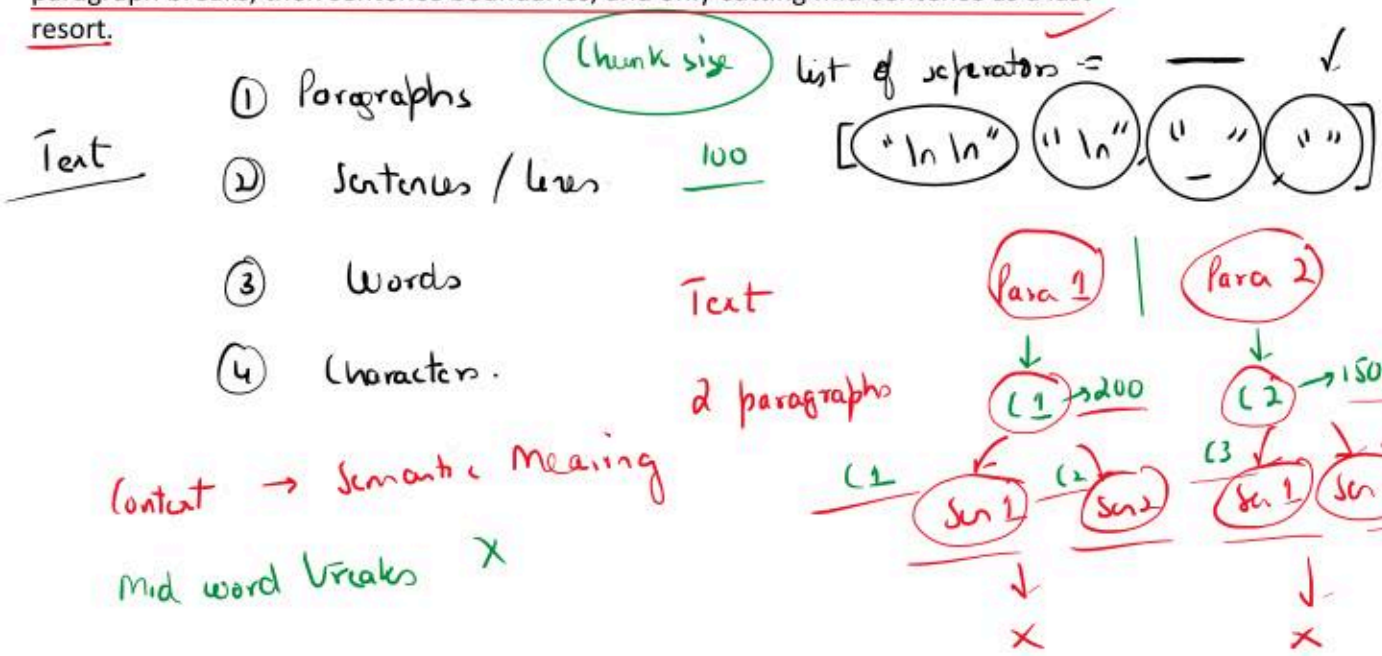
① mid sentence, mid work

② Document Structure

What is it?

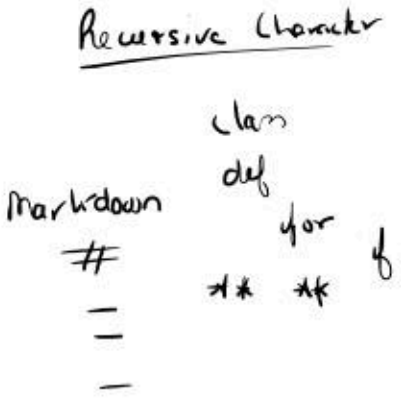
Recursive chunking is the **most recommended strategy** in LangChain. It attempts to split text hierarchically using a list of separators, starting with larger units (paragraphs) and recursively moving to smaller units (sentences, then words) if chunks are still too large.

Think of it as a smart scissors that tries to cut at natural boundaries - first trying paragraph breaks, then sentence boundaries, and only cutting mid-sentence as a last resort.



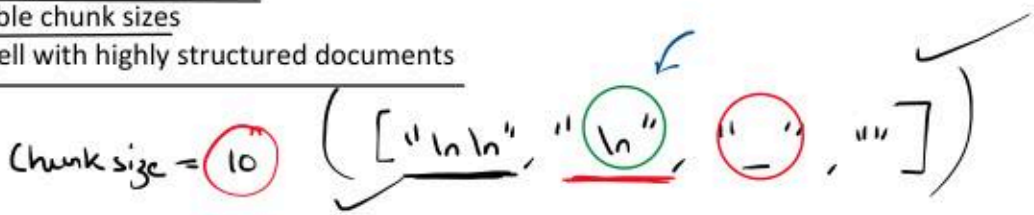
Advantages:

- Best balance between chunk size and semantic coherence
- Preserves natural language structure
- Respects document hierarchy (paragraphs → sentences → words)
- Recommended for most use cases
- Configurable separators for different document types
- Maintains context better than fixed-size chunking

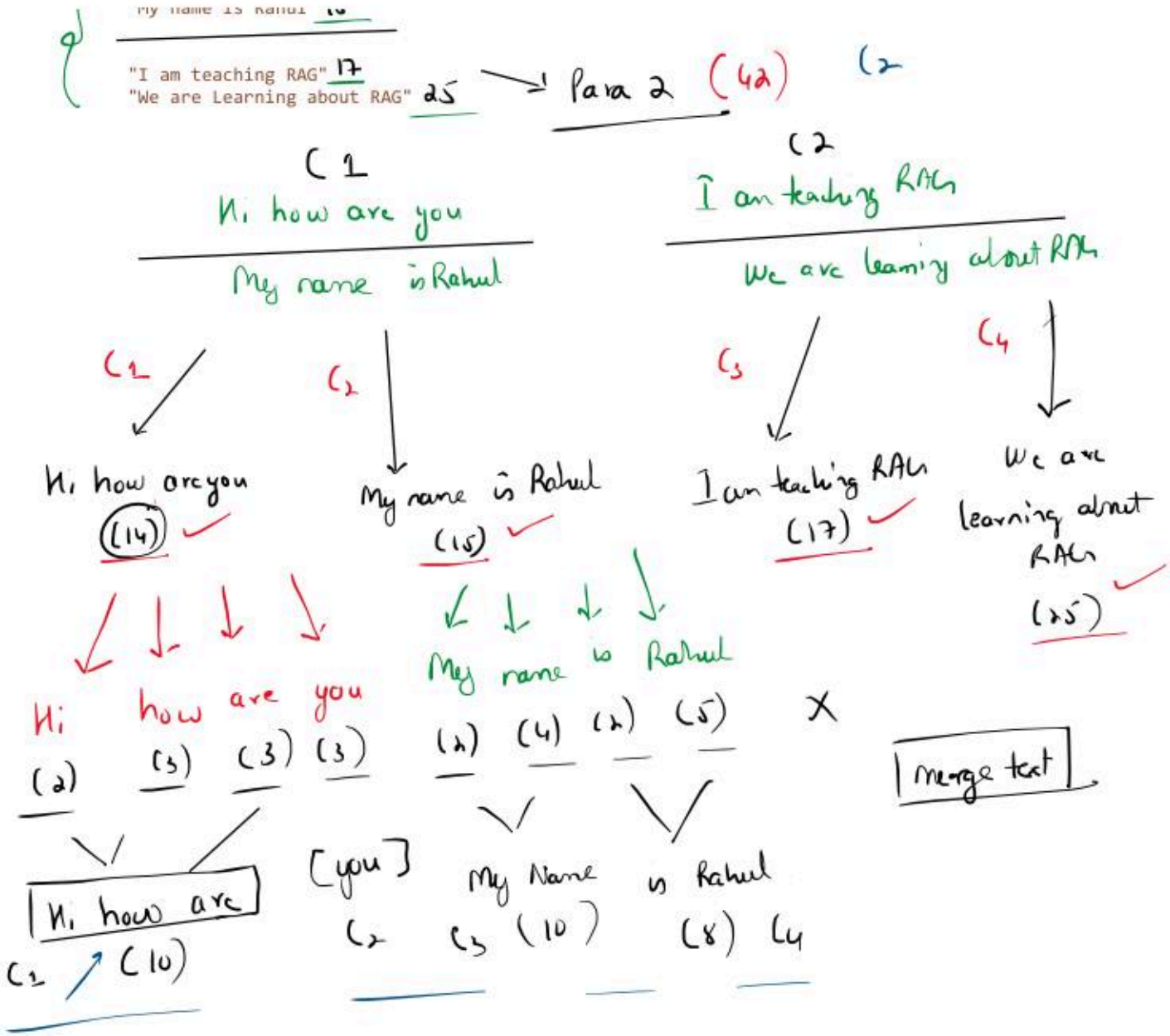


Disadvantages:

- More complex than simple fixed-size splitting
- May still break semantic units if they're too long
- Requires careful separator selection
- Can create variable chunk sizes
- May not work well with highly structured documents






My name is Rahul 16
"I am teaching RAG" 17
"We are Learning about RAG" 25
Para 2 (42)
C1
Hi how are you
My name is Rahul
C2
I am teaching RAG
We are learning about RAG
C1
C2
C3
C4
Hi how are you
(14)
My name is Rahul
(15)
I am teaching RAG
(17)
We are learning about RAG
(25)
Hi how are you
(2) (3) (3) (3)
My name is Rahul
(2) (4) (2) (5)
X
merge text
Hi how are
C1 (10)
[you]
C2
My Name is Rahul
C3 (10) (8) C4

What is it?

This strategy leverages the inherent structure of documents (HTML tags, Markdown headers, JSON objects, code functions) to create meaningful chunks. Each chunk respects the logical organization of the document.

It's like organizing a book by chapters and sections rather than just cutting it every 1000 words.

Advantages:

- Preserves logical document organization
- Maintains semantic meaning within chunks
- Metadata-rich chunks (headers, section info)
- Excellent for structured documents
- Better retrieval accuracy
- Natural chunk boundaries

Disadvantages:

- Only works with structured documents
- Requires document-specific splitters
- May create very large chunks if sections are long
- More complex to implement
- Not suitable for unstructured text
- May need combination with other strategies

Semantic Similarity Based Splitting

What is it?

Semantic chunking creates chunks based on the meaning and semantic similarity of the text rather than arbitrary size limits. It starts with an initial chunk and keeps adding sentences as long as they remain semantically similar (measured by cosine similarity of embeddings). When similarity drops below a threshold, a new chunk begins.

This is like grouping related topics in a conversation - you keep talking about one subject until the topic naturally shifts.

Chunk size ↑

→ Chunk Chunks

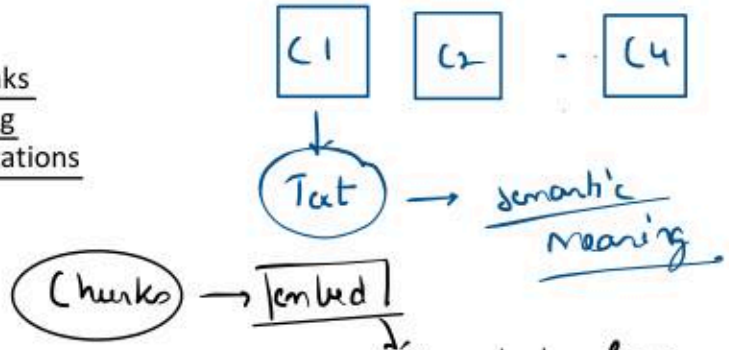
Similarity

Semantic

Meaning of text

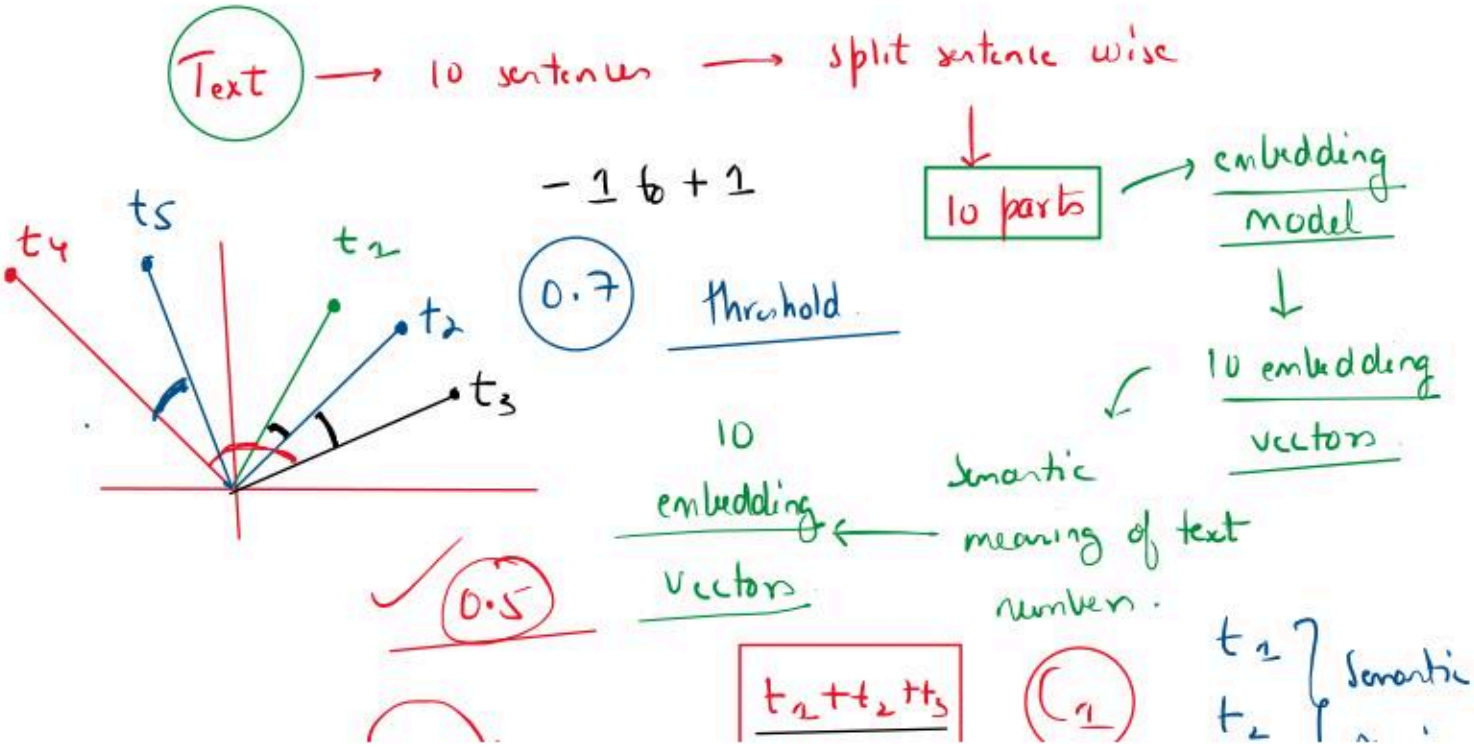
Advantages:

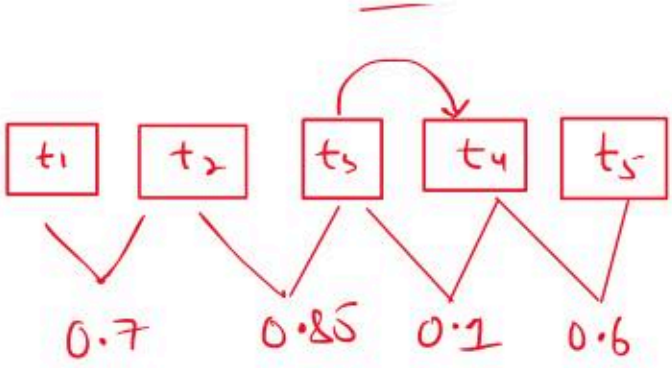
- Preserves semantic coherence within chunks
- More intelligent splitting based on meaning
- Better retrieval performance in RAG applications
- Natural topic boundaries
- Improved context preservation



Disadvantages:

- Computationally expensive (requires embeddings for every sentence)
- Variable chunk sizes can be problematic
- Slower processing time
- Requires an embedding model
- May create very large or very small chunks
- Difficult to predict chunk count in advance





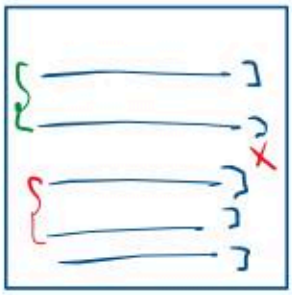
$t_1 + t_2 + t_3$ C_1

$t_4 + t_5$ C_2

$t_6 + t_7 + t_8 + t_9$ C_3

t_{10} C_4

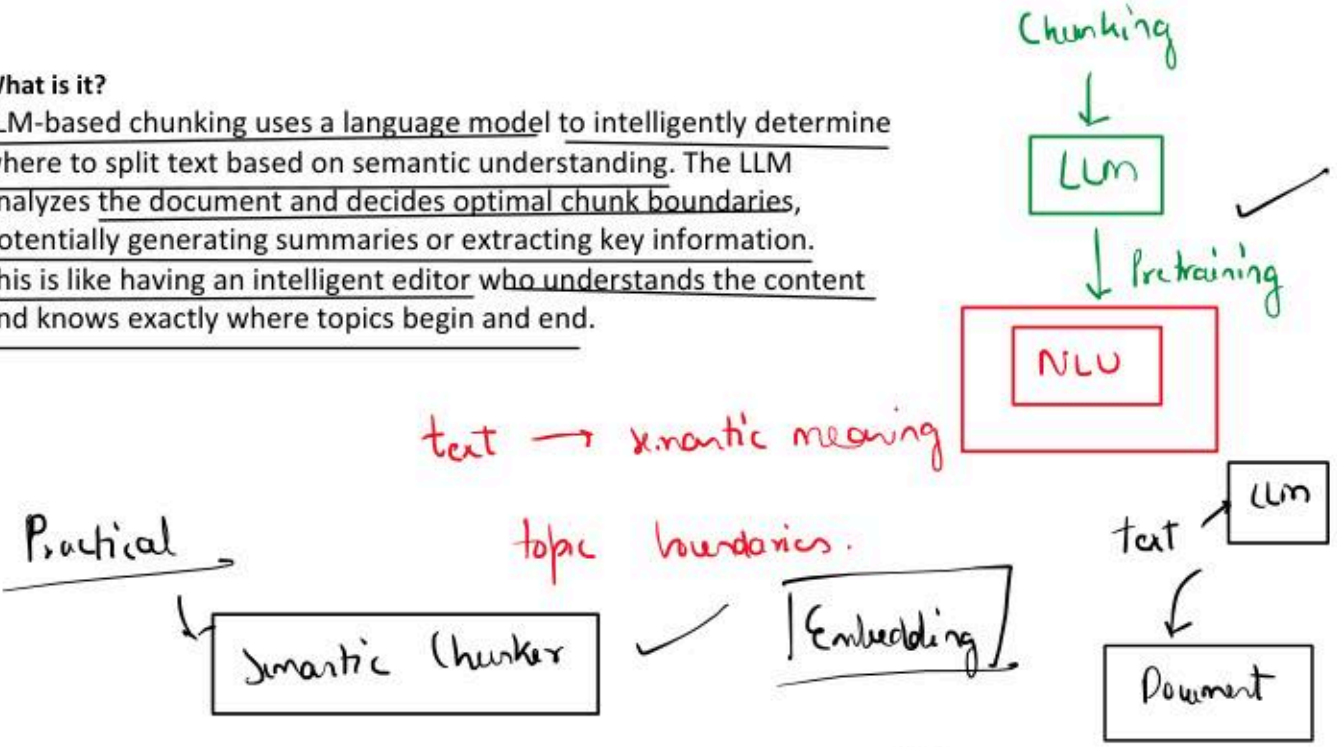
t_1 t_2 t_3 } Semantic Meaning



Semantic similarity

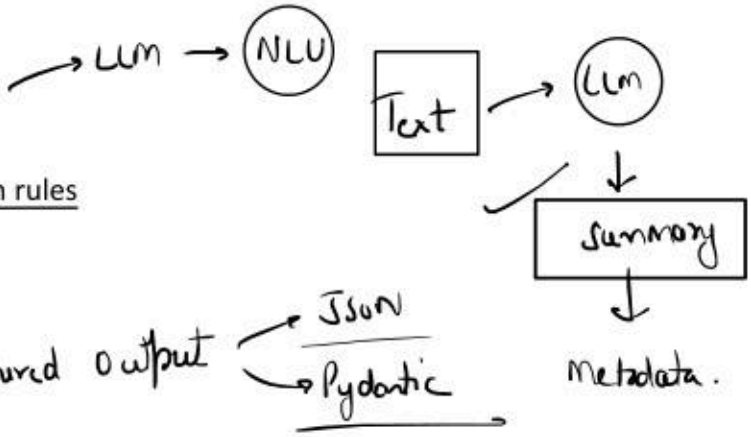
What is it?

LLM-based chunking uses a language model to intelligently determine where to split text based on semantic understanding. The LLM analyzes the document and decides optimal chunk boundaries, potentially generating summaries or extracting key information. This is like having an intelligent editor who understands the content and knows exactly where topics begin and end.



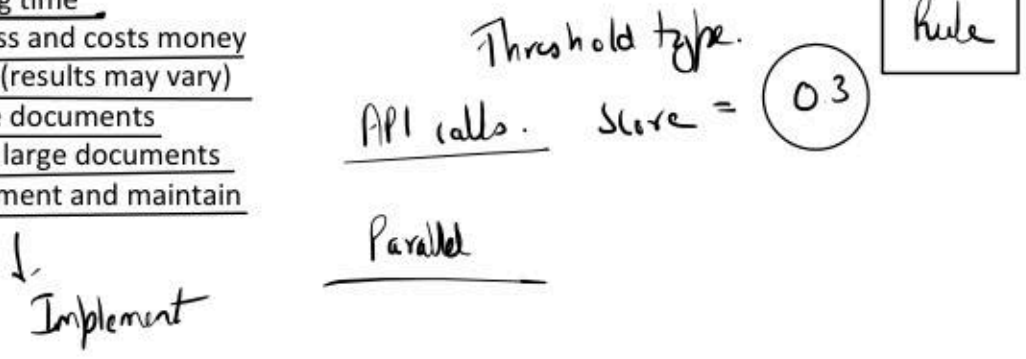
Advantages:

- Most intelligent and context-aware splitting
- Can add contextual summaries to chunks
- Understands semantic boundaries better than rules
- Can adapt to different document types
- Improves retrieval quality significantly
- Can extract key information



Disadvantages:

- Very expensive (LLM API calls for every chunk)
- Slowest processing time
- Requires API access and costs money
- Not deterministic (results may vary)
- Overkill for simple documents
- Latency issues for large documents
- Complex to implement and maintain



Text Embeddings

Embedding models → No③

Knowledge source.

External knowledge → Data ingestion

Document loaders → Unified format → Document

Document → Page content, Metadata

RAG → Why, What, How

Text splitter. → Documents → Chunks

5 Chunking Strategies

① Charater length Based Splitting → Text → chunk size → character count

Contextual continuity

Chunk

② Text Structure Based Splitting → Text → Paragraphs, Lines, Words, Characters

③ Document structure based splitting → Md, HTML, JSON, Code files, Python. Class.

Separators.

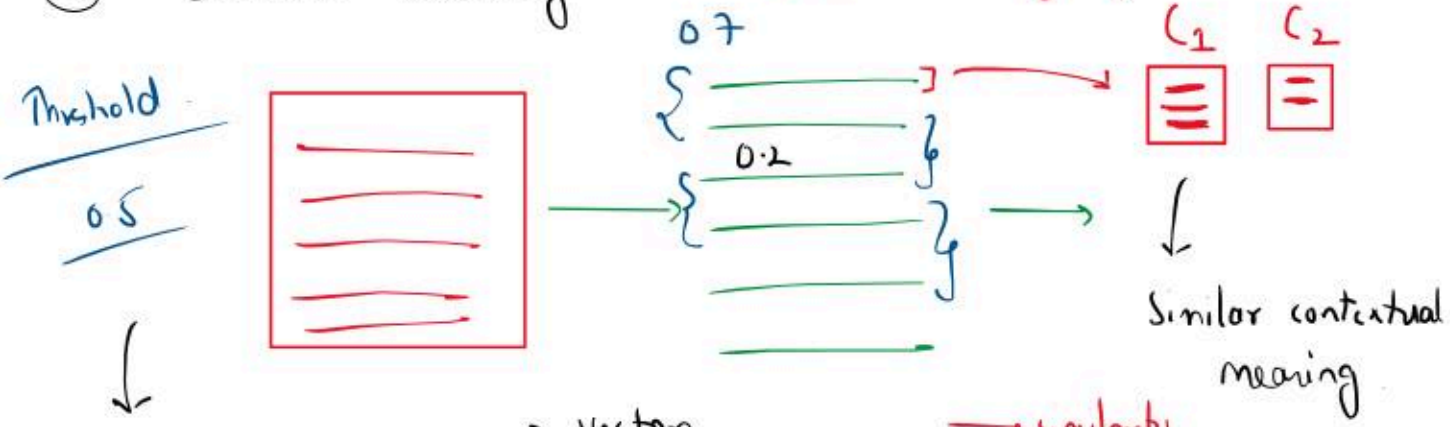
Separators:

Python. Class.

\n\n def

Headers

(4) Semantic Chunking → Semantic meaning of text



(5) LLM Based Chunking → LLM ← Text → Chunking → structured output

NLU

Semantic meaning change.

Pydantic model (schema)

[Document] → [chunk1, chunk2 ....]

Document

Page content → Text

Metdata. → summary of Chunk.

[Documents] → Chunktext → Structured


[Documents]
Chunk text
Structured
Output
Summary → Metadata.

Categorical CoL → (encode) → Numbers → OHE / Ordinal

Text X → Similarities → Numbers.

ML X

Text → Vectors

Stage 1 :- Classical Methods (BOW, TF-IDF) → Sparse Vectors

Count the frequency of words in your text

English lang → 10k

Vocabulary → Unique set of words

	Cat	Sat	Mat	Rat	Wall	Ball
Text 1	1	1	1	0	0	0
Text 2	2	0	3	2	0	1

Cat Sat on the mat

5 ✓

Sparse vectors → Most of the values are zero.

1 x 10000 → Non zero → 10k

Similarity → calculations (Element wise)

Stage 2 → Word Embeddings (Dense vectors)

Word2Vec GloVe

DL

768

Information

Dense Vectors.

Number of calculations ↓

① Reduced Dimensionality

② Each features holds info

768

Semantic Meaning

Word by word

Location

768

River Bank

E1 E2

Contextual Meaning

Money Bank → institution

E3 E2

Query → Chunk ✗

login issue in app

Password Recovery

Login

Username

Stage 3 → Contextual Embeddings. (Dense, contextual meaning)

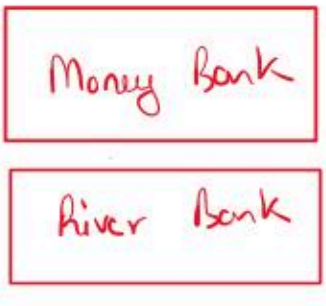
Context – aware embeddings.

BERT,

OpenAI,

Gemma Embeddings,

Sentence Transformers.



$\rightarrow E_1$

E_2

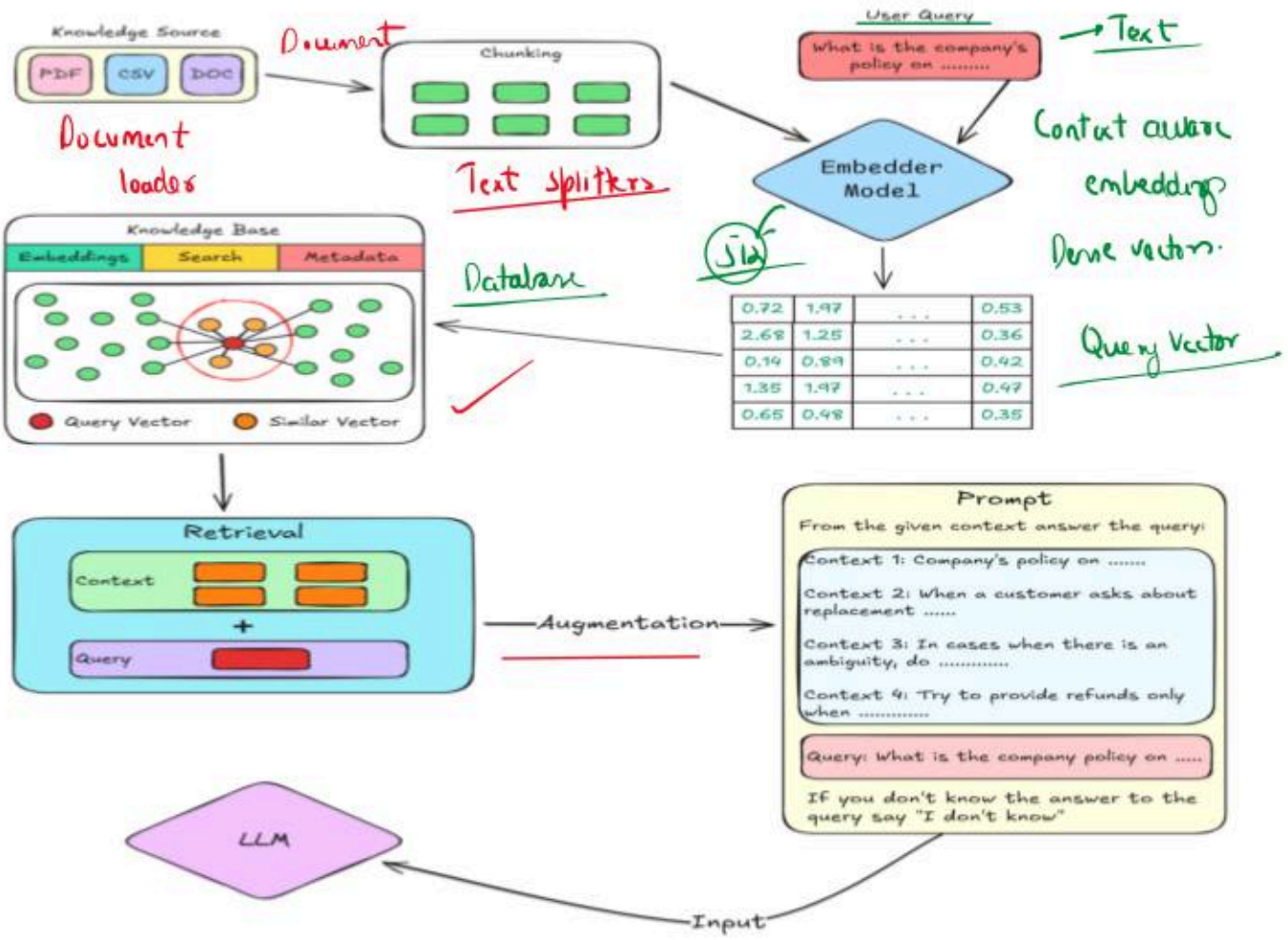
Sentences

Paragraphs

↓

numbers.

The Big Picture



Text

Current Embedding models → Calculations

Similarities

Content aware embeddings

Dense Vector

512

Database

ND → 512

Vectors → ND → 512

Embedding Space

Vectors → 512

Vectors

↓

Contextual meaning

Hyperspace

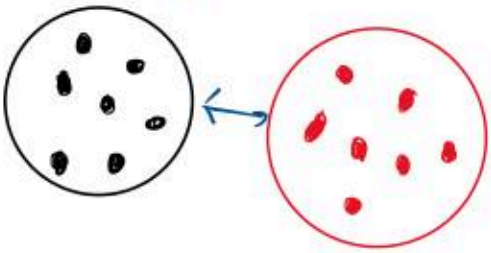
functions OOPS

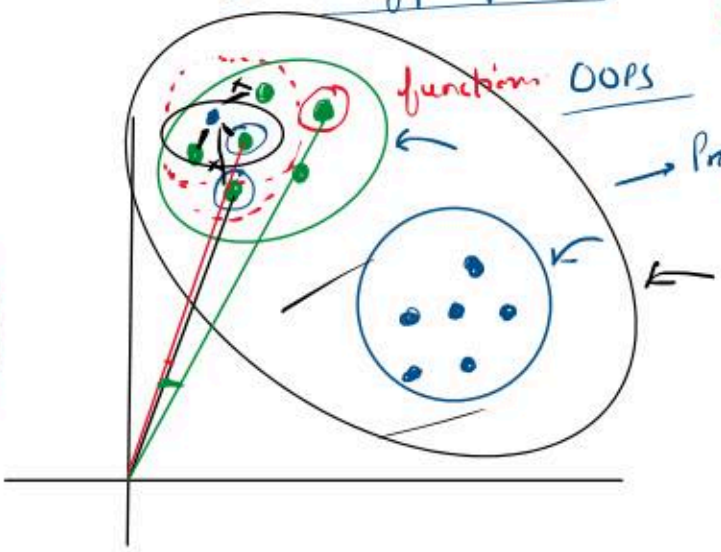
Python

Programming languages

JS

Farming





Contextual meaning similar

Embedding Space → dimensionality = dimensionality of vector

Directional meaning → Relationships:

Area ↑ → Broader Perspective (Programming language)

Area ↓ → Narrower Perspective (Python → Topics)

Query (Text) → Embedding model

↓

Dense, content aware vector → Hyperspace Embedding space

OOPs → Relevant documents

① Euclidean Distance → KNN

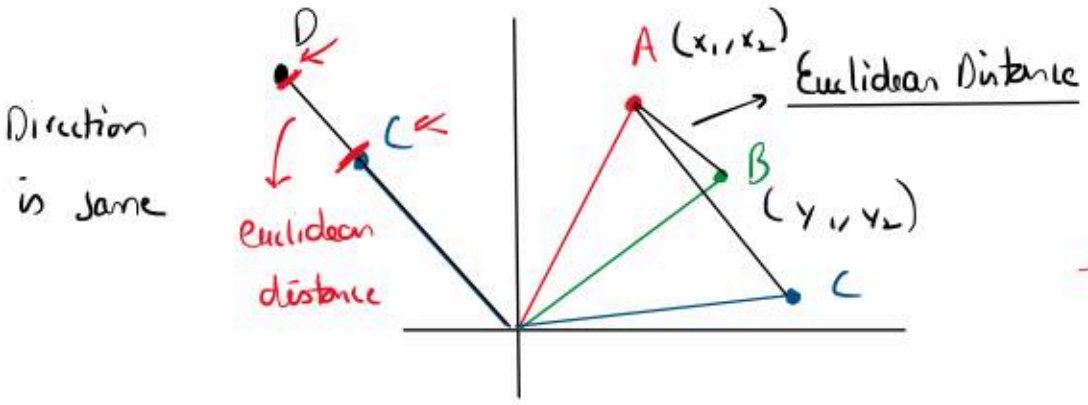
② Cosine Similarity

③ Dot Product

} Distance Metrics.

1) Euclidean distance

Straight line distance b/w two points / vectors.



A = (2, 4)

B = (3, 2)

128, 256

$$d = \sqrt{(x_1-y_1)^2 + (x_2-y_2)^2 \dots (x_n-y_n)^2}$$

$$= \sqrt{(2-3)^2 + (4-2)^2} = \sqrt{(1)^2 + (2)^2}$$

$$d = \sqrt{5} = \boxed{2.23}$$

euclidean distances ↓

lower the distance

More similar the vectors

Sensitive to your number of dimensions.

sensitive to both direction of magnitude.

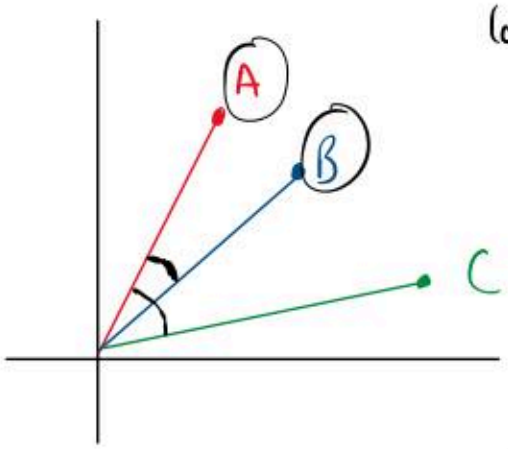
② Cosine similarity → Bounded Metric

Range
↓ lower Range: −1
↑ upper Range: +1

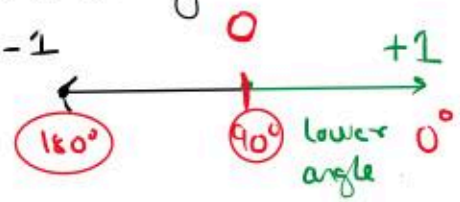
Cosine of angle b/w vectors

Angle b/w the vectors.

lower the angle
More similar the vectors.



Cosine similarity

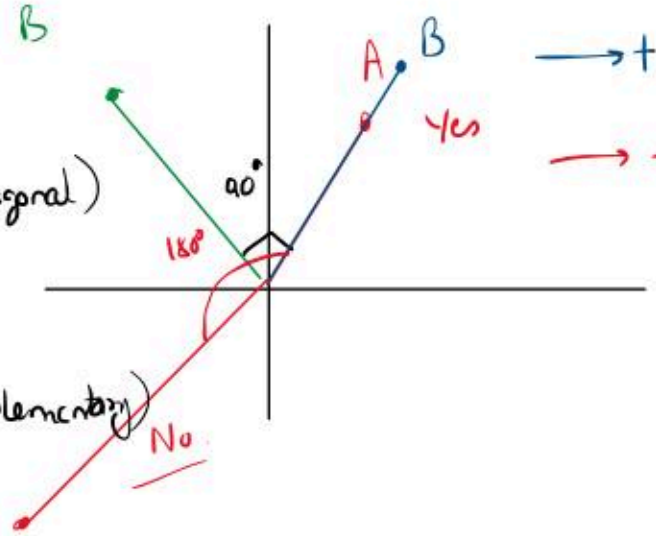


Cosine Similarity.

+1 → 0°
Most similar

0 → 90° (orthogonal)
Dissimilar

−1 → 180° (complementary)
Completely opposite.



→ 0 → not similar

→ +1 → Most similar

→ −1 → complementary

→ Unbounded

similarity = $\frac{A \cdot B}{\|A\| \times \|B\|}$

$A \cdot B$ → Dot Product ↓ Similarity

$\|A\| \times \|B\|$ → Product of magnitude of two vectors.

$\|A\|$ → Magnitude of A

$\|B\|$ → Magnitude of B

Normalization ↓ Bounded in

nature.

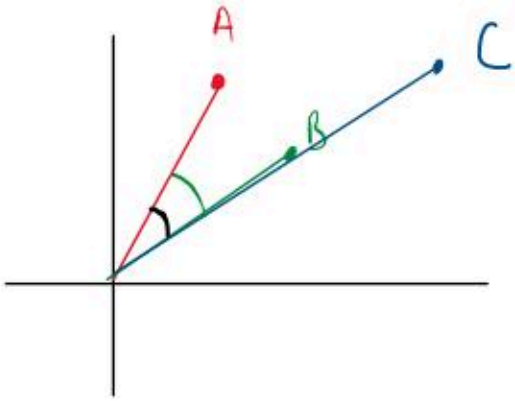
sensitive to only direction of vectors.

Magnitude ✗

Direction

↓

Angle b/w the vectors.



C > B

A → B = 0.7

A → C = 0.7

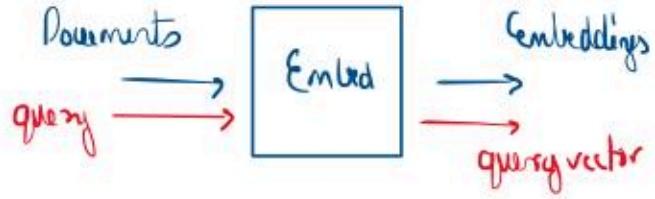
Angles / Direction → Embedding models → Embedding space.

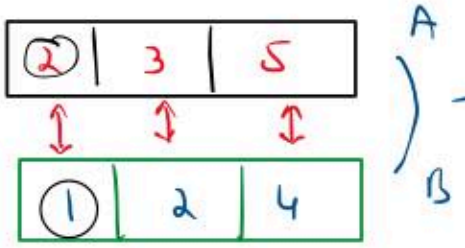
Accurate result

↓

vectors. → Output dimensions have least impact on sim.

3) Dot Product → Unbounded





→ dimensionality should be same

- +	+ +
- -	+ -

Sign of element

dot = (2 × 1) + (3 × 2) + (5 × 4)

= 2 + 6 + 20 = 28 → scalar

Unbounded

dot product = $(x_1 \times y_1) + (x_2 \times y_2) + \ldots (x_n \times y_n)$

Dot Product ↑ ⟶ More similar the Vectors.

Magnitude, direction

Dot Product → Vectors. ⟶ Normalized = 1

Cosine Similarity = $\frac{A \cdot B}{\|A\| \times \|B\|}$ → Dot Product

1) Numerator
2) Magnitude A, B
3) Division

$= \frac{A \cdot B}{1 \times 1} = A \cdot B$

Much faster ⟶ Normalized vectors.

Embedding models ⟶ Normalized embeddings.

Latency

faster retrieval

Stores.

cosine sim = dot product

Dot Product / Cosine Similarity

Bounded score → −1 to +1

Dimensionality of output ✓

Dot Product

Similarity → Calculations.

Vector → Element wise calculation.

No. of elements → Dimensionality of embeddings

Text → O/P → Embedding Model → O/P → Vector

Embedding → Fixed Dimensionality.

Vector → Numbers, Semantic Meaning

Common dimensions → 384, 768, 1024, 1536, 3072

Custom Dimensions → 512, 1000

Query → Query Vector

Similar Documents → Context → Prompt → LLM → Grounded

Dimension Represent

Learned feature captures some aspect of your text

Text → Tech, Robotics, Army → Defense → Embeddings → Dimensions

encode

encode

Dimension → 768 →

D_{50} → tech related 0.9

D_{80} → Robotics 0.95

D_{150} → Peace loving 0.1

Semantic Meaning

Embeddings → Minor Detail → Features

Name Age Address Contact Email id

Rows of data.

P1 ____ ____

P2

P3

Text → Meaning → Parts 1 → Number

→ Parts 2

→ Parts 3

DL

Train

1000 words → 768

1000 words → 1024

Dimensionality ↑

768

1024 ↑

Dimensionality ↓

Features

Nuanced semantic relationships.

X units

↓

1 unit ↑

4 Trade Off areas :-

① Semantic Richness / More Nuanced Meaning :-

Leave Policy.

Leave Policy.

Dimension ↑ Dimensions ↓.

CL
SL
CCL

↓ vector

① Nuanced semantic Relationships

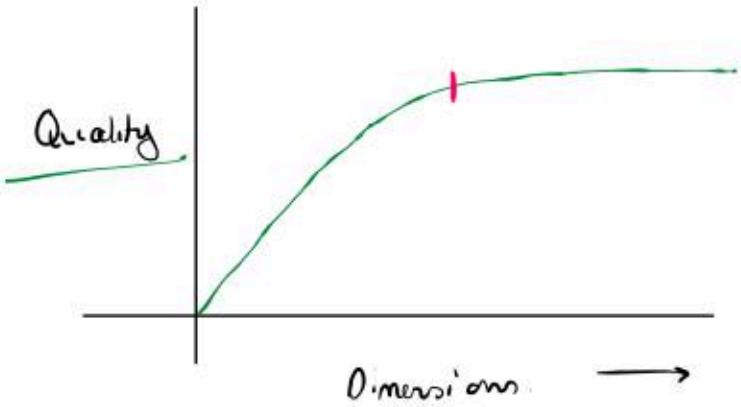
② Better diff, small differences in meaning have

③ Retrieval will be better ← Document → SL

Python

Makes no difference

Healthcare.



② Storage cost

Query
Documents
} Embeddings.

Knowledge Base (Database)

numbers → Floats → 32 bit → 4 bytes

↘ 64 bit

384 Dimensions

1536*4=6144

384

1536/384=4

1536 Bytes

384*4=1,536

384*4=1,536

1536 → 1.5 K

Knowledge Base
= 2 million vectors.

kB x 1 mB = GB

1.5*2=3

3 GB

3 GB

Database → Cloud

368 → 3 GB
1536 → 12 GB
} 4 times

→ Network cost
→ Storage cost

Dimensionality ↑ → Storage cost ↑, Network cost ↑

③ Computational Speed → Similarity → Cosine Sim, Dot Product } Element wise

↓

Scores → retrieval

↓

Latency will also increase

Compounding effect

1000 docs → 384 dimensions

↓

200 ms

1000 docs → 1536 Dimensions

↓

800 ms.

4) Model Quality and Availability

Output Dimensions → Training

Output Dimensions → Training Architecture.

ill-trained Architecture → 1536 Dimensions. → Features → Bad Quality

Trained Architecture → Features → Better nuanced relationship.

768

1536

Garbage In Garbage out

Proprietary Models vs Open Source Models

Theory → Proprietary
Code ↳ Open Source

Proprietary (Closed-Source) Embedding Models

These are commercial models accessed via API. You send text, they return embeddings. You don't have access to the model weights or training details.

Trade Secret

Download ✗
Weights ✗

✅ **Pros:**

- **No infrastructure needed:** Just API calls, no GPU servers to manage
- **State-of-the-art performance:** Usually trained on massive proprietary datasets
- **Regular updates:** Models improve over time without you doing anything
- **Simple scaling:** Handle millions of requests without worrying about capacity
- **Support and documentation:** Professional support teams and comprehensive docs

Hardware ✗
SOTA
API

❌ **Cons:**

- **Recurring costs:** Pay per embedding generated (can get expensive at scale)
- **API dependency:** Your system depends on external service uptime
- **Data privacy:** Your text is sent to external servers (concern for sensitive data)
- **Vendor lock-in:** Migrating to another provider means re-embedding everything
- **Rate limits:** May have throughput limitations
- **No customization:** Can't fine-tune on your specific domain

Python
model = X
= Y
5 $
10 $
RAG
↓
Production

Proprietary → Closed Source → Download ✗

Downtime

Parameters
Architecture
Training Procedure
} X

HTTP requests

API

Hardware

Avoid misuse

request
response

(Text → query)
(query vector)

(Documents → chunks)
(Document embedding

Scaling, Training, Update

API lost

1000 /min

2000/min ← Rate limits

Open Source Embedding Models

These are models you can download and run yourself. You have complete access to model weights.

✅ **Pros:**

Models → Embeddings → Compute

Open Source

- **No per-request costs:** Only infrastructure costs (but can be significant)
- **Data privacy:** Everything stays on your servers
- **Full control:** Can fine-tune on your domain-specific data

- **No per-request costs:** Only infrastructure costs (but can be significant)
- **Data privacy:** Everything stays on your servers
- **Full control:** Can fine-tune on your domain-specific data
- **No rate limits:** Process as fast as your hardware allows
- **Offline operation:** No internet dependency
- **No vendor lock-in:** You own the model

Open source

✓ Code → Public

✓ weights → training

Inference → compute

Model → Download

↓

Run

✗ **Cons:**

- **Infrastructure required:** Need GPU servers (expensive to set up and maintain)
- **DevOps overhead:** Managing deployments, scaling, monitoring
- **Performance gap:** Usually slightly behind state-of-the-art proprietary models
- **Manual updates:** You need to handle model upgrades
- **Initial learning curve:** More complex setup

1536, 3072 768 384 Self Hosted

PL Initial cost ✗ APIs Bypass

API cost ✗

Direct Access → Embeddings

Hardware cost GPUs. → Cloud

Skilled Team → Model

Productions.

```
START
│
├─ Is data sensitive/regulated?
│  ├─ YES → Open-source models only
│  └─ NO → Continue
│
├─ Do you have GPU infrastructure or budget for it?
│  ├─ NO → Proprietary (API-based)
│  └─ YES → Continue to Step 3
│
├─ Is volume very high (>100M embeddings)?
│  ├─ YES → Open-source (cost-effective at scale)
│  └─ NO → Either works, evaluate based on other factors
│
└─ Need domain-specific fine-tuning?
   ├─ YES → Open-source (so you can fine-tune)
   └─ NO → Either works
```

Cloud

Hardware ↓

1000

Proprietary

Proprietary → 5$

Open Source. → ✓

→ Text embedding-3-large

Proprietary → Open AI Embedding → Text embedding -3 -large
                                → Text embedding -3 -small

query , documents

Open Source → Ollama → Ollama .
                     → Ollama - langchain

4 RAG Pipeline

Vector stores → knowledge Base

1) What vs are ?
2) Used for
3) Storage and Indexing
4) Searching

CRUD

① Document loaders → Knowledge Source → Text

↓

Parsers

↑

Multiple documents

Various file formats

a) Load the document into memory

b) Parse the document → Text extract

Document → page content → Text

↓ metadata → { } → ✓

② Text splitters → Documents → Chunks

↓

Documents

Page wise → 3 chunks.

→ Document

Page wise ⟶ 3 chunks.

10 ⟶ 30 chunks ⟶ Document

③ Embedding models ⟶ Vector Embeddings

⟶ Sparse representation

⟶ word embeddings ⟶ Static embeddings.

⟶ dynamic, contextual embeddings (embedding models)

Semantic meaning ⟶ number (Dense vector, high dimensional vectors)

User.

↓

query

↓ ↗ Vector

most similar documents

↓

fetch

30 chunks ⟶ 30 documents

↓

embedding model

↙

30 contextual embeddings } ⟶ Text semantic meaning inside numbers.

query ⟶ Embedding model ⟶ query vector ⟶ similarity search

documents (30) ⟶ Embedding model ⟶ document embedding

↗ ① Euclidean distance.

Calculations -

→ (1) Euclidean distance.

→ (2) Cosine similarity → Default

→ (1) Dot Product ✓

Dot Product

Normalize.

Embedding → Normalized embeddings.

X

Documents (30) → embedding model → 30 embeddings (1536)

Query 1 → embedding model → Query vector

E3, E5, E8

Query 2 → embedding model → Query vector.

E1, E23, E18

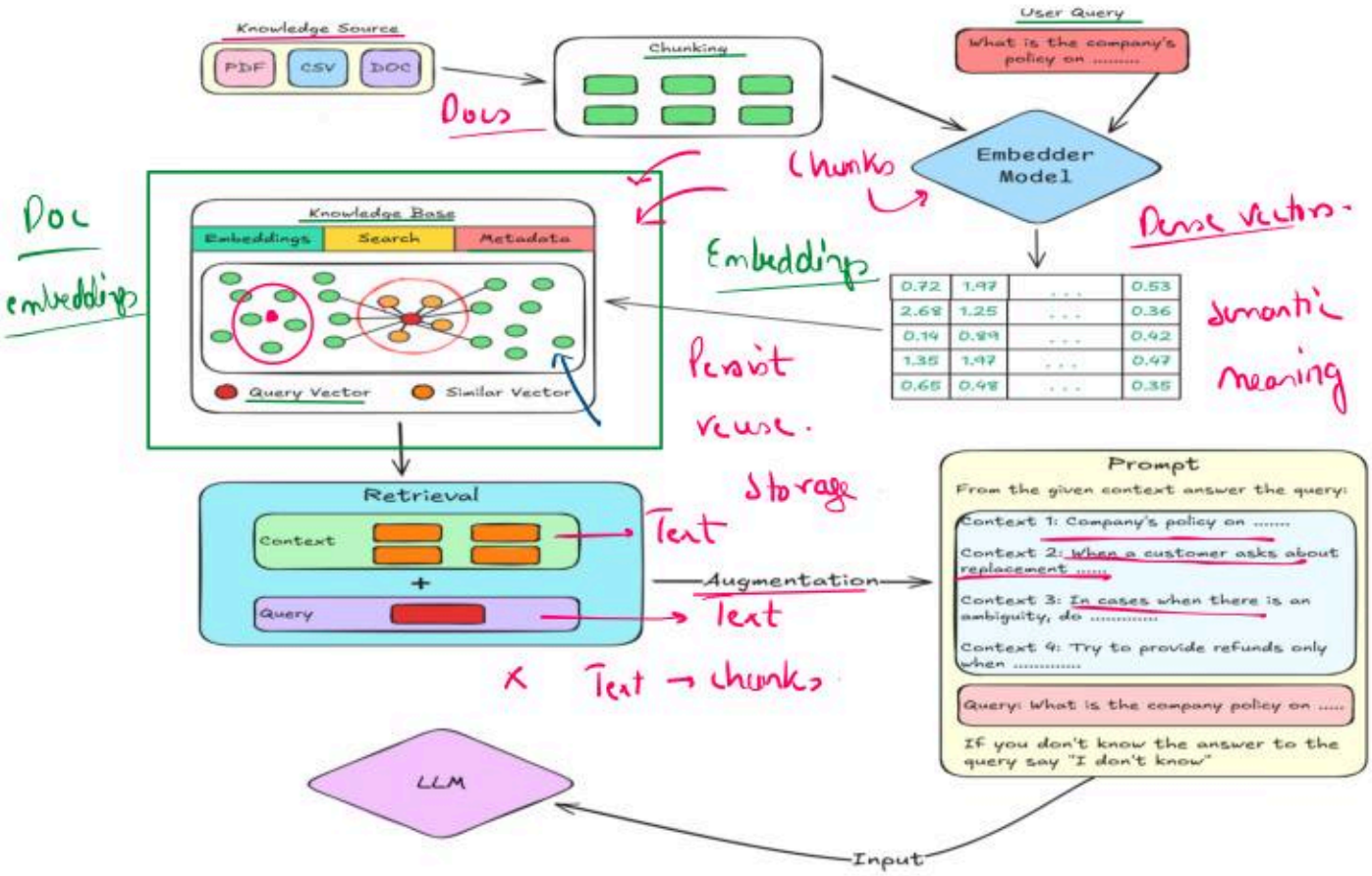
Embeddings → Store.

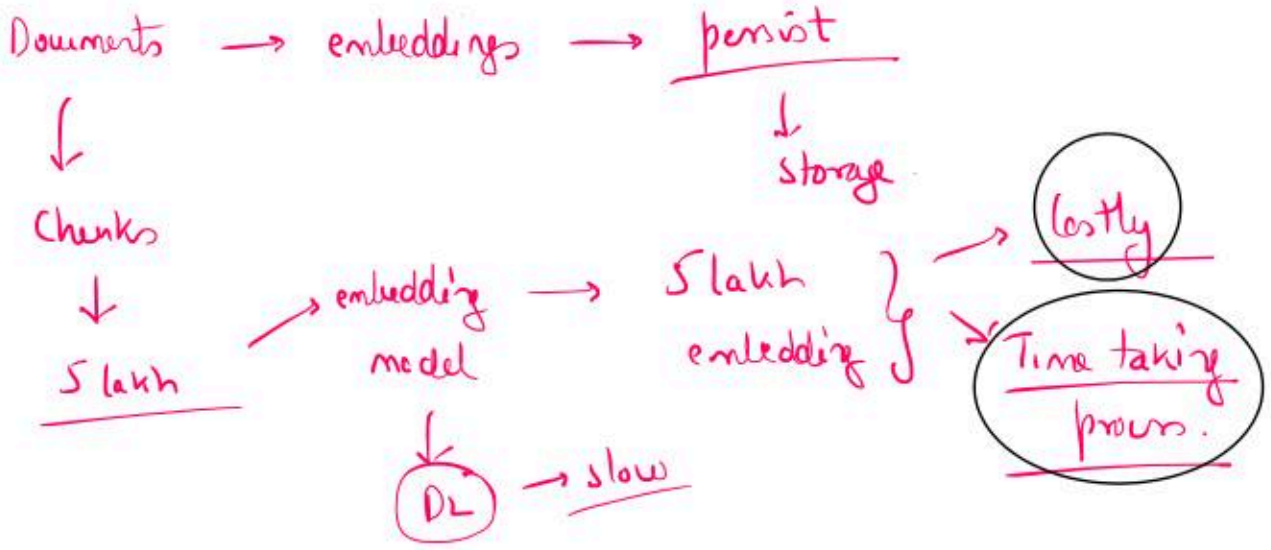
Vector Store → Store vectors.

Document embeddings → Store → vector store

Query vector → similarity search → Return similar docs

The Big Picture





Vector Stores. → CRUD

Create → Add

Update → Add new embeddings.

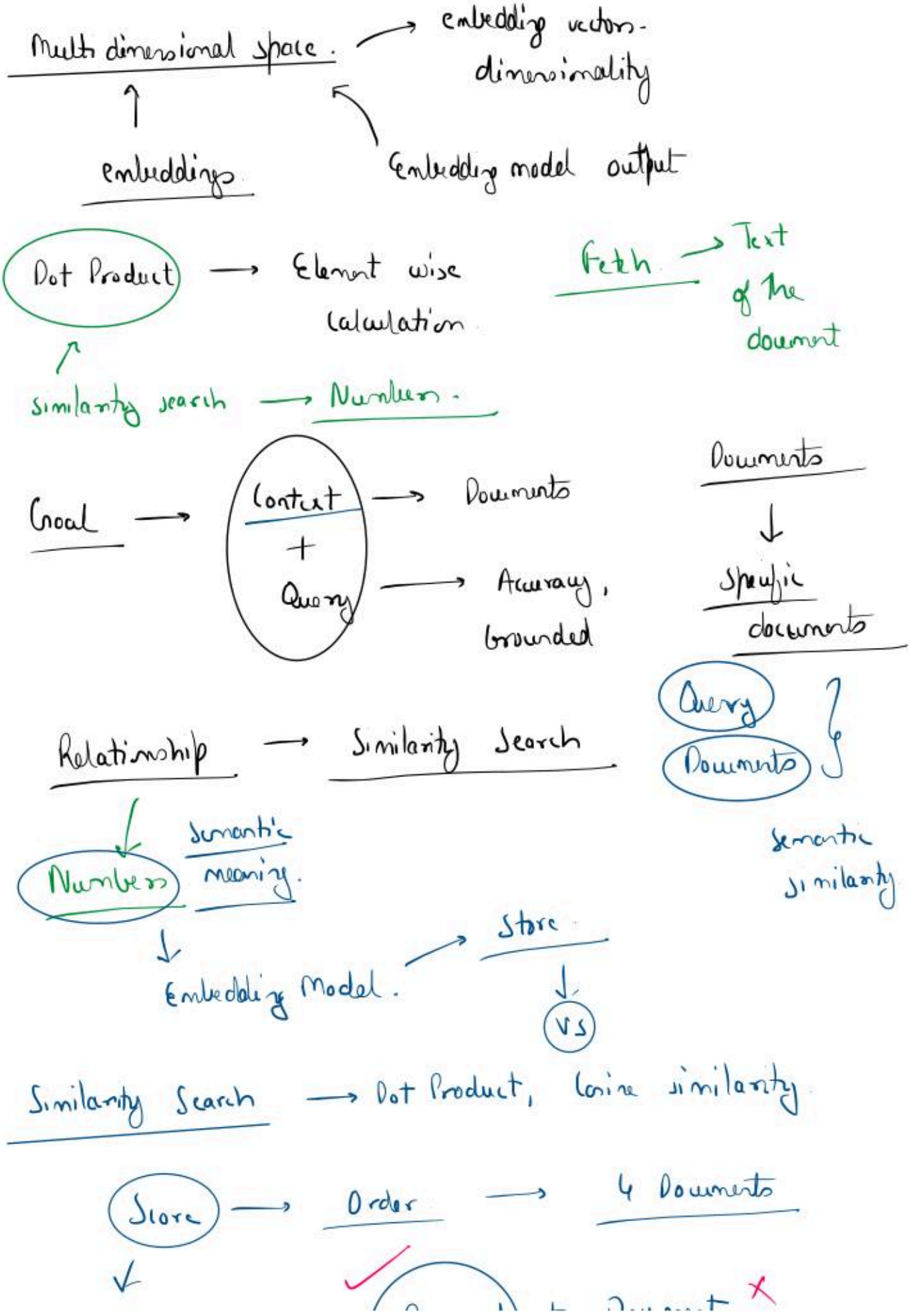
Delete

Read

Similarity

Similarity search

Read


Multi dimensional space.
embedding vectors.
dimensionality
embeddings
Embedding model output
Dot Product
Element wise
calculation
Fetch
Text
of the
document
similarity search
Numbers.
Goal
Context
+
Query
Documents
Accuracy,
grounded
Documents
Specific
documents
Relationship
Similarity Search
Query
Documents
Numbers
semantic
meaning.
semantic
similarity
Embedding Model.
Store
VS
Similarity Search
Dot Product, Cosine similarity
Store
Order
4 Documents

Embeddings

Document / Text + Document / Embedding

Document → 4 key elements of each document

- id →
- embedding vector →
- text →
- metadata →

Embeddings

Similarity search

Ranking X

Document text
Context
Augmentation
Metadata.
→ Prompt

How vectors are indexed

Unit of information.

```
{
  id:         "chunk_001",                          # unique identifier
  embedding:  [0.23, -0.17, 0.85, ..., 0.61],      # the vector
  document:   "LangChain is a framework for...",    # original text
  metadata:   {"source": "doc1.pdf", "page": 3}     # filterable info
}
```

Text

Metadata.

Similarity search ✓

Why ?

Documents → Embeddings → Persist

↓

store.

① costly

② compute

Static

Query → Dynamic ✗

Similarity Search

Query → Computations → Retrieve

① Dot Product

② Cosine Similarity

Brute force / Exact Search

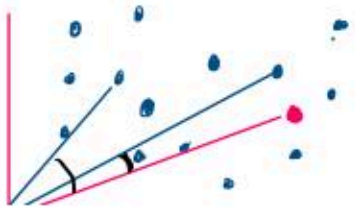
Document Embeddings → 1 lakh

Q1 (Text) → Embedding model → Query Vector

1 lakh → similarities

Q D

Vector Store.



2D

Semantic meaning

Score → More similar query vector

Top 3

(



query vector

Retrieve

id, embeddings, text, metadata.

$S(Q, D_i)$

1 → 10000

Brute force.

10 lakh

1 crore.

1000 docs

Latency

O(n)

n → no of embeddings

Best → Accuracy

Subset of Documents

Indexing → Way to organize your embeddings

Fast Retrieval → $S(Q, D_i)$ → Subset of docs.

ANN → Approximate Nearest Neighbor Search

Accuracy → 99% Exact Search

Maximum Accuracy

Suboptimal solution

odd no.

5

Sort

Top 1, 2

Q1

D_1

D_2

D_3

D_5

D_{10} most similar ✗

D_9 ✓

→ Embedding vector

ANN → ① Clustering (IVF) Inverse file System.

② Graph (HNSW) Hierarchical navigable small world

① Clustering → Unsupervised

↓
Categories

1 lakh document embeddings

VS → Indexing → ANN → Clustering

Central Point → Avg.

↓

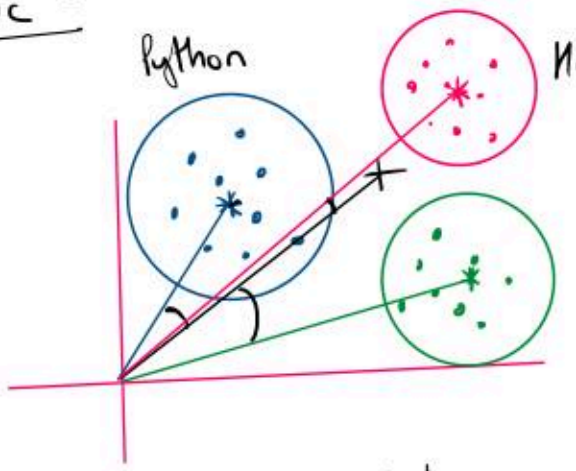
Centroid.

K Means Clustering

Clusters.

Topic

Topic ✓



ANN

Avg semantic meaning.

(Summary)

+ Query Vector

Most similar

Heart Rate

$S(Q, D_i)$ → Exact Match

$S(Q, D_{ci})$

1 lakh

1 lakh

10 clusters.

Clusters.

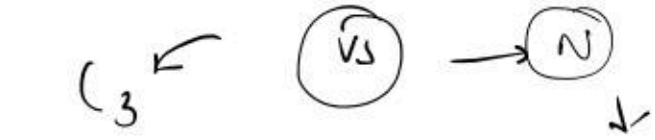
10K Points

10K Points

Centroids

10K + 10

1 lakh



Query → centroids → Clusters.

Subset of Documents

Similarity

Index → Record → IVF (Inverse File system)

C1 (Python) → [E3, E5, E7, E10] X

C2 (Heart Disease) → [E1, E2, E6]

C3 (AQI) → [E4, E8, E9]

Query vector → Cosine similarity

3 scores

↓

Sort → Top score → C2

Search space.

↓

3 embeddings

Subset of docs

10 embeddings

Query → Scores

↓

Sort → Retrieval.

→ exact search

10 calculations.

3 centroid + 3 embeddings.

= 6 calculations

↓ Time

1 lakh → 20 clusters.

↓

5000 documents

→ 1lakh calculations.

20 centroids + 5000 embeddings

vector.

20 times ↓ = 5020 calculations.

VS → initialize cluster. 20 clusters.

add update add.

indexing → update

20 topics. 100 docs doc

↓ centroid

20 centroids score.

→ sorting → add

② HNSW (Hierarchical Navigable Small World)

Graph based

VS → Initialization

Six Degree separation → Social Source

5 person. → Mutual connections.

↓

Every person → Other Person → Donald Trump.

Nirvanshu (A)

Six degrees.

A → B → C → D → E → F

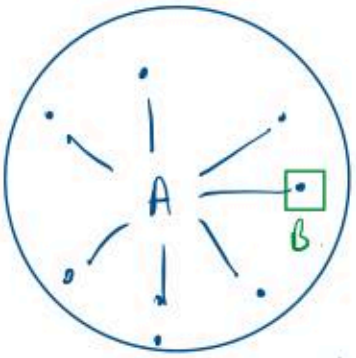
B: relative

C: friend.

D: Professor

E: US Ingenor

F: Donald Trump



Connections - 100

B → 300 connections

exponentially connections ↑

Local neighborhood

A → 100 → 200

↓ ↓ ↓

100 connections 200 200

100 × 200 × 200 × 200

5-6 hops

Entire Pop of Earth

Graph Based structure

Person → Nodes

Connections → Edges



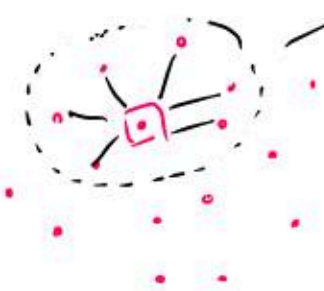
Person

Edges

Indexing

Document embeddings

High Dimensional space → Embedding model output dim



→ local neighborhood

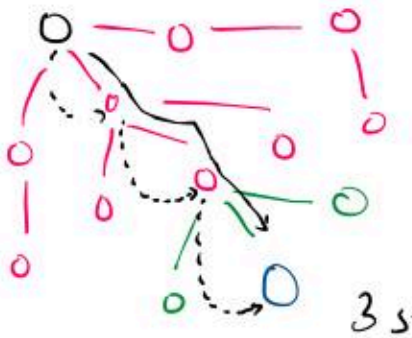
↓ semantic meaning

Connections - Graphs

HNSW

Navigable

Small world.



Path.

Graph

Number of searches

Subset of embeddings

3 steps.

Navigating through graph

H → Hierarchical.

Graph → layers.

Stack layers of graph.

5 Chunks → embedding model ✓

↓

embeddings (1536)

(5 X 1536)

→ scalar value. (5 X 1)

Embedding → 1D embeddings.

C_1 → Python = 1.0

C_2 → JS = 1.5

C_3 → Photosynthesis = 5.0

C_4 → Oxygen as byproduct = 5.3

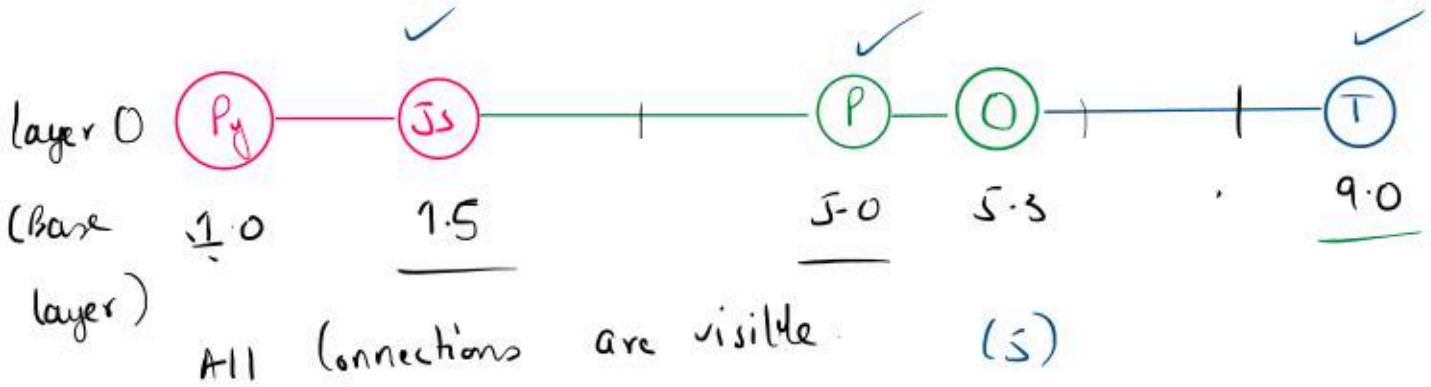
C_5 → Tarifs = 90

V.1 → Indexing → HNSW

VS → Indexing → HNSW

Distances / Number line

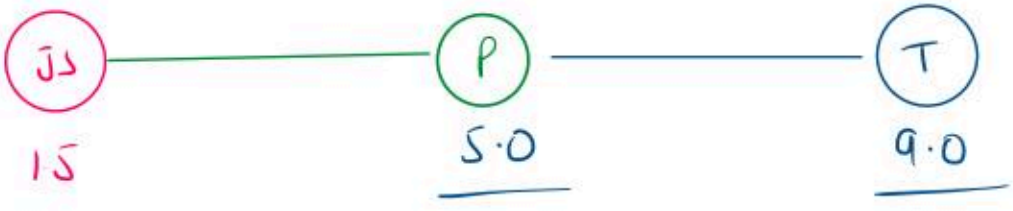
Linear



All connections are visible (5)

layer 1 → 1) Randomly pick embeddings from layer 0

2) Rebuild the new graph with the promoted embeddings

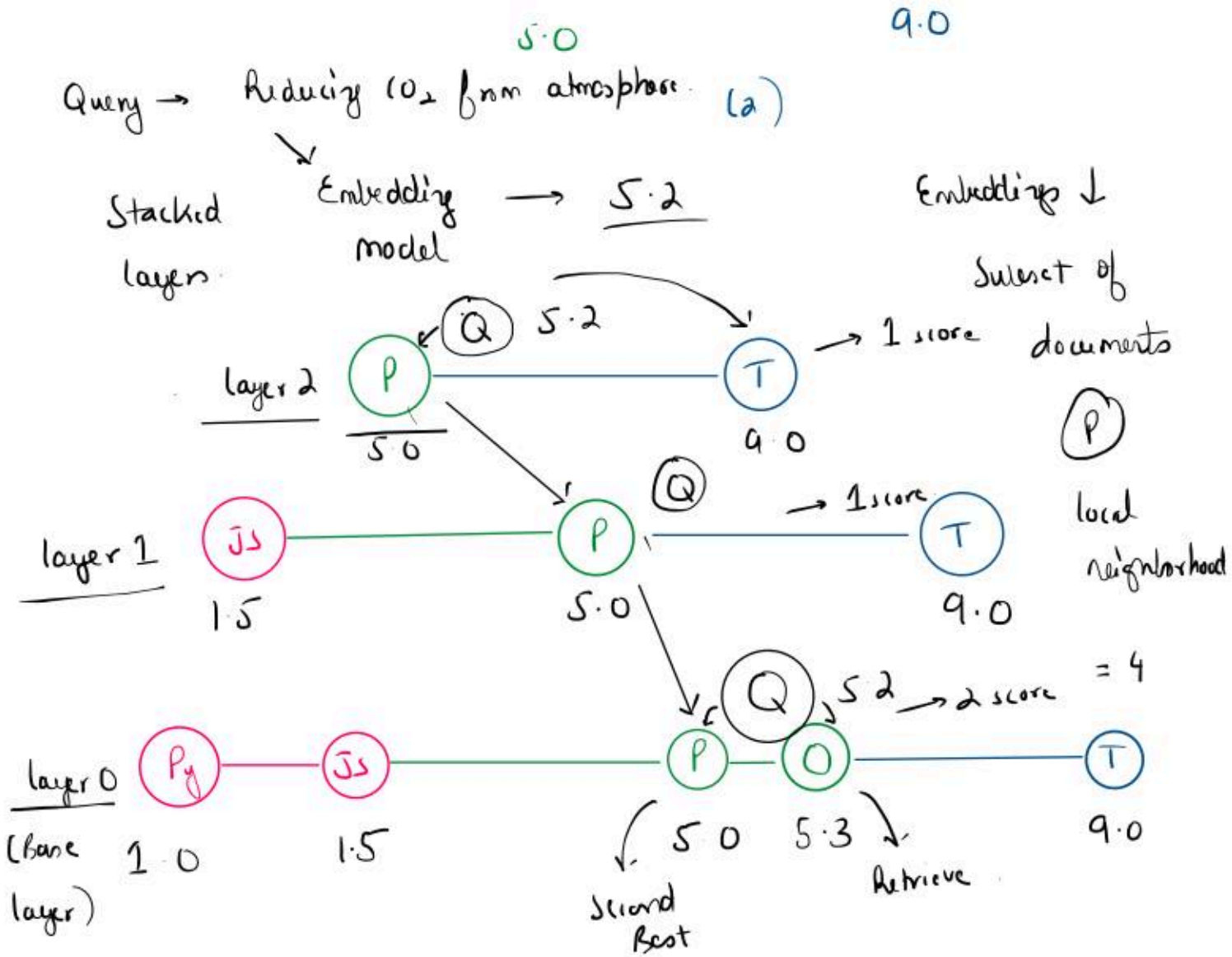


(3)

layer 2 → 1) Promote some vectors randomly from layer 1

2) Rebuild a new graph

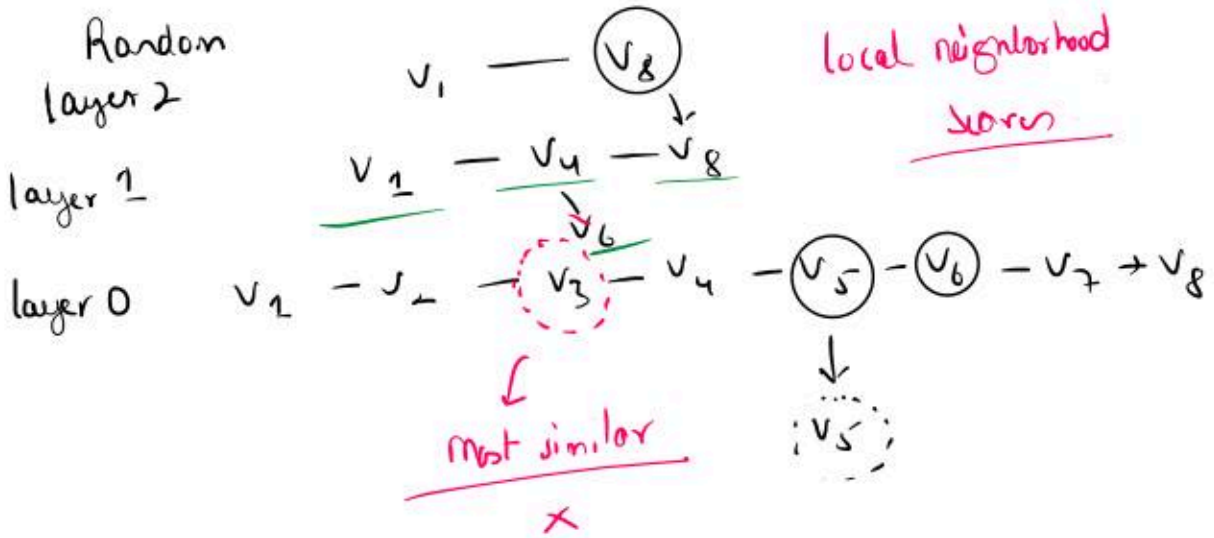




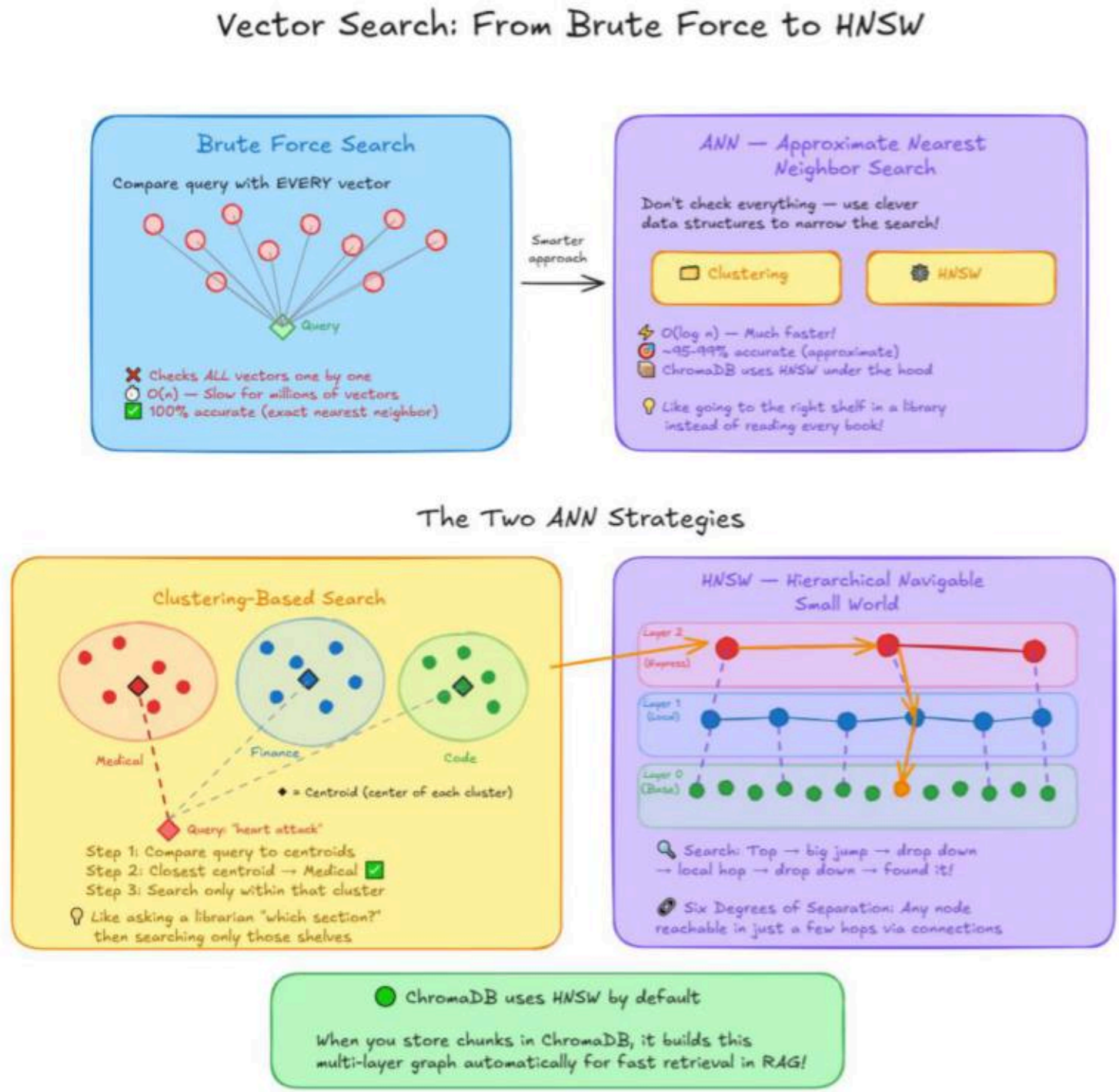
Suboptimal solution

greedy search

95 - 99%



Diagram


Vector Search: From Brute Force to HNSW
Brute Force Search
Compare query with EVERY vector
Query
Checks ALL vectors one by one
O(n) — Slow for millions of vectors
100% accurate (exact nearest neighbor)
Smarter approach
ANN — Approximate Nearest Neighbor Search
Don't check everything — use clever data structures to narrow the search!
Clustering
HNSW
O(log n) — Much faster!
~95-99% accurate (approximate)
ChromaDB uses HNSW under the hood
Like going to the right shelf in a library instead of reading every book!
The Two ANN Strategies
Clustering-Based Search
Medical
Finance
Code
◆ = Centroid (center of each cluster)
Query: "heart attack"
Step 1: Compare query to centroids
Step 2: Closest centroid → Medical
Step 3: Search only within that cluster
Like asking a librarian "which section?" then searching only those shelves
HNSW — Hierarchical Navigable Small World
Layer 2 (Express)
Layer 1 (Local)
Layer 0 (Base)
Search: Top → big jump → drop down → local hop → drop down → found it!
Six Degrees of Separation: Any node reachable in just a few hops via connections
ChromaDB uses HNSW by default
When you store chunks in ChromaDB, it builds this multi-layer graph automatically for fast retrieval in RAG!

Chroma DB → VS

FAISS → minimal

Vector stores → Embeddings Searching

Vector Database → Database functionalities

Pinecone, Milvus, Qdrant → Persist Cloud.

ChromaDB → Open Source → free to use, Cloud features

Paid features

RAG Pipeline

Persistance → In memory / On Disk

Chroma DB → integration

documents → Chunks → Embedding model → Embeddings → Vector store

One time process { Time, cost }

Save

In memory → RAM → volatile memory

VS ✗

VS

On Disk → Non volatile → HDD / SSD → Chroma DB

embeddings → { id: —, embedding: — }

Unit of information

embeddings → { id: —— ,
embedding: —— ,
document: —— ,
metadata: —— }

Implementation → ① CRUD operation
- Create (2 vars)
- Retrieve, Read, Searching
- Update
- Delete

→ ② VS accept → persist

→ ③ RAG Pipeline → Pdf → Doc loader → Split → embeddings → VS

Chroma DB → collections
Folders, Table

Required

① Company docs → employee

② Customer Policies → Customers

Retrievers → Basic

Advance Retrievers

RAG → Augmentation
RAG → Retrieval
RAG → Generation

80%

Retrieval → Query → Relevant Docs

Goal → Query → External Knowledge → retrieve → relevant

LLM → Context ← External information
Context ← Add
Query
Grounded, factual

LLMs have limited context window

Useful information

dynamic query

Query-1 → (X) → Accurate response

Query-2 → (Y) → Accurate response

Retrieval → Retriever

Retriever → Useful info
Retriever → noisy info ×

Accurate, grounded, factually correct

Retriever → Understands my query

↘ Get me useful info from external source

① Build the external knowledge source.

② We perform retrieval based on input query

Build → External knowledge source

① Document loaders.

Multiple sources → extract → Dump as a unified format

② Text splitters / Chunking strategies

Big document dump → Break into multiple parts

a) Cannot add to context

b) Downstream procs

③ Embeddings → Text → chunks

↓

Numerical representation → Dense vectors

Semantic

Numerical representation — Semantic meaning

Calculations

Query → Useful information

↓ comparisons → Similarities → Store.

Metrics → L2
→ Cosine sim
↓ Dot product
} → Similarities

Documents vectors + Query vector.

(4) Vector Store → Document embeddings

↓

Store → Persist

+

Index → You organize the storage proces for faster searching

Vector Store

↓ Store
↓ indexing
↓ similarity search

Vector Store → Embedding model.

↓ Retrievers

Indexing

Exact Search

Query → Doc

Smart Search

O(n) → linear search.

Approximate nearest neighbor search (ANN)

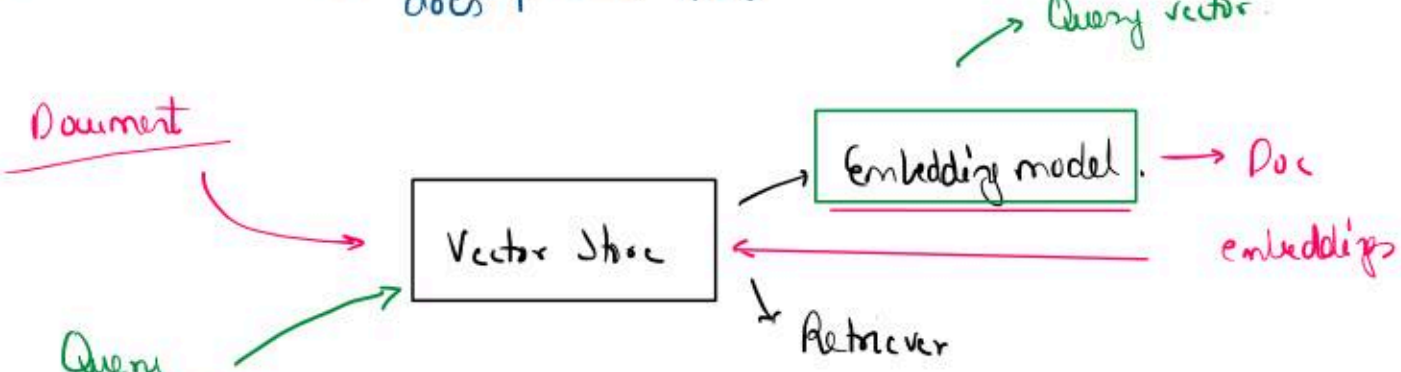
time ↓ accuracy → 95 - 99%

① IVF (Inverse file system) KMeans Clustering

② HNSW Graph layered graphs → Stacked graphs

dynamic → docs ↓ → Exact Search

↘ docs ↑ → ANN



Similarities

Document vectors ✓

Query vector ✗

Query

Vector Store → Document embeddings (static) → Persist

↘ Query Vector → dynamic → Persist ✗

Retriever → Query vector → VS → Searching Similarity → Similarity Scores → Sorting (Similarity) → Higher score (Descending) → Retriever

Calculations

Document embedding → 10

Query vector → 10 calculations → 10 scores

Metric → L2 → lower score → more similar

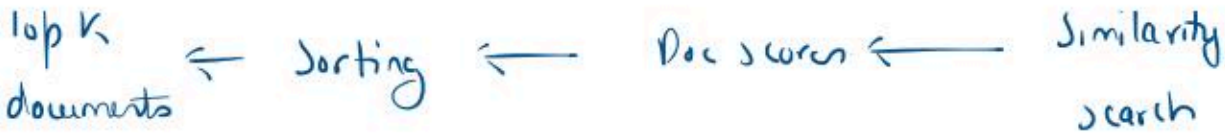
Cosine sim → Higher score → more similar

Retriever → k = 3 → Top 3

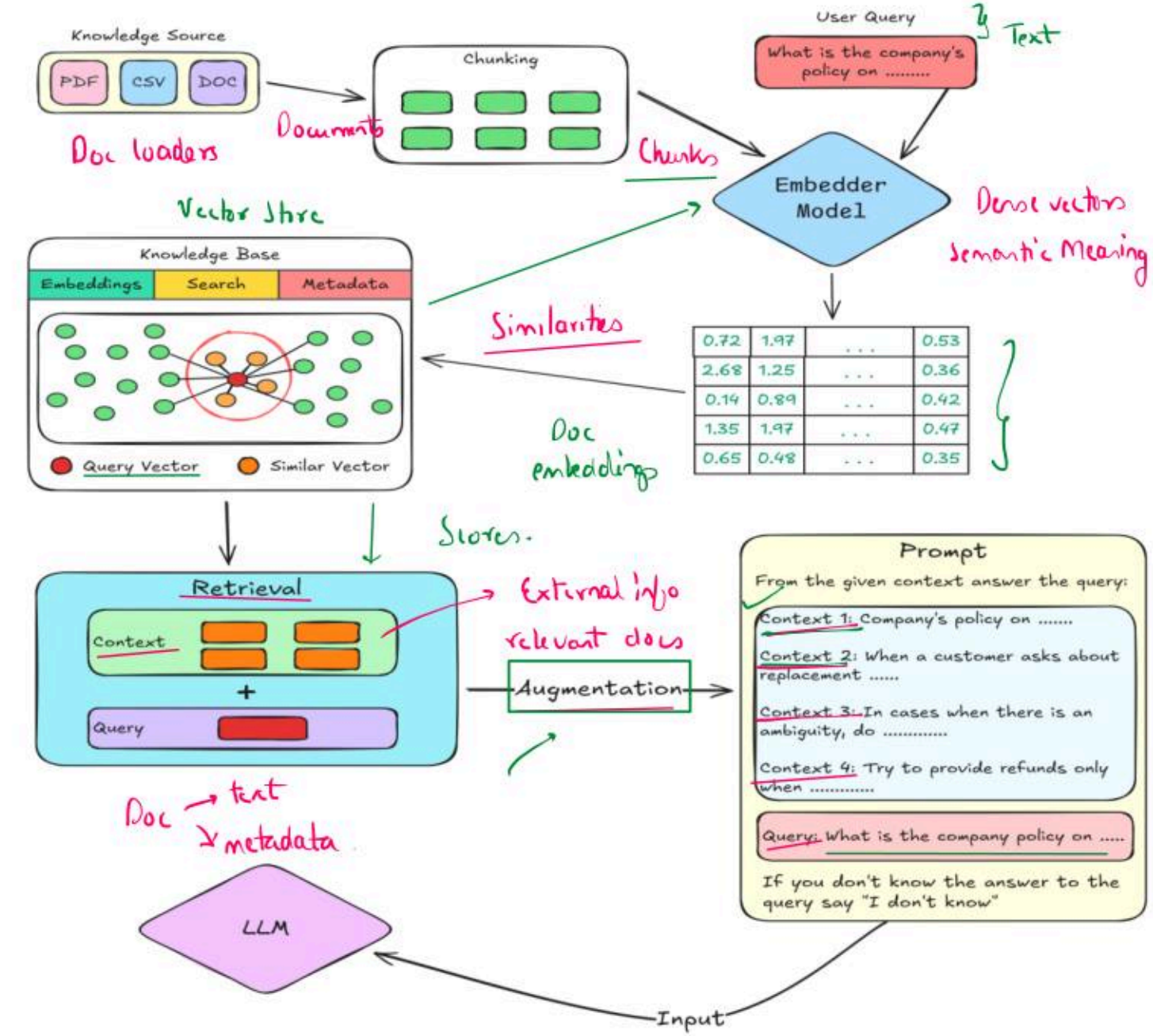
k = 5 → Top 5

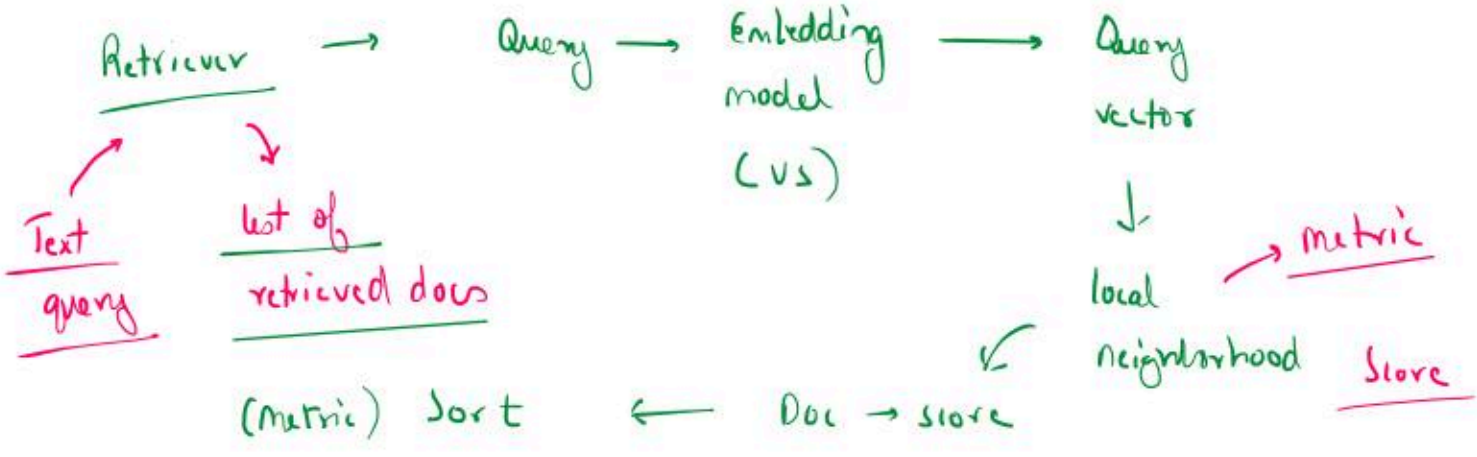
Retriever → sub component of VS

Query → Embedding model → Query Vector → VS → Similarity → Doc scores → Sorting → Top K documents


top K documents
Sorting
Doc scores
Similarity search

The Big Picture





↓

Retrieval (k) $k = 4$

Embeddings → Calculations → Documents → metadata
                                      ↘ page_content

Vs → Embeddings → Unit of information

{ id: — , metadata: ,
embeddings: — , text: — }

{ metadata.
text (page content) } Doc object → Output

list of Documents

Vector stores

Storage indexing Searching

vs. similarity-search (query)

tuning, customization

vectorstore → Basic retrieval

Advanced Retrieval → Retriever

searching, parameters

Runnables → chains integrate

RAG Pipeline.

→ Query | Retriever | Prompt | LLM | Output Parser → text response

generation

Text

List of documents

Template.
context. Retriever
Query: Input

Retrieval → 80%

What after Retrievers

Augmentation → Prompt design

↓

enhance.

ICL

Final prompt

Context: Retrieved context

Query: Query as input

↓

LLM Pretrained

Useful/Relevant information + External sources

Query → Response

① factually correct → Accuracy

② latest information → latest knowledge

Document metadata → Use while generating response.

↓

Proper citations

Generation

Prompt → LLM → Response

Context

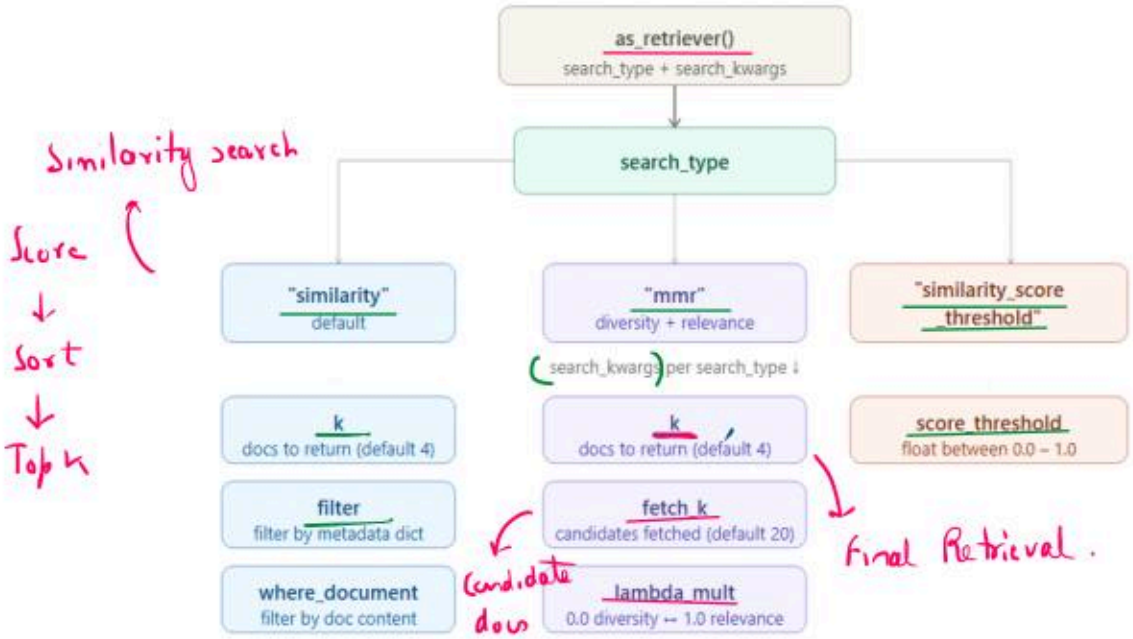
Query →

↓

Correct, Grounded, having updated info

Basic RAG Pipeline.

Basic Retriever



Retriever → Sub-component of Vector Store

Vector Store

vector store integrations

• as_retriever()

Retriever object → Retrieval

query

List of Relevant Docs

Runnable

Chains,

invoke.

Parameters → Tune.

① search type → algorithm → method → similarity → with threshold. (similarity search)

→ mmr

② → parameters

② search_kwargs → parameters to tune the search method.

a dict

{k: 3}

k

filter: { source: wikipedia }

k = 3

Metadata filtering

Threshold : 0.75

Vector store similarity score ≥ 0.75

retrieve.

Query

Doc 1 → similarity score

Doc 2 → score.

Doc 3 → 0.78

Doc 8 → 0.81

Doc 10 → score.

Doc 10 → 0.93

Threshold value.

Threshold → 0.90

0.95 → return no documents → [ ]

2 files → similarity search → k

filter → Metadata filter.

similarity_threshold → Values

MMR → Maximal Marginal Relevance.

Relevancy + Diversity

Retrieval

Relevant: similar to Query

Diversity: Non-similar from each other

Similarity Search

Goal → Retrieve top k most similar docs from Vs (User)

having high sim with input query

Retrieved docs → Highly similar amongst each other

Goal: Query → Context → Retriever ← Useful info

Highly similar K = 5

[C1, C2, C3], C4, C5 → Context

C1, C2, C3: duplicate info

C4, C5: redundant info

① Useful for the given query (High Relevancy)

② Dissimilar amongst each other (High Diversity)

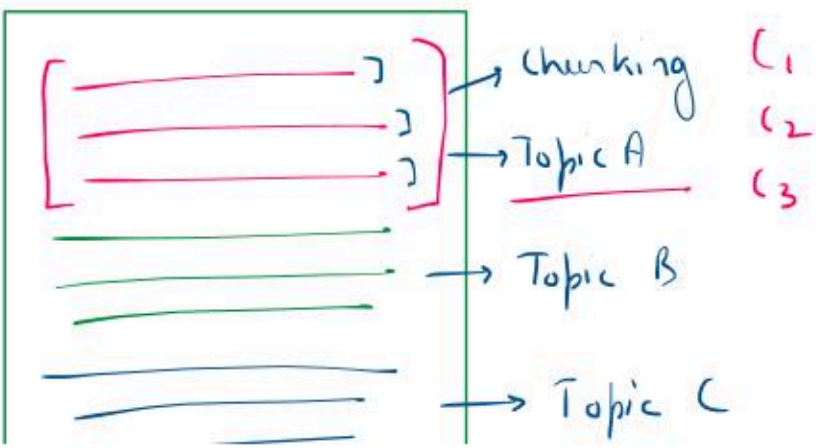
K = 3

Redundant info.

Query

Score

[C1, C2, C3]





→ Topic C

[C1, C2, C3]

Context → Prompt Augmentation ↓

LLM → Generate Response.

k=3

Redundant info.

Accurate, diversity ↓

Context ← 3 Chunks

← 1 Chunk

Trade off

C1, C2, C3 → Context

↓ diverse

↓ 3 pieces of info

Response ↓ less accurate, coverage ↑

diverse information

Diversity ↑ Accuracy

Research related Chatbot

Redundant

Similarity → Accurate

Diverse

Coverage ↓

MMR → Relevance + Diversity ↓ Responses → Accurate

→ Diverse.

Step by Step implementation:-

K, fetch_K, lambda_mult

{ K: 3, ✓ Final Retrieval

Relevancy + Diversity

fetch_K: 30, Candidate docs

lambda_mult: 0.5 } Relevance + Diversity

→ Query vector.

+
Diversity

① Query → Embedding model → Query vector.

Vector store → Document embeddings → Similarity search → Scores → Sorting → Retrieval

Retrieval → Top fetch K documents

Query → 200 docs → store similarity → Sorting → Top 30 docs based on similarity

Top 30 similar docs to my query (Relevancy)

Query similar → Top fetch_k docs similar to query vector (Relevancy)

fetch_k → Candidate docs

k=3, fetch_k = 30

200 doc ✗ Top 3 doc retrieve

② Candidate Docs → Final Retrieval

Candidate Docs → 30 docs

Final Retrieval → 3 docs (Diverse)

Relevancy + Diversity

lambda_mult

↓

Trade off lrlw

Relevancy + Diversity

↓

Tradeoff b/w

0 - 1

Relevance | Diversity

1 | 0

0.5 → By default equal focus on Relevancy + Diversity

Relevance ↑ Diversity ↓

Relevance ↓ Diversity ↑

300 Query top 30 similar docs to my query

fetch_K = 30

① Retriever → Query vector → VS → Similarity Search → Score → Sorting → 30 Candidate Docs

Doc → 300 → top 30 Position 1

② 30 candidate docs → Final retrieval

Most Relevant Doc

a) Retrieve doc that is most similar to my query. Query

k: 3 Final Retrieval

b) Apply a formula → Remaining 29

formula → New Score

MMR Score → 29 Docs (Diversity)

Doc-100

Doc-18

↑ Relevance increase + ↓ Diversity inc

$$\text{MMR Score}(d_i) = \lambda \cdot \text{Sim}(d_i, Q) - (1-\lambda) \max_{d_j \in S} \text{Sim}(d_i, d_j)$$

$$\text{MMR Score}(d_i) = \lambda \cdot \underbrace{\text{Sim}(d_i, Q)}_{\text{Similarity score b/w doc and query.}} - (1-\lambda) \max_{d_j \in S} \underbrace{\text{sim}(d_i, d_j)}_{\text{Inter document similarity}}$$

$d_j \rightarrow$ Already selected doc

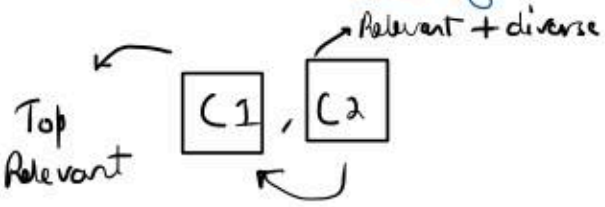
doc 1 ⋮ → Scores ⋮ doc -29

↑ ① Sim b/w doc - 1 and Query ✓ (Relevance)

Redundant info ↓ ② similarity b/w doc - 1 and doc - 100 (Already selected)

↓ diversity

Relevance + Diversity



doc ⟷ Query

(Relevance)

Sim 1 → High (Relevance)

Sim 2 → low (Diversity)

λ - lambda_mult → MP Retriever

1 — Relevance only

0 — Diversity only.

$\lambda = 1$

$= \text{Sim}(Q, d_i)$ Relevance

$\lambda = 0$

$\max_{d_j \in S} \text{Sim}(d_i, d_j)$ Diversity

⟶ 29 candidate docs → MMR Scores 29 for each remaining doc

MMR Score is highest Doc-18 ↑

① It is second most relevant to my query
+
Also diverse from my first selected query

K:3

© Candidate Docs → 28

MMR score → 28 MMR Calculations { Doc-18 / Doc-100 } ← S

Sim (Q, 28) docs.

Sim (d_j, $d_.$) → 2 ↗

d_i → candidate doc - MMR calculate (1-28)

Q → query vector

S → set of documents already selected so far.

d_j → Doc selected from S

$\max_{d_j \in S} Sim(d_j, d_i)$ → Maximum value between S and candidate doc

Penalty.

between S and candidate doc

doc-1 → doc 100 → 0.93 (circled) Why?

doc-1 → doc -18 → 0.62 ↓

S

Penalize similarity in favour of diversity

MMR score ↑ Relevant

Redundant info

C_1, C_2, C_3

MMR ↑

Penalty term −

I want to select that doc amongst remaining candidate docs that is least similar to my already selected docs

Higher diversity

$\lambda = 0.5$ doc - d_1 → Relevant

	Sim to Q	Sim to selected d_1
d_2	0.9	0.88
d_3	0.75	0.20

$\lambda = 0.5$

MMR for d_2 = $0.5 \times 0.9 - (0.5 \times 0.88)$

$= 0.45 - 0.44 = 0.01$

MMR for d_3 = $0.5 \times 0.75 - (0.5 \times 0.22)$

= $0.375 - 0.11 = 0.274$

d_1, d_3 ← Diverse — Non Redundant info — Very less

Max → 2 values. — diversity ↑ — Diversity

Mean — Penalize

Similar — Maximum

Redundant — ✓ C_1, ✓ C_2, ✓ C_3, C_4 ✗ — completely diff from all of my selected docs

C_4 → Information

Chunk ✗ similar

We want complete diversity.

$$\text{score}(D,Q) = \sum_{q \in Q} IDF(q) \cdot \frac{f(q,D) \cdot (k_1 + 1)}{f(q,D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)}$$

① Intuition

② Standalone.

BM25 → Best match 25 — 25 iteration

↓

keyword matching

Working → TF-IDF (Term frequency - Inverse doc frequency)

↓

improve → BM25

Vectorizer — Text → Vectors. — Embedding — text → num representation

Embedding models → DL models → capture the semantic meaning → numbers → Dense representation

Vectorizer → ML algorithms → Keywords → number

corpus

↓

Sparse representation

↓

mostly zeros.

TF-IDF → term frequency (TF)

TF-IDF

↘ Inverse doc frequency (IDF)

Retrieval
↓
keyword matching

✓ Term frequency → Word text → repeat
count of the word inside text

✓ Inverse doc frequency → Word Document → repeat

Query → What are AI Agents and what do they do?

Docs → AI Agents (Relevant to my query) Keyword matching

AI Agents frequency ↑ relevant → Chunk

AI Agent → important

document frequency ↓

5 pages

Pdf. → for AI

↓ ↘ 100 pages

knowledge source

↙ Chunks

Chunk ↑ document frequency ↑ X Keyword matching

unique, rare → Chunks retrieve

Term frequency ↑ → Chunk } → Score. ↑

Term frequency ↑ → Chunk

Inv doc frequency ↓ → Rare words → Score.

TF IDF → Chunks → retrieve

Term frequency ↑

Doc frequency ↓

Useful chunks

Bag → chunk

3 balls orange

100 Balls

3 balls | orange

Rare. → Imp

TF-IDF → Problems. → Retrieval → Relevant.

AI Agents

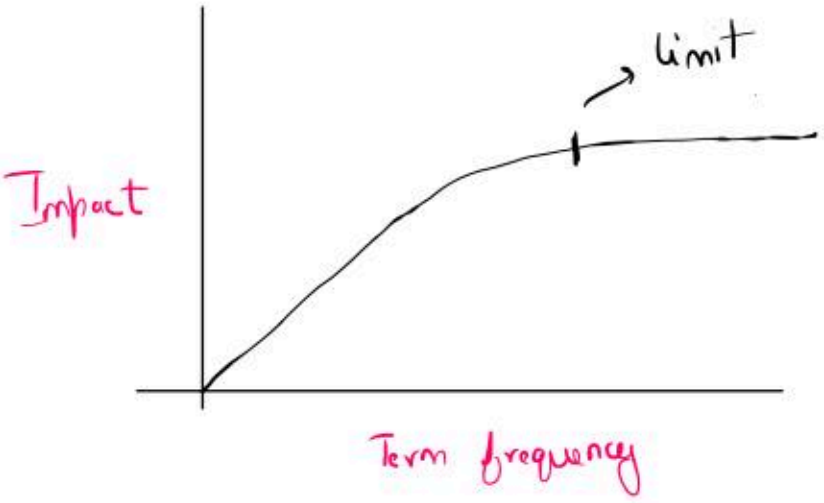
① Term frequency → limit / cap → Limit

AI Agents AI Agents AI Agents . . . . . 100 times.

Useful info ✗

10-20

No impact



② Normalize doc length → IDF Rare word ↑

doc length

AI agents → 5 repeat

Pdf → 3 pages

Pdf → 500 pages

AI agents → 100 repeat

length normalize

Normalize

Standardize → Ranty IDF

BM25 → ① Cap the term frequency (TF)

↓

② Normalize the doc length (IDF)

Keyword AI Agent → q f(q, D)

$$\text{score}(D,Q) = \sum_{q \in Q} IDF(q) \cdot \frac{f(q,D) \cdot (k_1 + 1)}{f(q,D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{avgdl}\right)}$$

|D| → Number of words

avg dL

q → AI Agents

score (D, Q)

$$\text{Score} = IDF \times \frac{TF\,(k_1+1)}{TF + k_1\,(\text{length norm})}$$

length → Normalization

avg dl

$k_1 = 0 + 1$

$= 1$

k_1 → constant / parameter → capping your TF

VS → Embeddings → semantic meaning → Dense vectors.

① Keyword matching completely diff

BM25 → Reranker

↓ docs → score → ranking

langchain

Score → Sparse vectors.

BM25 → Embedding model ✗ TF-IDF

MMR → Query → Embedding model → Query vector → Dense.

Similarity search

semantic meaning

Automobile service.

↕ related

Car service. similar.

BM25

✗ Automobile ✓

Car ✗

Keyword matching

Standalone. ✗

specific ✗

Other retrievers → Hybrid search

① Keyword matching → BM25

② Semantic similarity → Similarity, MMR

} → Hybrid search

2 topics → Hybrid search (technique)
↓ Ensemble Retriever (component)

Ensemble Retriever → ML → Ensemble models → Bagging, Boosting, Voting, Stacking

Multiple models → Prediction → final prediction

↓ 10 models → xg Boost → 10 estimators

Ensemble Retriever → Multiple Retrievers

↓ List of Retrievers

↓ Weight to retrievers → similarity score

Say in final results

Retrievers → ① Similarity search → 0.6 → 60%
② MMR → 0.4 → 40%

Retriever → similarity search → list [Documents] D_1, D_5, D_6

MMR → D_5, D_6, D_8

↓ 0.6 × simsearch

Weight value

↓

scores × 0.4

0.6 ×

simsearch

scores

Score

Different → Score diff, Rank diff

D_1 → 1st

D_5 → 2nd

D_6 → 3rd.

D_5

D_6

D_8

Final Retrieval

↓

D_1, D_5, D_6, D_8 → Rank

Final score

Merge.

How do you fuse / merge the results -

RRF → Reciprocal Rank Fusion.

RAG Fusion → Advanced RAG technique.

↓

RRF → mechanism.

Component → Ensemble

Hybrid search → similarity search, Key word search → Retrieval

Q similarity search → Algorithm, Dense vectors.

↓

embedding models

Text → semantic meaning.

→ Dense

meaning.

Semantic search → similarity, threshold, mmr. → Dense

② Key word searching → BM25 → Sparse

Retrievers → list of documents

merge / fuse. → Ensemble Retriever.

Ensemble Retriever → ① similarity search → 0.5

② Key word matching → 0.5

Retrieve → merge into a single result

↓

RRF technique.

Ensemble Retriever → class

② Notebooks

classic

① Ensemble Retriever. ✗

② Custom ensemble Retriever (Base Retriever) ✓

RRF technique.

Advanced Retrievers → Basic retrievers

deprecate

Code → 2 implementations

→ langchain - classic ✗

→ Custom Implementation

deeper understanding

Code ✓

Retrievers → Component → Retrival.

Query → Embedding model → Query Vector.

VS → semantic search

Results → Sort (metric) → Retrieve top k docs → Relevant docs

Doc → Score

VS → Retrievers

Chroma DB → as_retriever

search type (method)

'similarity'

kwargs

{k: 5}

(1) Similarity search K = Top k docs to retrieve

(2) Threshold → K ✗ Score threshold → 0.8

→ semantic similarity

↑

1

② Threshold → K X Score threshold →

Scores → Retrieve. → 1, 2, [ ] → 10

③ mmR → Maximal Marginal Relevance. Topk documents

Relevance + Diversity Redundant info

X

Context → External knowledge

(Previous)

Relevant → Query

Diverse → redundancy ↓

Relevance + Diversity

→ ① Diversity

② Variation.

③ Coverage } → It will be relevant to my input query

mmr → final retrieval = k.

→ candidate docs = fetch_k

↓

lambda_mult = 0.5

1: Only Relevance, Diversity ✗

0: only diversity, Relevance ✗

④ BM25 → Best Match 25 → Sparse vectors → Key word matching

↓

TF-IDF

improved

Embedding models ✗

Vector stores ✗

Standalone ✗

improved

TF capping.

length normalization

→ accurate results

Standalone ✗

less accurate as compared to semantic similarity

(5) Hybrid search → Multiple Retrievers

↙ ↘

Semantic Search Embeddings

Keyword search BM25

Ensemble Retrievers → Hybrid search → list of Retrievers
① similarity search
② BM25

→ Weight of retrievers.
[0.6, 0.4]

2 Retrievers → 2 list of documents

↓ results

merge / fuse.

↙

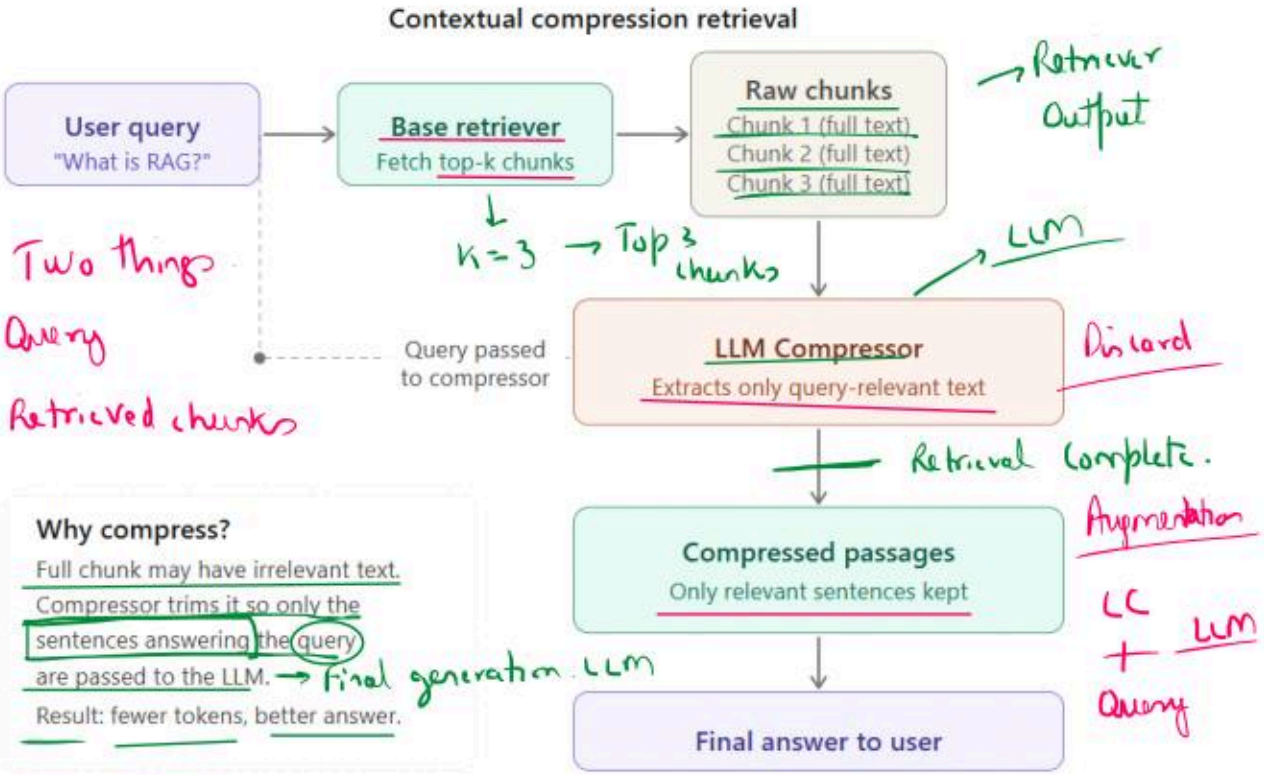
RRF → Reciprocal Rank fusion

Fuse → final results

Ensemble Retrievers → langchain - classic.

Custom.
RRF

Contextual Compression Retriever



Retrieval Pipeline

Query → Query Vector → VS → similarity search

K

Top scoring docs ← score.

topic → 1000 words.

VS → 10 chunks

Chunk 1 → 5000 words

Chunk 2

⋮

Chunk 10

→ Embeddings

Compression of info

5000 words → 1556

Chunk 1/10

↓

accurate

avoids loss of information

Chunk → 5000 words

↓ semantic meaning

of information

1000 words → topic → 5 topics → info → numbers.

Query → Specific → 1 topic ↓ { Chunk ? Query vector

Store

K = 1

Chunk 1

↓ → 5 topics.

Context → Using Prompt → generate response

query → answer ✓ → Additional info.

↓

not relevant to the given query

K = 5

Chunk 1 : Chunk 5 → 5000 words. → query

25000 words

↓

generate response

Context → Previous relevant + noisy info

LLM → confuse → noise add

↓

noisy

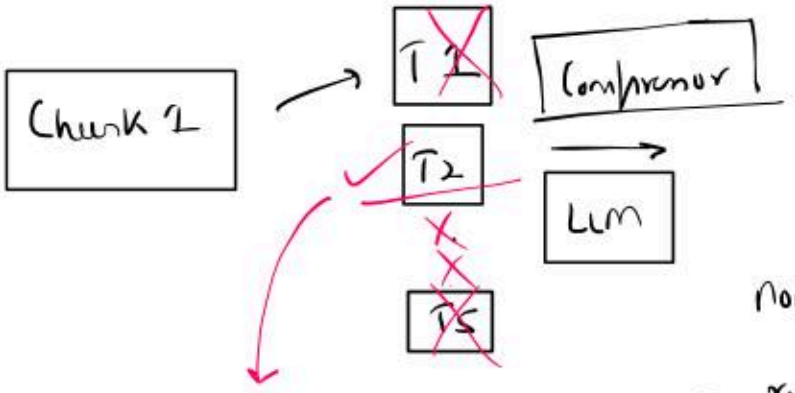
→ Docs Retriever

→ Retriever → Base Retriever

Contextual Compression Retriever

Chunk 1 → 5 topics.
5000 words
1000 words

3 Docs Query Compress Query



Prompt
Chunk, Query
noisy info ✗
relevant info → retain

None ✗

Augmentation. Context + query

Chunk → relevant info
↓
Compressed chunk

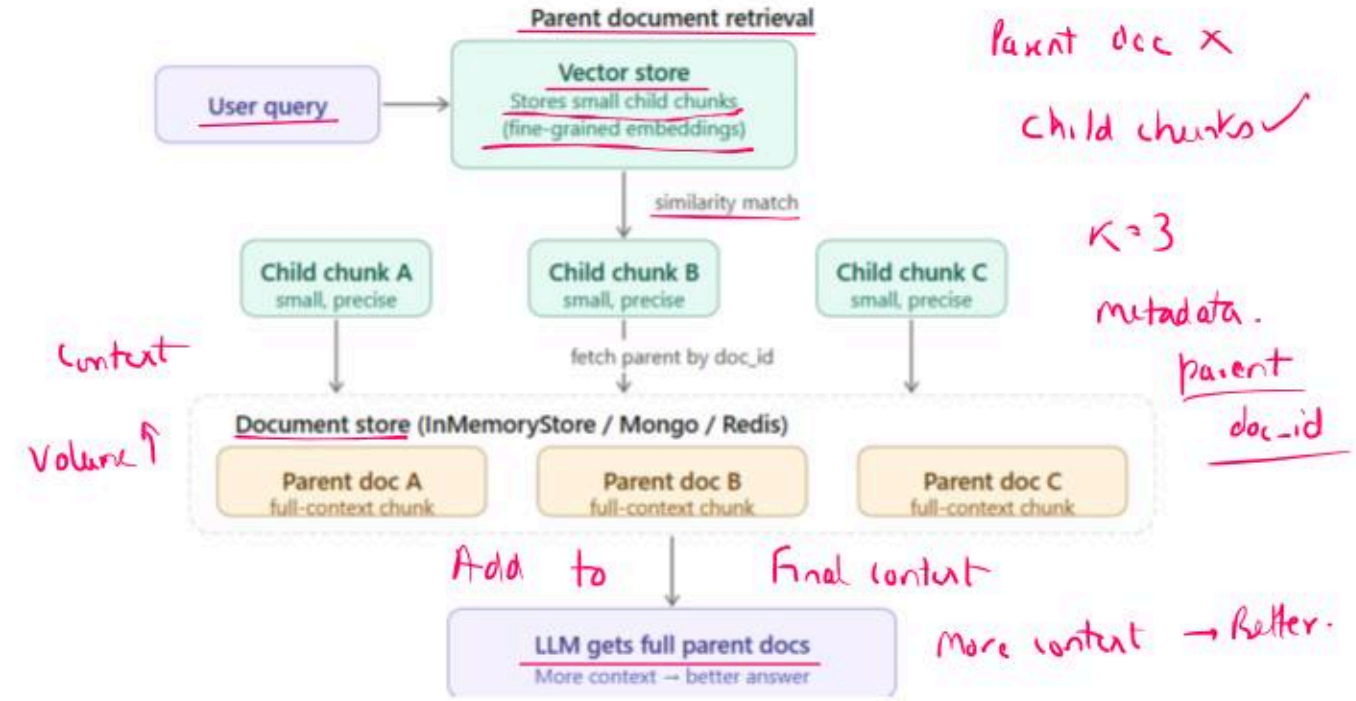
Generate Response → Context

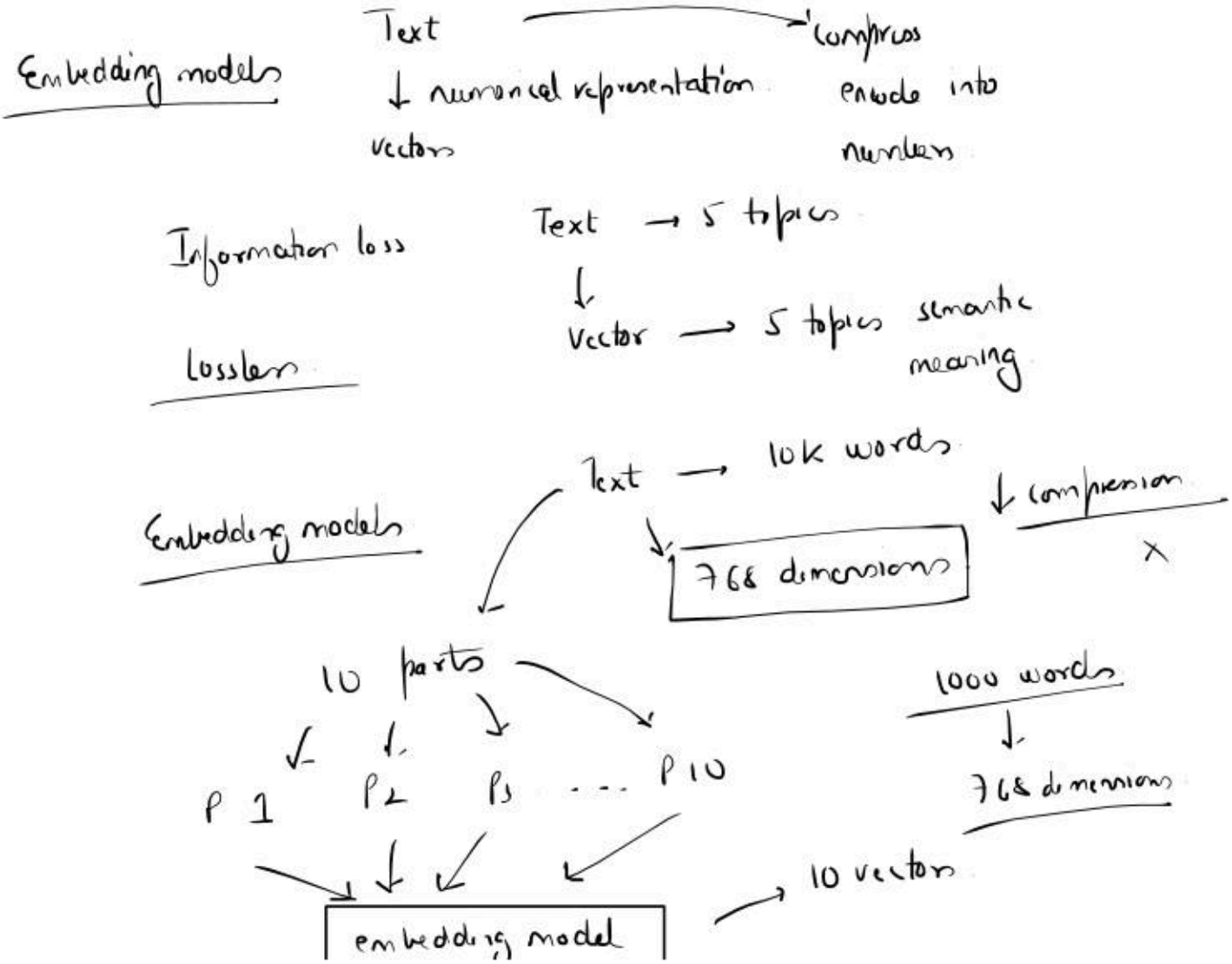
accurate, relevant no additional info ✗

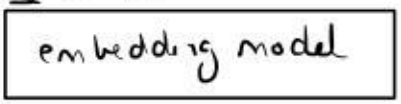
Context → Pollute
non-relevant info. ✗

→ Shorter, Precise, relevant context based on user query

2 implementation → langchain
↘ Custom → Reuse


Parent document retrieval
User query
Vector store
Stores small child chunks
(fine-grained embeddings)
similarity match
Child chunk A
small, precise
Child chunk B
small, precise
Child chunk C
small, precise
fetch parent by doc_id
Document store (InMemoryStore / Mongo / Redis)
Parent doc A
full-context chunk
Parent doc B
full-context chunk
Parent doc C
full-context chunk
LLM gets full parent docs
More context → better answer
Parent doc ✗
child chunks ✓
K=3
metadata.
parent
doc_id
Context
Volume ↑
Add to
Final context
More context → Better.


Embedding models
Text
numerical representation
vector
compress
encode into
numbers
Information loss
Lossless
Text → 5 topics
vector → 5 topics semantic meaning
Embedding models
Text → 10K words
768 dimensions
compression
✗
10 parts
P1 P2 P3 ... P10
1000 words
768 dimensions
embedding model
10 vector



Context length → Embeddings ↓

① similarity search
② Augmentation

RAG → External Knowledge.
↓
Retrieval → context

similarity search accurate

Retrieval → highly similar to query

similarity → embeddings → embedding model.
↓ quality ↑

Retrieval quality ↑

smaller chunks of text

Retrieval quality ∝ Embedding model quality

Smaller chunks → VS → Retrieve

Query similar context → Augmentation

Retrieved doc → info

External info add
↙
inadequate (volume ↓)

chunks ↑ → embedding model → embeddings → retrieval quality ↓
↘ semantic meaning of the original text

Tradeoff

Responses better → Bigger context → More volume of info.
→ Smaller chunks → Better Retrieval.

Augmentation. (chunks ↑) , (chunks ↓) Retrieval.

Context create — sim search

Volume ↑

Parent Document Retriever.

① Similarity Search / Retrieval → Smaller chunks

Retrieval quality ↑

② Context form / Augmentation → larger chunks

Volume of info ↑

Components

① Parent splitter. → Text splitter → large chunks from your docs

② Child splitter → Text splitter → Smaller chunks from your parent docs

③ Vectorstore (child.) → ChromaDB → child chunks → embeddings

④ Document store → In Memory / File storage → key : value store.

↓ Parent ↓ Parent doc.

Doc loader

Parser / Etcs.

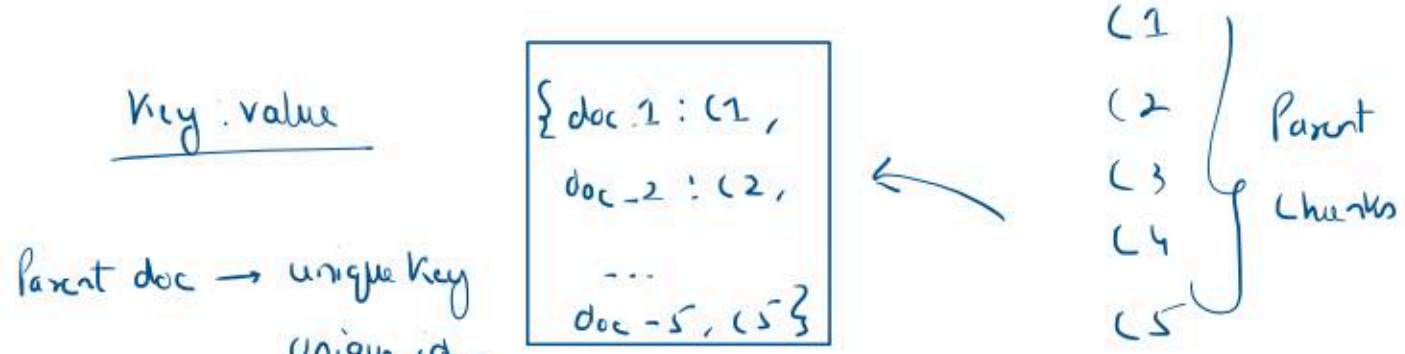
Doc loader

Parent / steps.

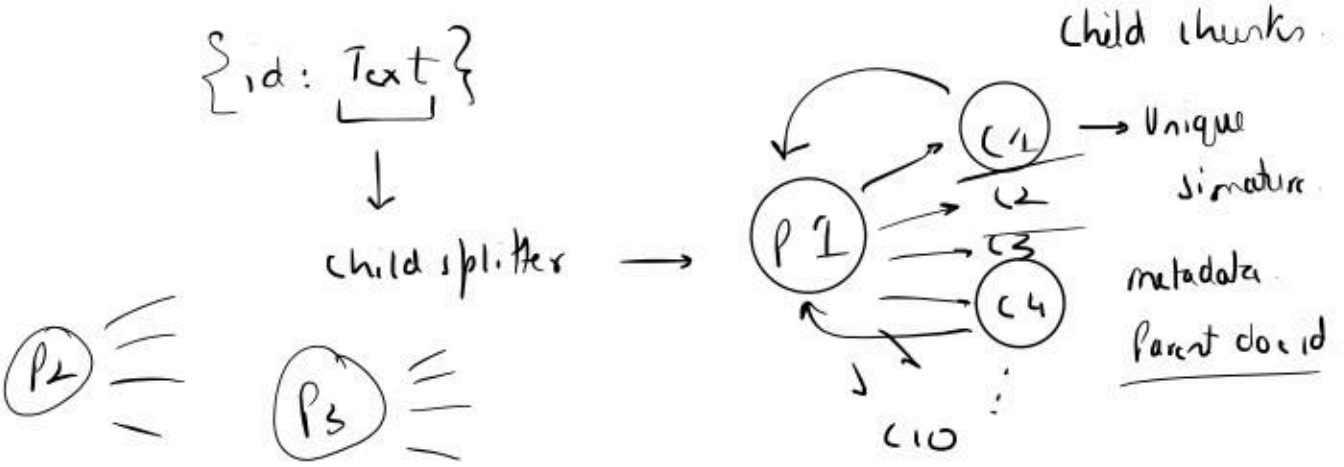
① Orginal documents → text splitter ✕

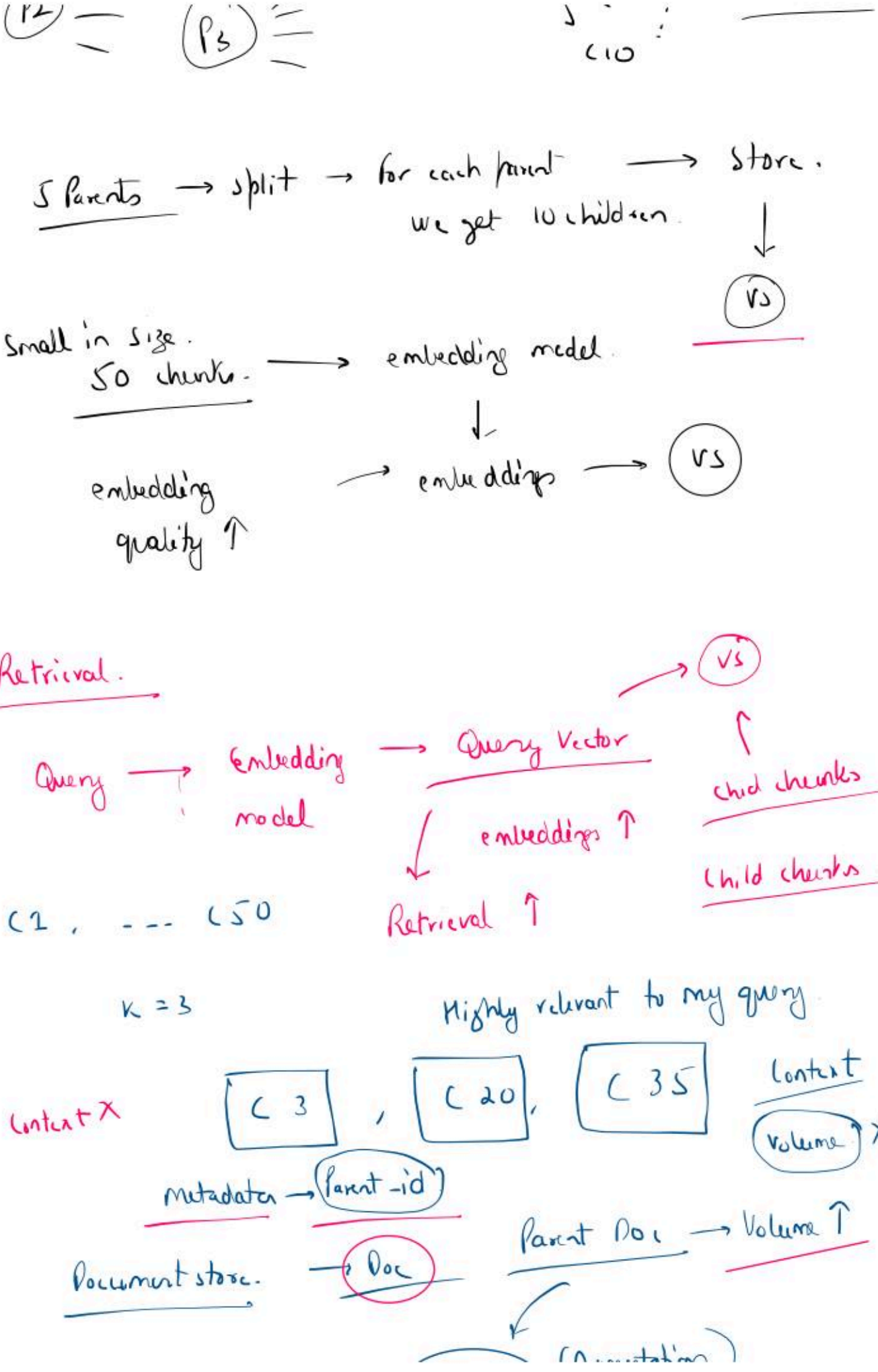
② Parent splitter → orginal docs → split them
↓
Parent chunks

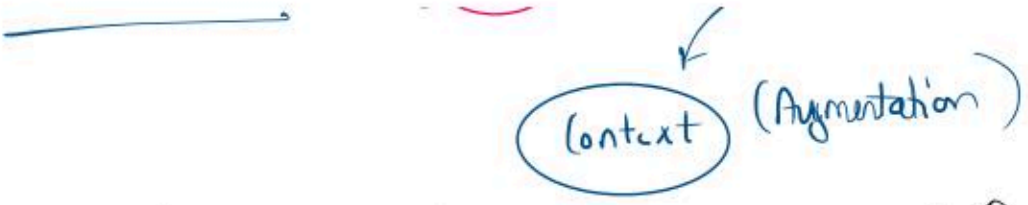
③ Parent chunks → Add them to
Document store.



④ Parent chunks → Now you pick → child text
each parent chunk splitter.
↓
Child chunks




P2
P3
C10
5 Parents → split → for each parent → store.
we get 10 children
VS
Small in size.
50 chunks.
embedding model
embeddings → VS
embedding quality ↑
Retrival.
Query → Embedding model → Query Vector
VS
child chunks
embeddings ↑
child chunks
C1, ... C50
Retrieval ↑
K = 3
Highly relevant to my query
Context X
C3, C20, C35
Context
volume X
metadata → Parent-id
Document store.
Doc
Parent Doc → Volume ↑

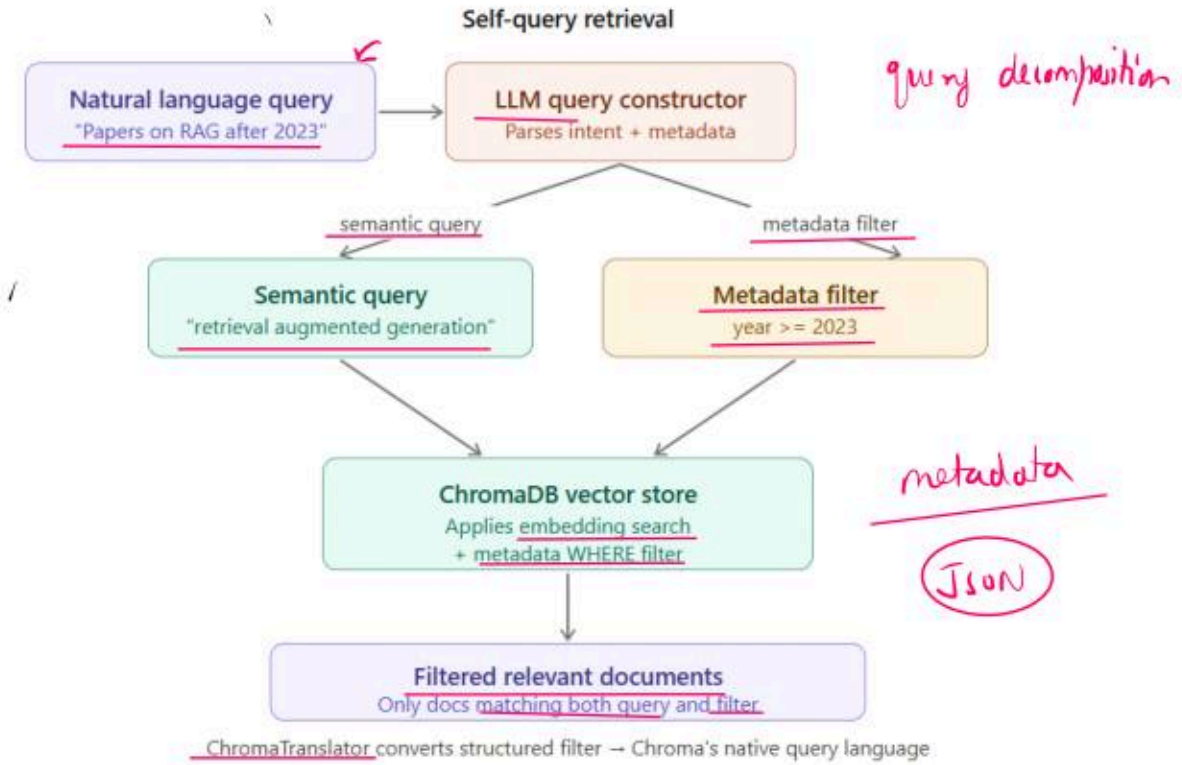


Retrieval ↑ Child chunks.

Augmentation ↑ Parent chunks.

Embedding quality ↑

Similarity search



Special scenarios

similarity search

query → free flowing text → query vector → VS

NL Text

semantic meaning of text

What are LLM's and what do they do?

similarity search (semantic search)

Scores → rank → Top k docs retrieve

Semantic meaning

More structured

Ecommerce → Amazon → 100

query vector → structured query

Ecommerce → structured query

query : Recommend 5 laptops with price under 60k and having a bigger screen

price = < 60k
specs = > 15 inch

100 → CSV

Knowledge source

price →
category →

Document loader

Row by Row

100 Docs

embedding model

Description → Page content

Metadata →

{ category : , price : , specs : }

Doc → Description

embedding model.

embeddings → semantic meaning

VS → 100

Dynamic → size, color, fabric

smart approach

retrieval → query → semantic similarity → subset of docs

metadata filtering → { category laptop , price : < 60k , screen-size : > 15 inch }

query decomposition task

query → semantic part → query

100

query → filtering part → metadata → 100 docs

similarity search (query, filter)

making transformation on yourself

Self query retriever → (SQL)

decomposing → semantic part, metadata (filtering)

unstructured query → structured query.

query → semantic part

query → metadata → structured object

LLM

Schema → metadata field info

Structured output → Pydantic Schema → JSON

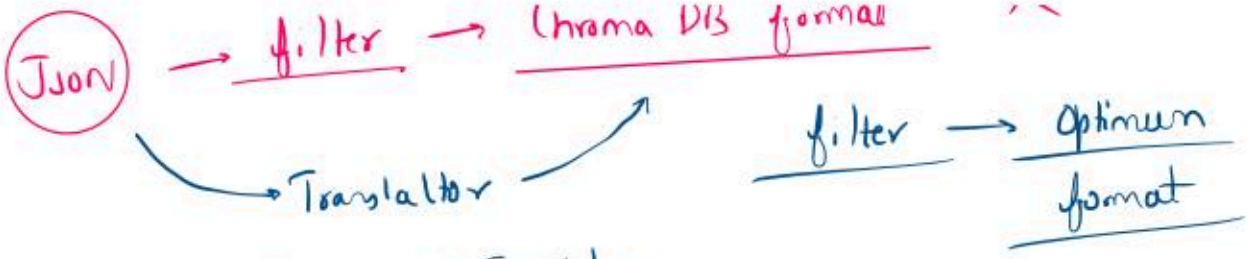
name : str
age : int
weight : float
height float

field name
datatype
description

Attribute Info

name, type, description → fetch → structured output → metadata.

JSON → filter → Chroma DB format ✗



query : Recommend 5 laptops with price under 60k and havin a bigger screen

Self query retriever → LLM → decompose.

① semantic part → Recommendations for laptops

② metadata part

↙ Operators: $ and, $ or

↘ comparators: $ gt, $ eq, $ gte., $ Lt, $ lte

filter = {"$and": [{"price": {"$Lt": 60k}}, {"screen-size": {"$gt": 15}}]

Academic

```
*"Action movies from the 1990s rated above 7"*

query_string = "action movies"

filter = {
    "$and": [
        {"genre":  {"$eq": "action"}},
        {"year":   {"$gte": 1990}},
        {"year":   {"$lte": 1999}},
        {"rating": {"$gt": 7}}
    ]
}
```

Action movies.

year = 1990

rating = above 7

2 implementations :

① Langchain

```
query_string = "action movies"

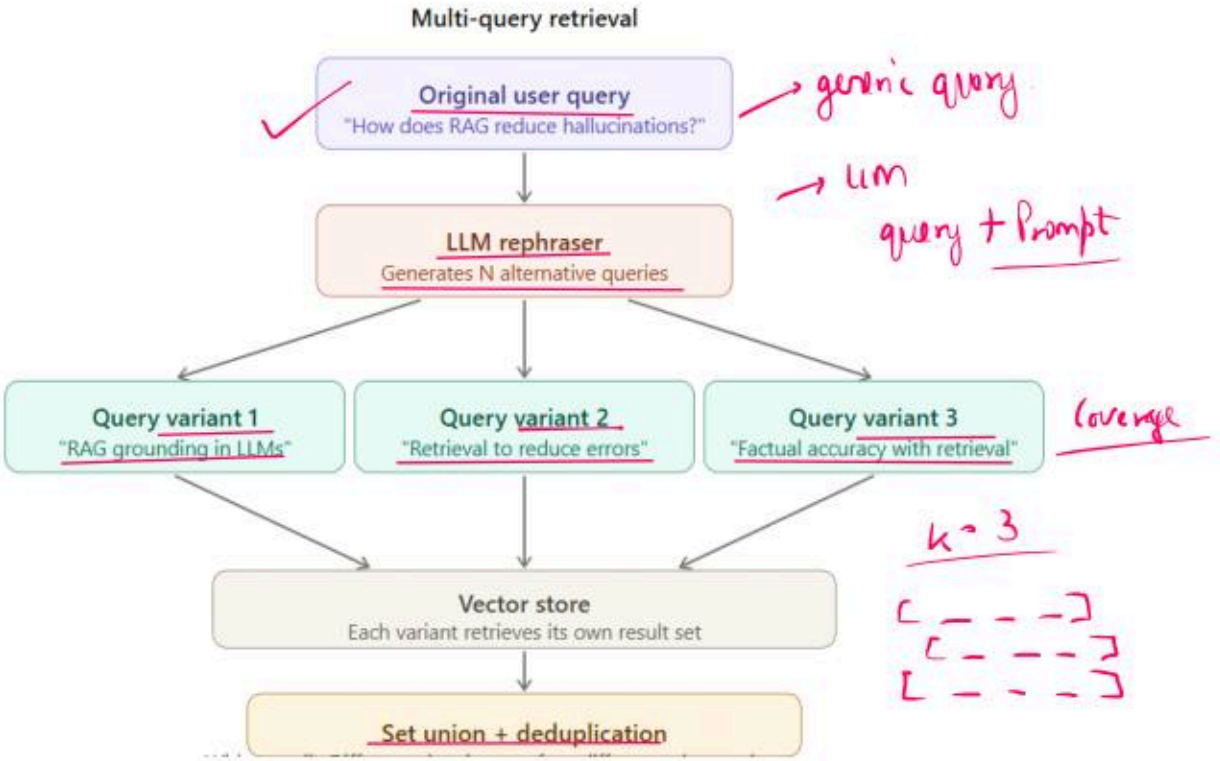
filter = {
    "$and": [
        {"genre":  {"$eq": "action"}},
        {"year":   {"$gte": 1990}},
        {"year":   {"$lte": 1999}},
        {"rating": {"$gt": 7}}
    ]
}
```

rachy = above +

2 implementations :

① Langchain

② Custom implementation



Similarity search → query

↓ semantic meaning

query vector → semantic search → retrieval.

Users → queries → generalized → specific → Multi query retriever

↓

accurate + better

simplicity

RAG chatbot → Medical Consultant → Medical Documents

↑

Knowledge base.

generic question

similarity

generic question

similarity

query How can I boost my health?

Health → Physical, Mental, Immunity

coverage ↓

Context → Physical, Mental, Immunity

generic question → Rephrase → specific

coverage ↑

embedding model → specific / nuances

Physical health, Mental health, Immunity

k = 3

Rephrase

retriever → Physical health → retrieval → [_ _ _]

retriever → Mental health → retrieval → [_ _ _]

retriever → Immunity → retrieval → [_ _ _]

Input generic query

Context → coverage ↑

3 results

3x → 9 results → Deduplicate info → Add it to the context

User experience ↑ → accurate +

user query → +

better

+

coverage high.

similarity search — query → query vector

input — 1 query → 3 queries

specific coverage ↑

query → [_ _ _ _ _] (K = 5) → 15 docs.

↓

deduplicate

+

compile → context

input → generic query

↓ → LLM

specific queries

generic → 3 specific queries

wrapper

① retriever → Base retriever → K

② LLM → model

Multi queries.

Prompt → input prompt → 3 queries

5 → 10

generic queries → coverage.

LLM → queries → API call

retrive → latency ↑

3 queries (3)

5 queries (5)

retrive → latency ↑

deduplicate latency ↑

Slow retrieval

K=3 rephrased query = 3

[1 , 3 , 6]
[3 , 5 , 8]
[1 , 3 , 8]

→ merge

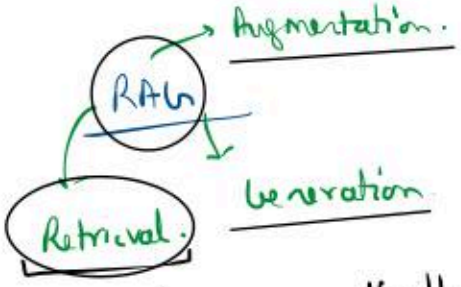
[1, 3, 6, 3, 5, 8, 1, 3, 8]

set → Unique documents

Recap

Advanced RAG → RAG fusion

Pipelines



RAG → Why
↘ What
↓ How

Components of RAG

① Document loaders. → ingest

② Chunking strategies → chunks

③ Embeddings

④ Vector Stores → Persist / Similarity search.

⑤ Retriever → Basic
sim, mmR, threshold, BM25
↓
Advanced

80% of Pipeline

Vanilla/RAG skip

Advanced Retrievers.

① Contextual Compression Retriever → Chunks → non relevant / noisy

Topic boundaries

character / token count



③

Query → Topic 2

Topic 2

LLM

↓ Extractor model.

Query + Retrieved Chunks → LLM → noisy info X

query

Retrieved Chunks → (LLM) → Noisy info ✗ model.

relevant information → context

② Self-Query Retriever → Hybrid approach

sim search (query, filter → metadata)

unstructured query

↓

LLM → structured query → ① Semantic part

→ ② Metadata Part

Schema → Attributes (optional) → metadata.

↓ Rephrase → intent

LLM → Structured Query → JSON

similarity search

Translator

↓

Retriever friendly language.

③ Parent Document Retriever → Tradeoff

embedding / Retrieval quality vs Volume of info

Chunk ↑ → embeddings ↓

|

Similarity ✗

Chunk ↑ ⟶ embeddings ✗

↓

Retrieval quality ↓ ↗ Similarity search ✗

Parent Chunks ⟶ Doc Store (K:V) id key text-value {doc-1 : ——— }

Parent Chunk → C1, C2, C3, C4

Child chunks ↓ ⟶ metadata

↓

embedding model ⟶ VS

Query ⟶ Query vector

↓

sim search ⟶ Top 3 child chunks

Retrieval quality ↑, Volume of info ↑ coverage ↑

↓

metadata ⟶ Parent doc id

↓

Doc store. ⟵ Parent chunks ⟵ final context

(4) Multi query retriever ⟶ LLM ⟵ query

↓

Rephrase into multiple queries

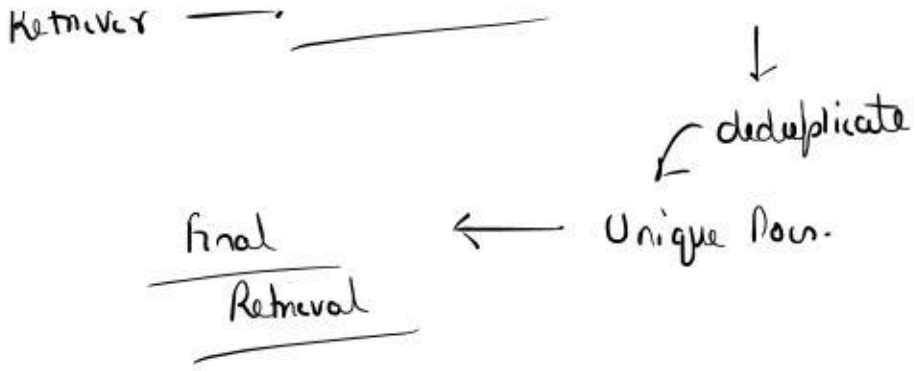
↓

coverage ↑

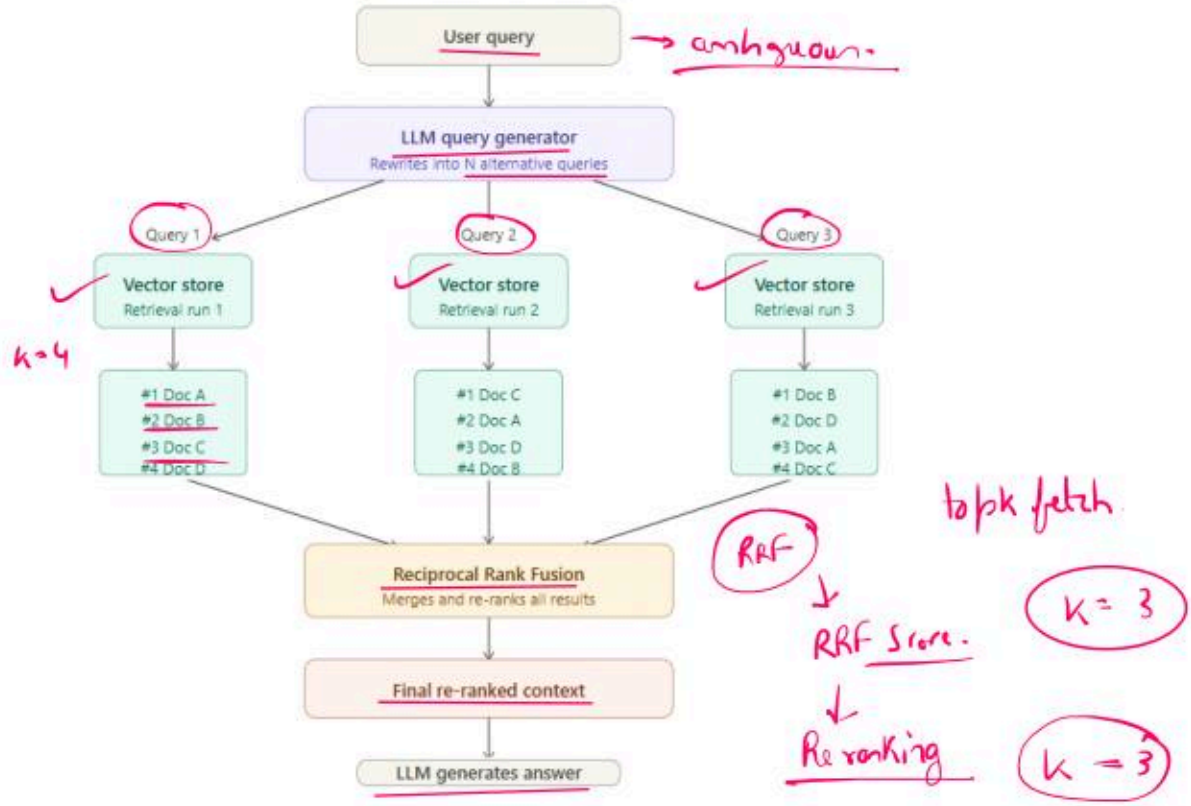
(3) K:5

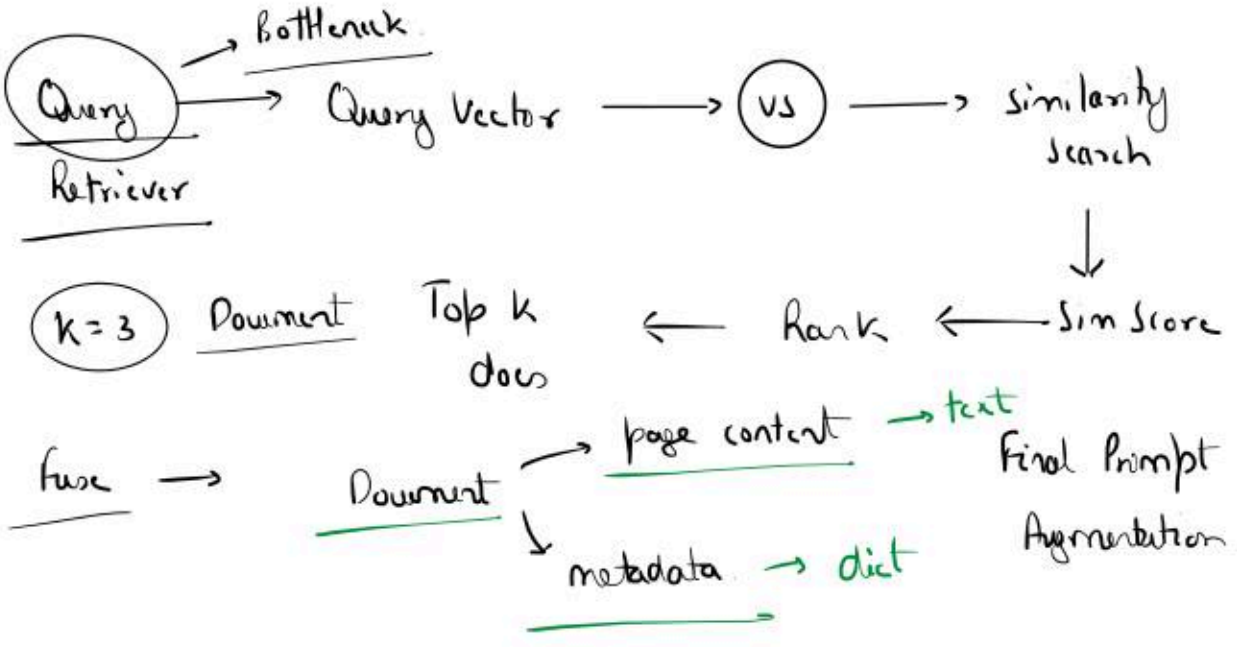
Retriever ⟶ Retrieval. ⟶ 15 docs

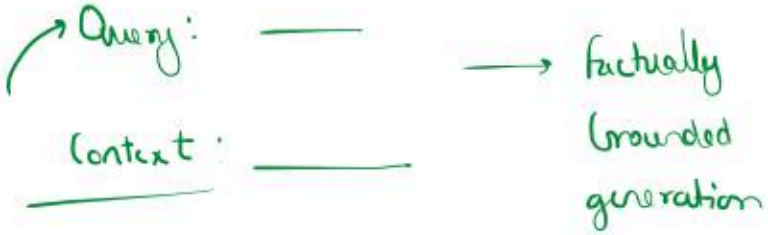
↓


Retriever
deduplicate
Final Retrieval
Unique Docs.

What is RAG Fusion







Augmentation → Text → merge → content

Prompt → query → LLM → generate response
Prompt → context → LLM

query → output
query → User
query → query vector → VS → Retrieval.

Similarity search → retrieval.

we are dependent on the user to ask the right questions / Use the right words.

Accuracy

knowledge base

RAG Pipeline.

User

Multi Query Retriever

Input query → Rephrase

coverage ↑

Query → LLM → Rephrased query → Retriever → Sim search

Retrieved docs → combine → deduplicate → Rank x

Doc 1, Doc 2 Doc 5

↓ Rank x Doc 1, Doc 2 Doc 5

(3) Retrievals (1)

[Doc 2, Doc 5, Doc 1]

[Doc 3, Doc 2, Doc 1]

[Doc 2, Doc 4, Doc 3]

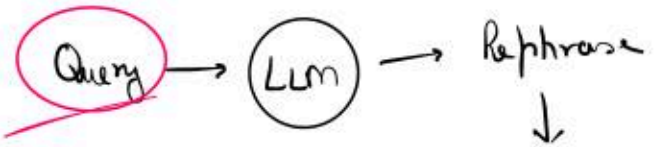
[Doc 2]]

↗ deduplication

RAG Fusion.

① 1st step → Multi Query Retrieval.

② 2nd step → Reranking → RRF (Reciprocal Rank Fusion)



Coverage

↓

Words → Retrieval.

3
5

↓

Retrieved docs.

Deduplication ← ×

15 docs.

→ Similarity

Ranking strategy → ReRank. → RRF

Reciprocal Rank Fusion.

Rank of your retrieved doc

$$\text{RRF Score} = \sum \frac{1}{\text{rank} + k}$$

$k = 60$

$= \frac{1}{\text{rank} + 60}$

3

RRF → High score. → Consistent → score ↑

Consistent × → score ↓

Rank Similarity

Merge / use RRF

③ →

① [Doc 2, Doc 5, Doc 1]

② [Doc 3, Doc 2, Doc 1]

③ [Doc 2, Doc 4, Doc 3]

$= \frac{1}{\quad} + \frac{1}{\quad} + 0$

$$RRF = \sum \frac{1}{rank + 60}$$

Doc 1 $= \frac{1}{3+60} + \frac{1}{3+60} + 0$

Doc 2 $= \frac{1}{1+60} + \frac{1}{2+60} + \frac{1}{1+60}$

Doc 3 $= 0 + \frac{1}{1+60} + \frac{1}{3+60}$

Doc 1: 1/63+1/63=0.0317

Doc 3 = 1/61+1/63=0.0323

Doc 2: 1/61+1/62+1/61=0.0489

Rerank → [Doc 2, Doc 3, Doc 1]

Consistency → Present most of the time covering all of the info

$\frac{1}{rank + 60}$

1/61=0.0164 ✓

1/62=0.0161 ✓

1/63=0.0159 ✓

1/64=0.0156

1/65=0.0154

Scores ↓ rank

$\frac{1}{rank + c}$ → 60 denominator ↑

Value ↓ Rank

Factor ↑ 100

1/101=0.0099

1/102=0.0098

1/103=0.0097

$\frac{1}{Factor}$ ↓

1/2=0.5

1/3=0.3333

Rank ↑

1/2=0.5

1/3=0.3333

1/4=0.25

Rank ↑

Rith fusion

Query → Multiple → Retrieval

Versions

↓

Fuse / Merge

↳ RRF scores → Top k doc.

retrieve.

RRF

Reciprocal Rank Fusion

$$\frac{1}{rank + c}$$

Query → Accuracy + Coverage.

Simple to apply → RAF

Intuitive / Easy to understand.

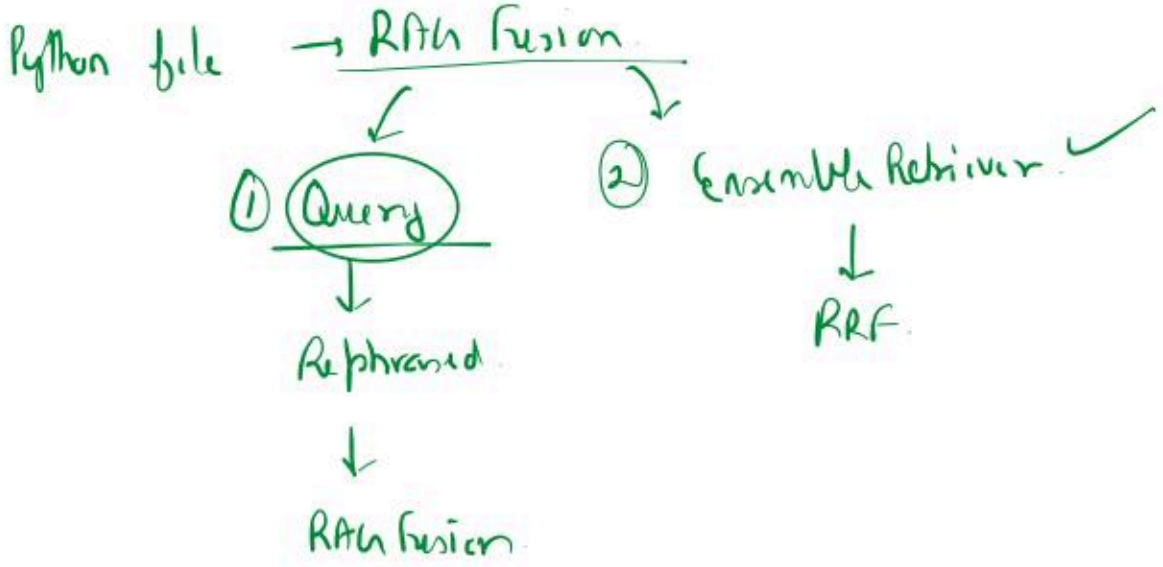
3

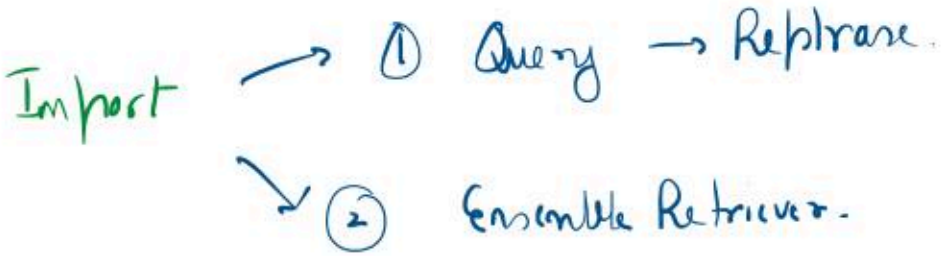
① API calls → LLM

② Latency →

③ Cost →

Code


Python file
RAG Fusion
1 Query
2 Ensemble Retriever
RRF
Rephrased
RAG Fusion


Import
1 Query → Rephrase
2 Ensemble Retriever.

HyDE Hypothetical Document Embeddings

RAG Fusion → Query → Retrival quality.

↓ Multiple versions → LLM

2 Phases

① Generate Multiple Queries → Multi Query Retrival.

② Retrieved results → Fuse / merge → RRF (Reciprocal Rank Fusion)

sort ⇐ Top k highest scoring docs

Query → Not well defined

Misses a lot of imp keywords

Simple

Not detailed

Ambiguous.

Query

↓

Not Detailed

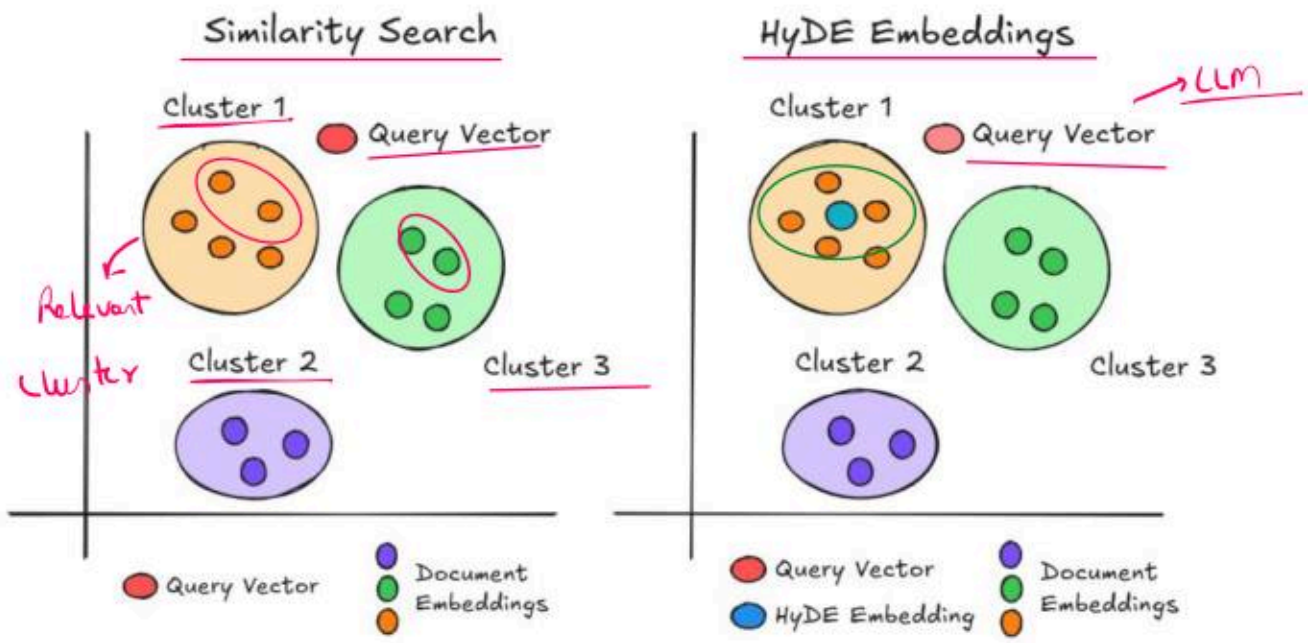
Simple. → Similarity

detailed ✗

↓

Docs - detailed

Apples to oranges



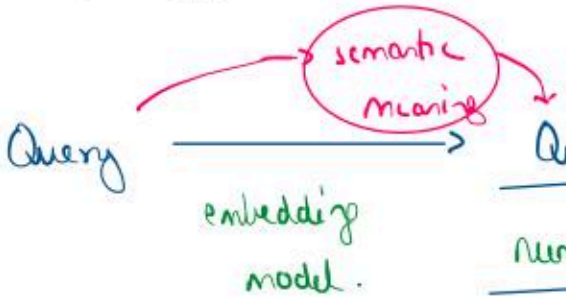
Query → Well defined ✗ → (VS) → Similarity search

Detailed ✗

Query → weak

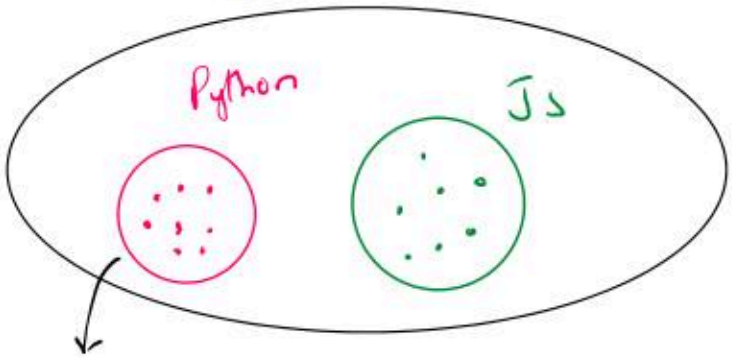
↓

↑ low quality



Well defined text → semantic meaning ✗ embedding space

Ill defined text → semantic meaning



Programming lang

Query + Doc } Similarity

Query Vector

similarity

semantic meaning

LLM → NLU → query intent

LLM → NLG → hypothetical doc

Query vector

Completely diff.

Relevant Cluster

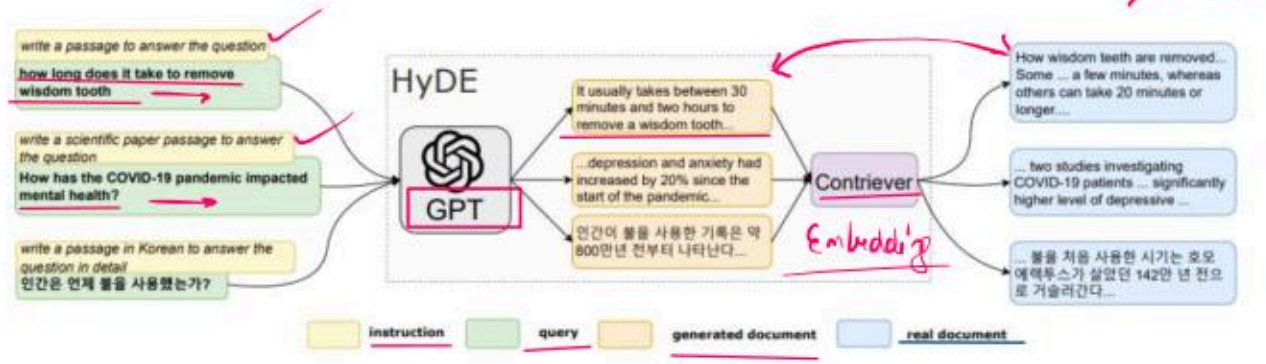
→ Quality ↓

Accuracy ↓

Well defined text + detailed → embedding vector.

VS

2022



doc + doc } similarity

Query + Docs } Weak Match

Hyp Doc + Doc } Better Match

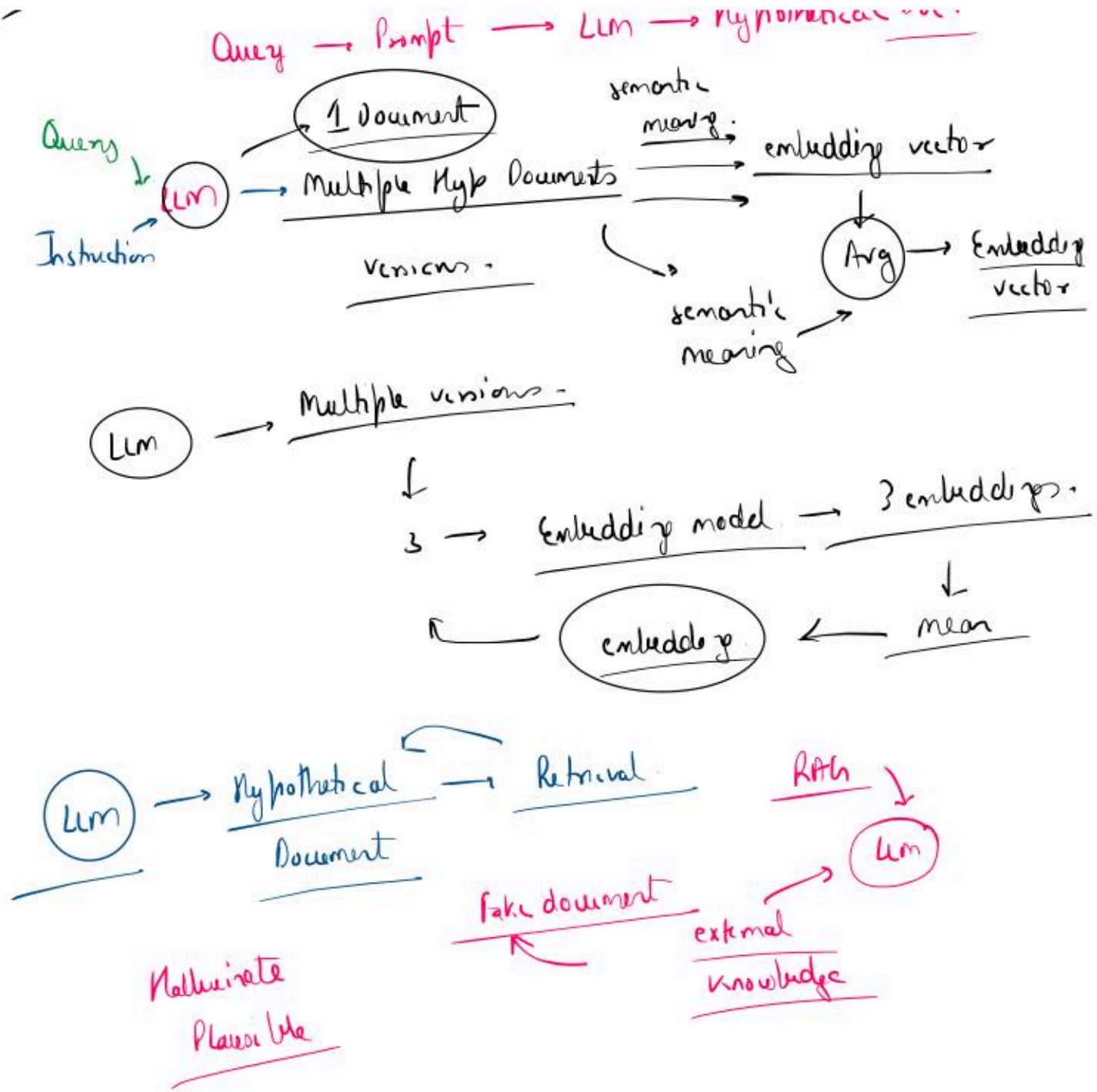
HyDE → Custom Implementation. → Notebook

↓ HyDE ← RAG Pipeline

Replace Hyp document

Query → Vector → VS → Retrieval.

Query → Prompt → LLM → Hypothetical Doc.


Query → Prompt → LLM → Hypothetical
Query
Instruction
LLM
1 Document
Multiple Hyp Documents
versions
semantic meaning
embedding vector
Avg
Embedding vector
semantic meaning
LLM
Multiple versions
3 → Embedding model → 3 embeddings
mean
embedding
LLM
Hypothetical Document
Retrieval
Fake document
Hallucinate
Plausible
Both
LLM
external knowledge

Agentic RAG

CRAG

Self-RAG

Traditional RAG

Query → Query vector → VS → Similarity search → Top k Docs retrieve.

Simple to implement

Latency

Cost

Control — decision making

Dumb pipeline — LLMs smart

results → check / verify

CRAG → evaluator → Relevancy score.

Selective

query + retrieved docs → score. → final context

Web search

knowledge refinement → refine

Self RAG → Self Reflective RAG.

Query

Smart approach

decision making

↓

LLM

1. Whether we need retrieval ?
2. Relevancy of retrieved docs ?
3. Response → grounded / not ?
4. Whether response → query ?

↓

Useful

→ Order does not change

Agentic RAG → upgrade

AI Agents

Evaluator → LLM.

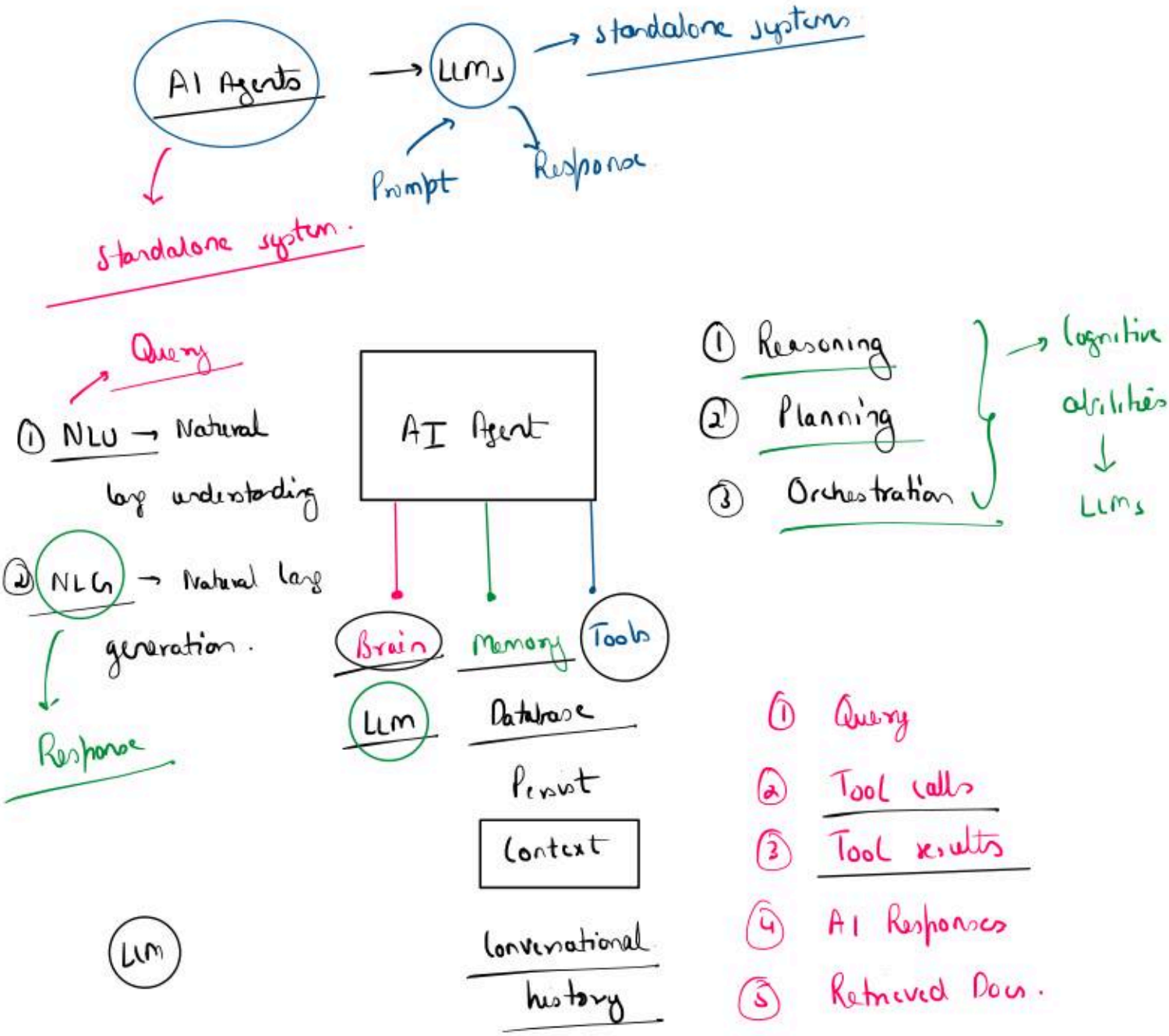
Decision → LLM

1. Autonomous → decision making → autonomous
2. Adaptive → Retry, Try multiple sources, knowledge refinement
3. Dynamic → Breakdown, Rewrite.

↓

generation prompt → Tweak.

What are AI Agents?



Properties of AI Agents -

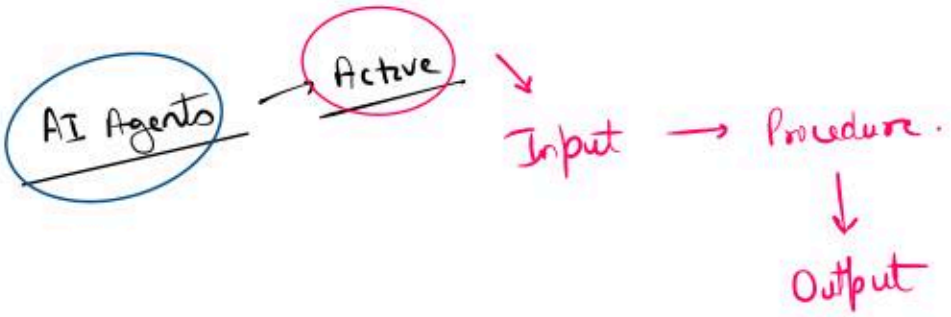
① Autonomous / Autonomy → Decision Making

② Planning → Series of steps → Actions
Roadmap

③ Reasoning → Thinking → Execution of each step
How
Query → Ambigous
↓
Rewrite / Multiple Queries

④ Orchestration → Planning / Reasoning

⑤ Adaptability → Dynamic, Robust, Error free.



ReAct Loop

Re → Reasoning Reasoning + Action.

Act → Action.

Retrieval

① Reasoning / Thinking → Decision making for the step

② Action → Implementation.

↓ Result

③ Observation. → Action step → Observe.

① Exit the loop

② You retry again

Reasoning → Action → Observe.

↓

Observe ← Action ← Reasoning

} ReAct Loop

Traditional RAG

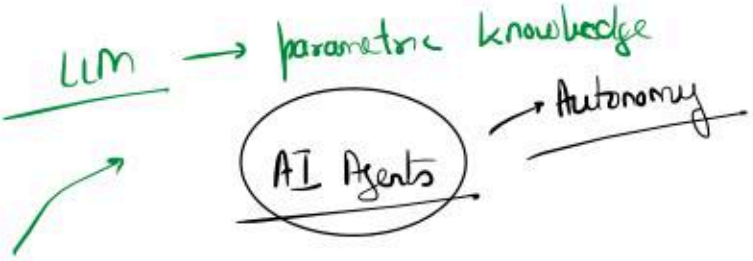
① control on retrieval. → Reason

② Order is static → dumb approach → Action

ReAct Loop

③ No verification of results

Agentic RAG



IF → Whether to do retrieval or not

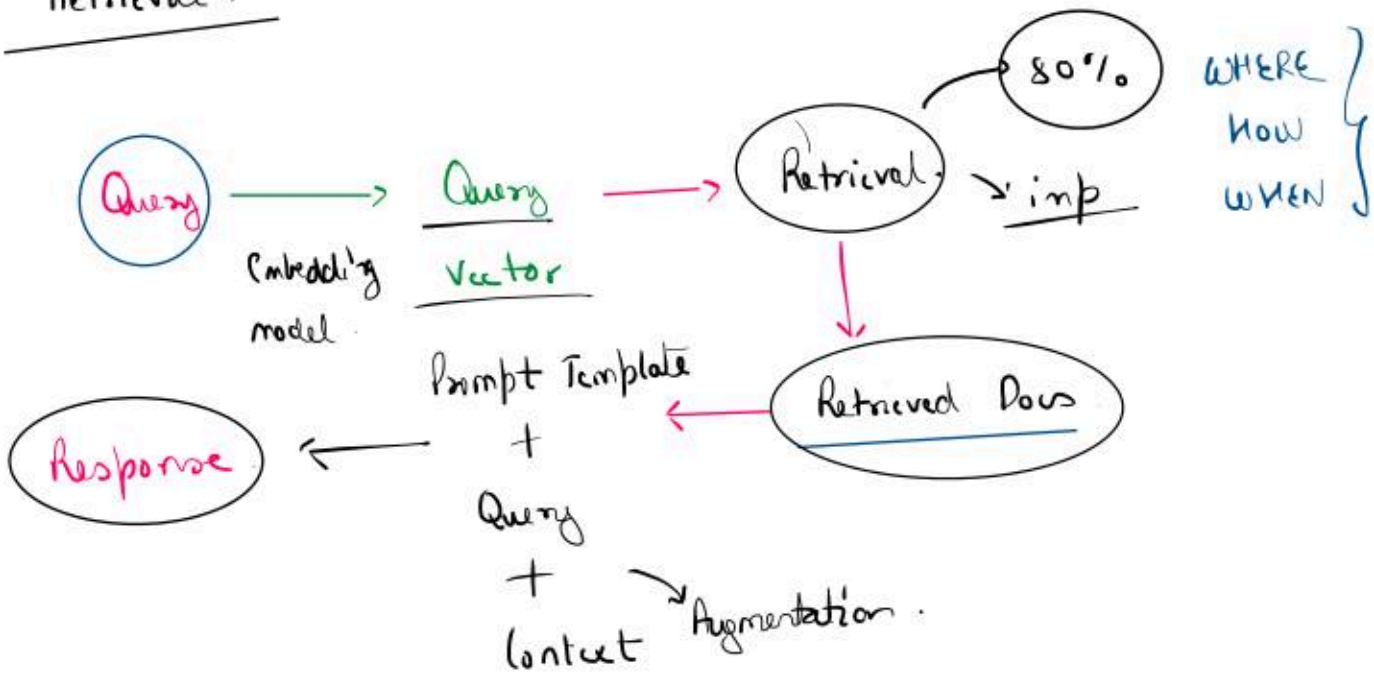
WHERE → Where to do retrieval from (tools) → Similarity search

How → How to do it (tools)

k →, metadata filter → fields.

WHEN → Dynamic workflows

Retrieval.



① Query → Input

① Check for retrieval.
② Rewrite / Rephrase.

① Whether retrieval is required or not? ③ Query decomposition.

AI Agent → Reasoning

Query → Intent, Parametric knowledge → Answer

↓

Response

② Query is ambiguous.

↓

Rewrite, Multiple versions.

③ Query is complex

→ Compare Python and JS and give me a comparison report

Decompose or break down

① Planning
② Reasoning

① Python
② JS

Multiple Retrievals

③ Comparison

④ Response

HOW

WHERE

Source of Retrieval → dynamically

Retry logic

Query → leave policy of company

VS → company related docs

Query → Web search → Retrieval

tool → Source of retrieval → query

Retriever → tool define → Bind

HOW → Retrieval from selected source.

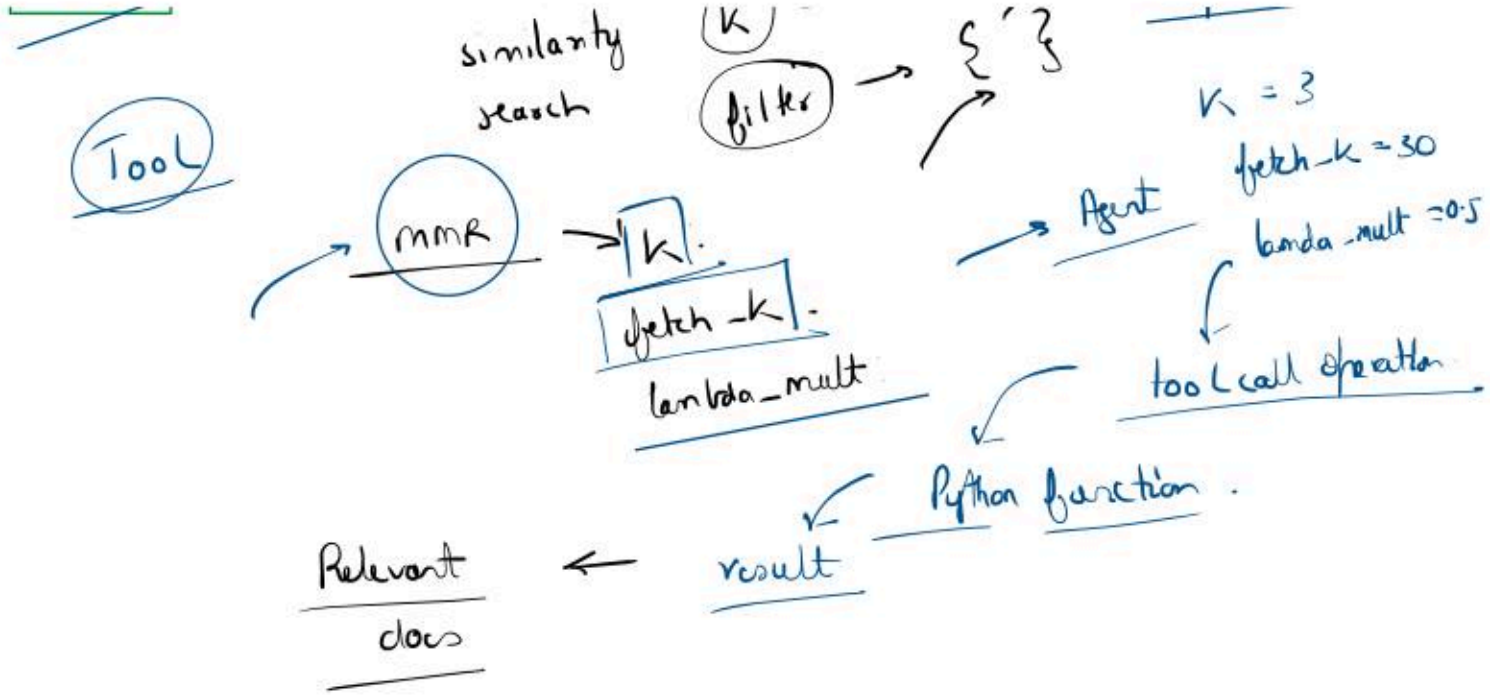
Tool schema

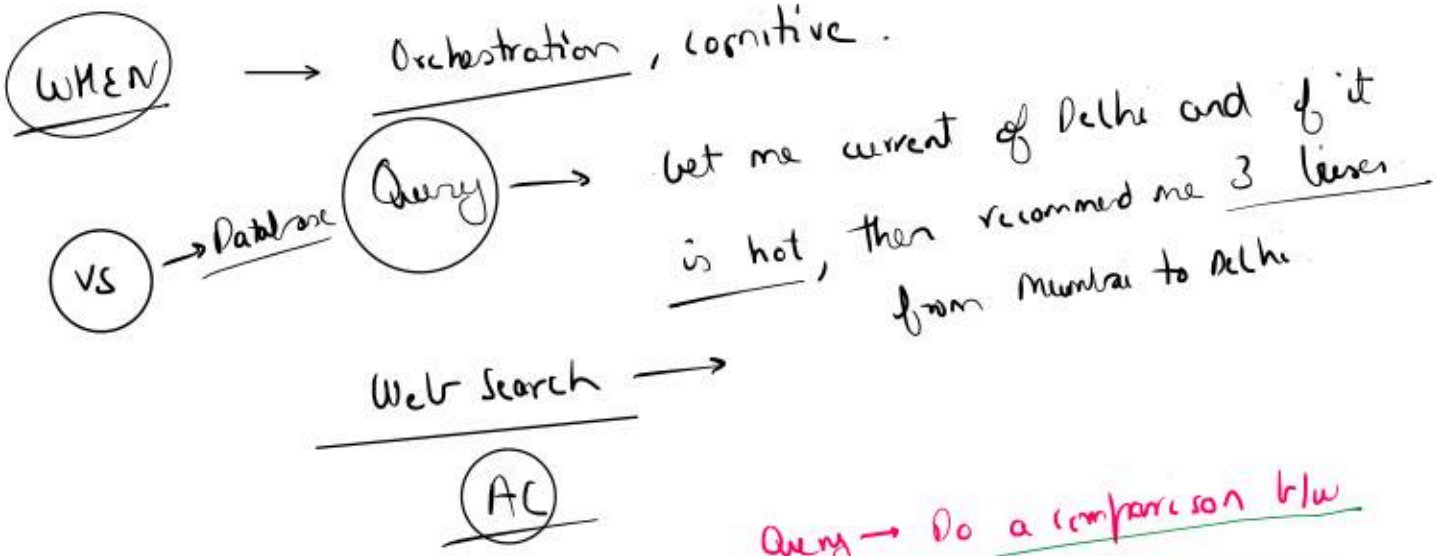
similarity

K → ③

parameters

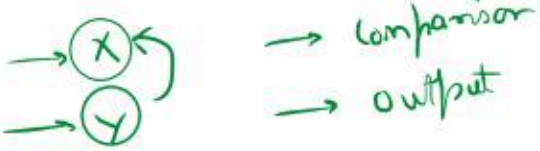
K = 3





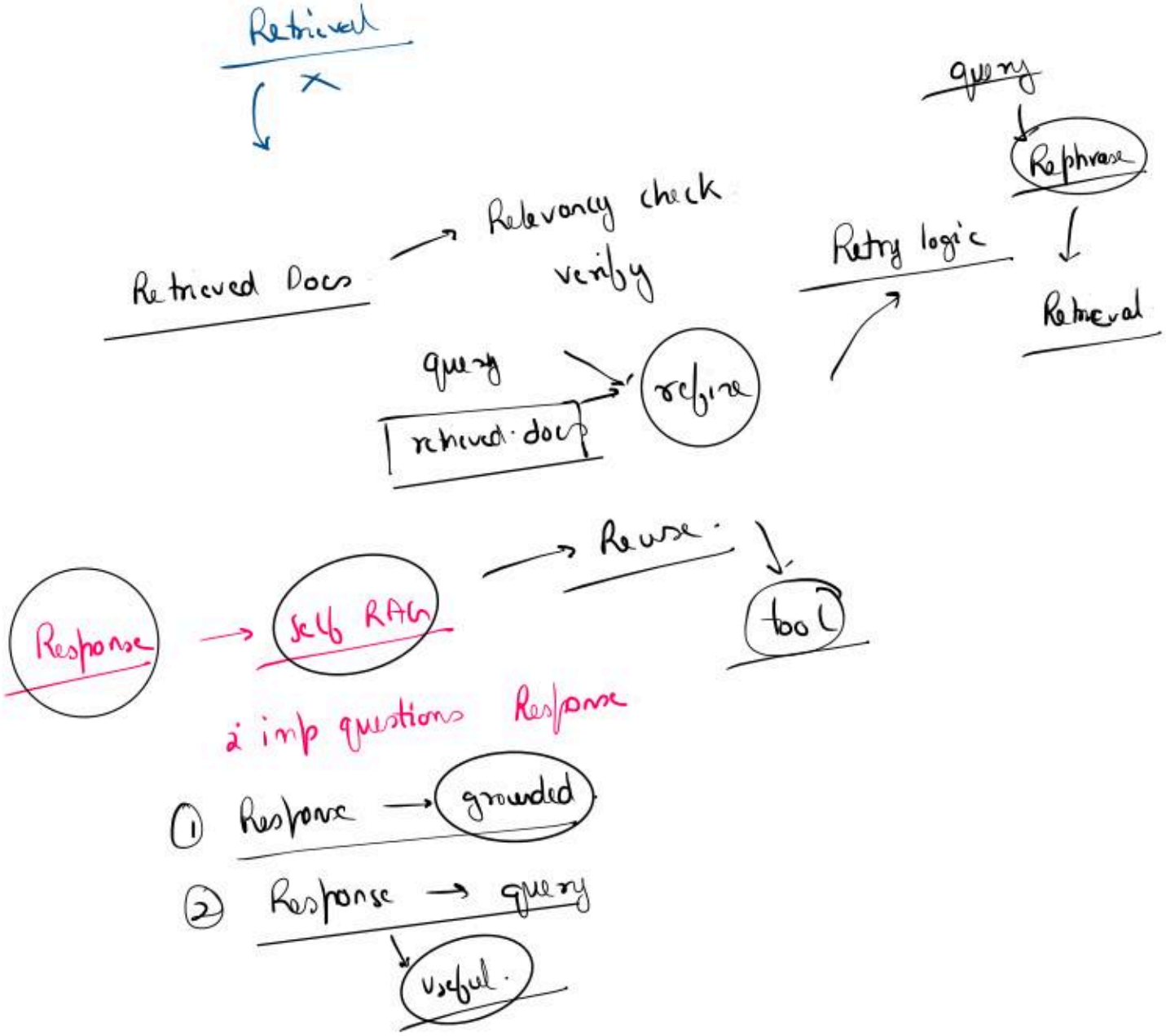
Decompose

Query → Do a comparison b/w X and Y and show me the comp report



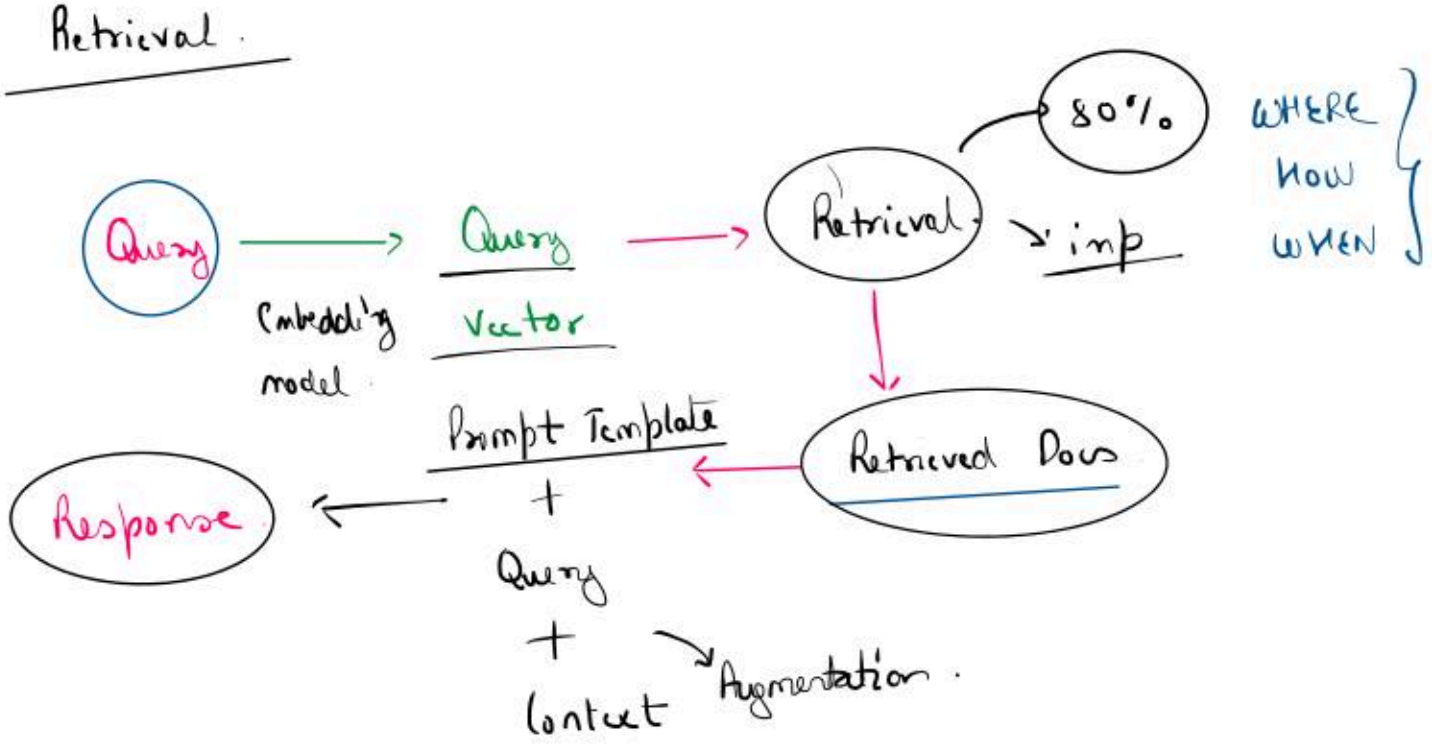
Retry Logic → Multiple Sources

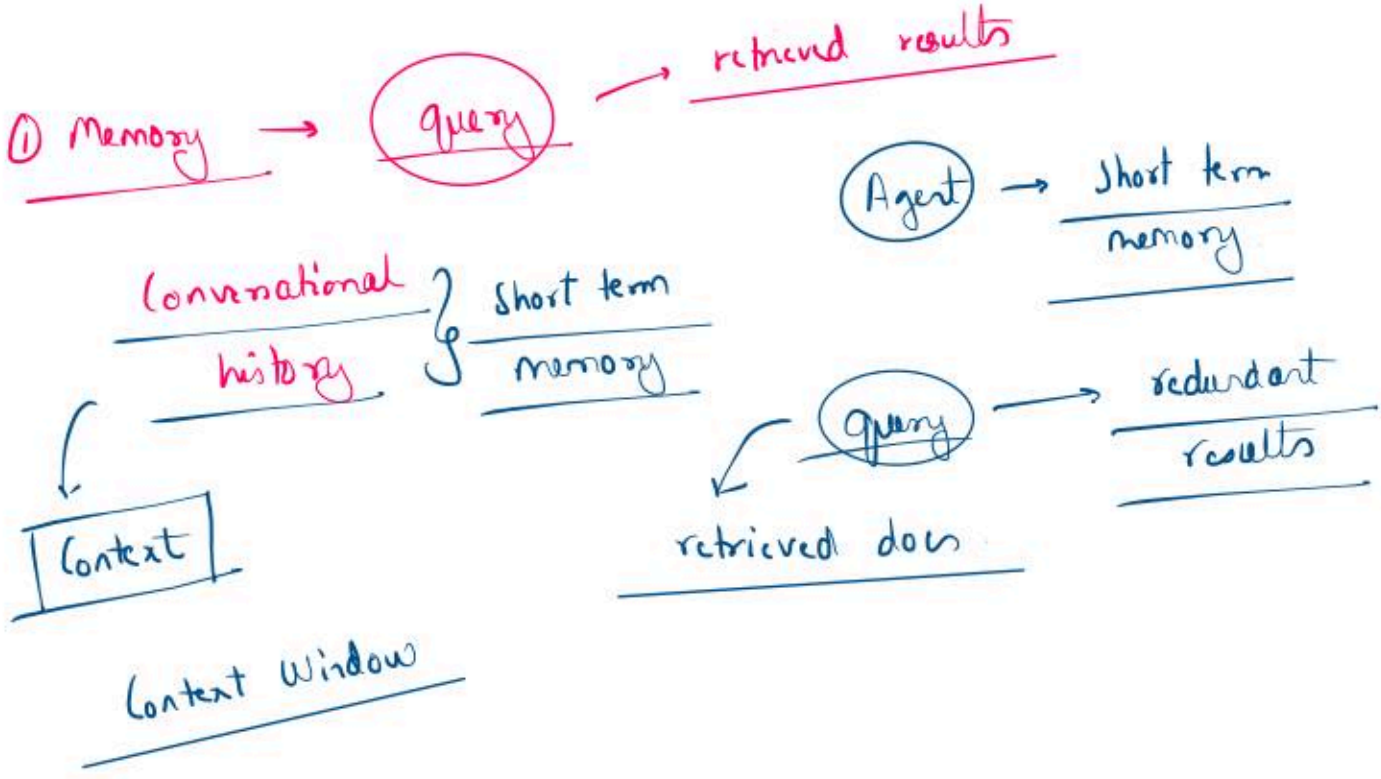
Retrieval

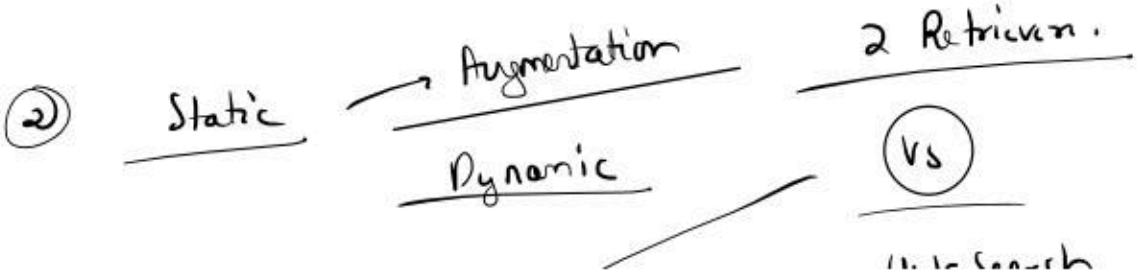


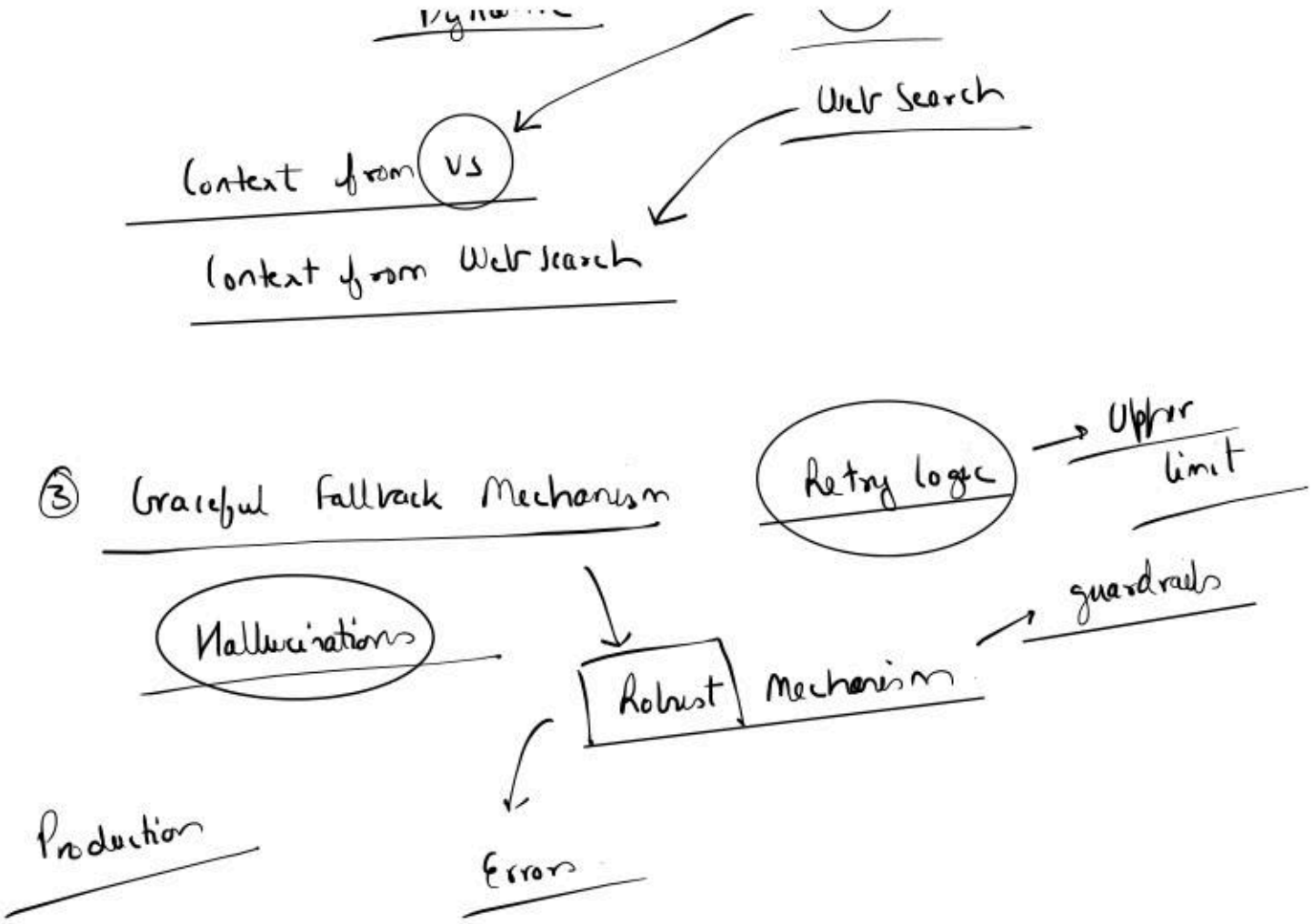
1) Select Retrieval
2) Query Decomposition.
3) Query Rephrasing / Rewriting
4) Source of Retriever → Tools
5) How to do retrieval.
6) Have to perform any retries
7) Relevancy check on retrieved docs.
8) Response → Verify
   - grounded
   - useful.

Properties of Agentic RAG(good to have)

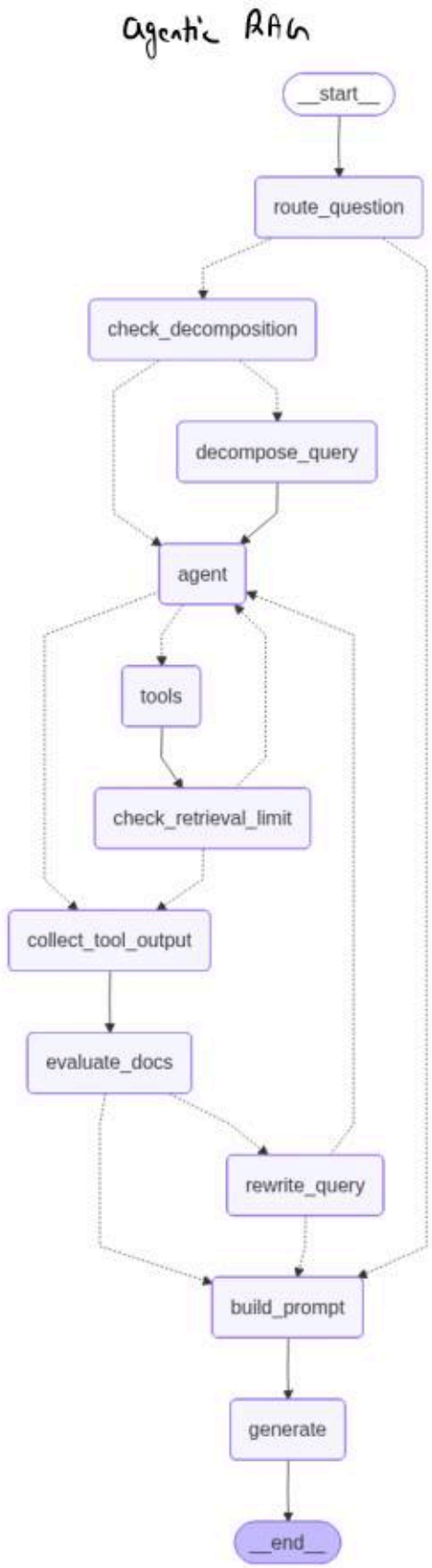




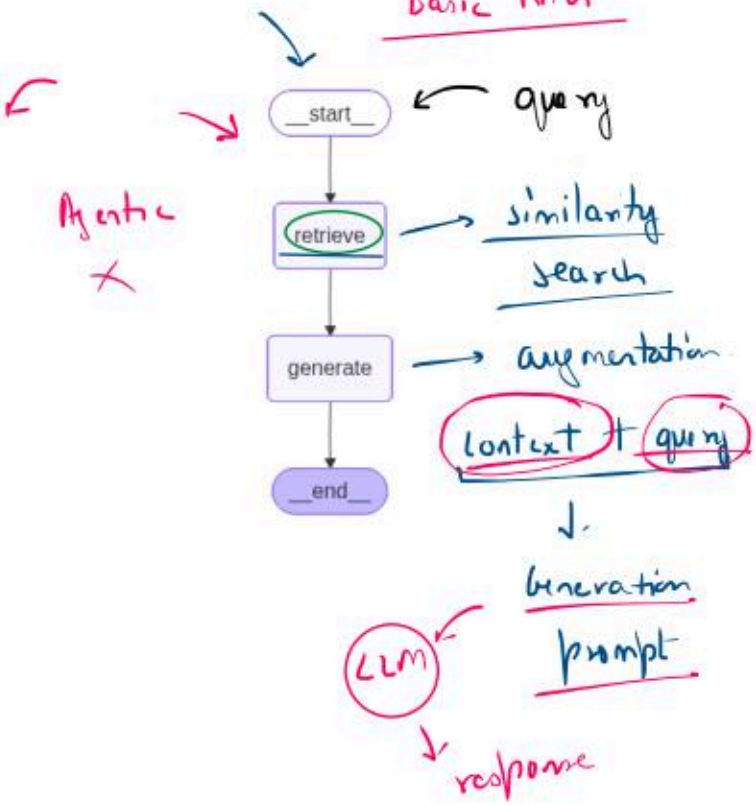



Web Search
Context from VS
Context from Web Search
3 Graceful Fallback Mechanism
Retry logic
Upper limit
Hallucinations
guardrails
Robust Mechanism
Production
Error

Code - Base RAG

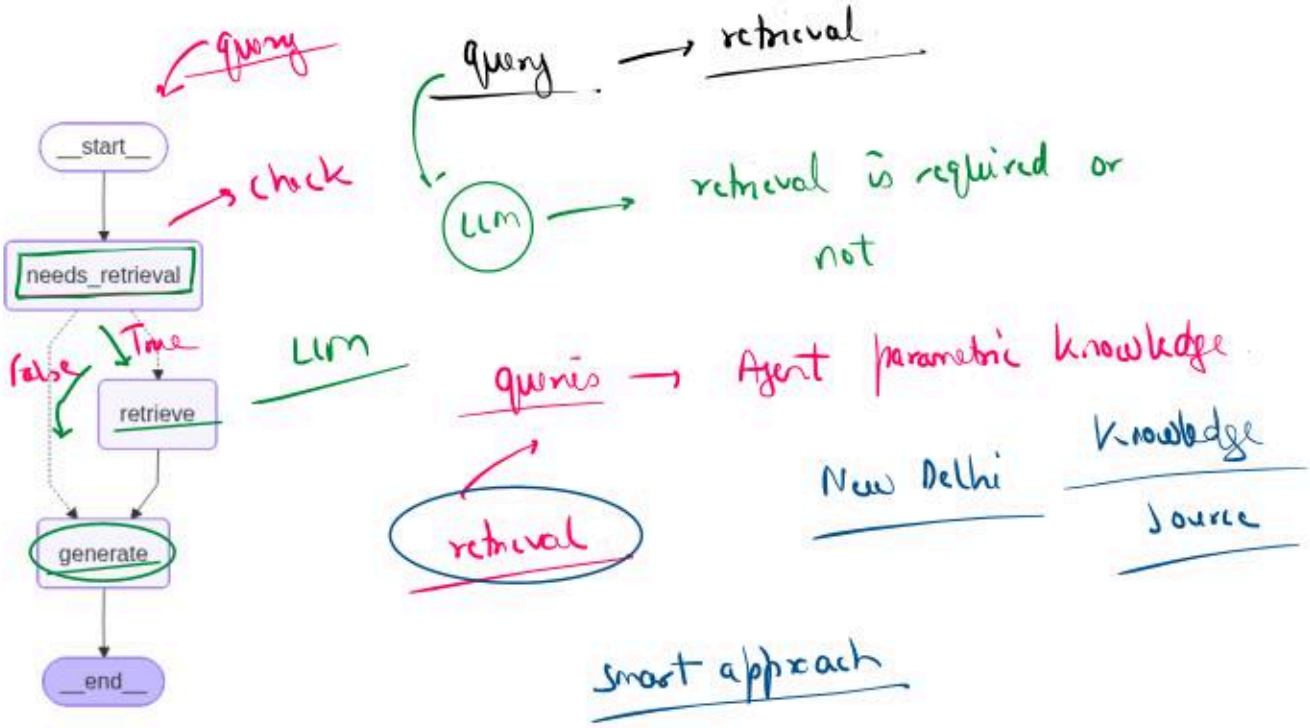


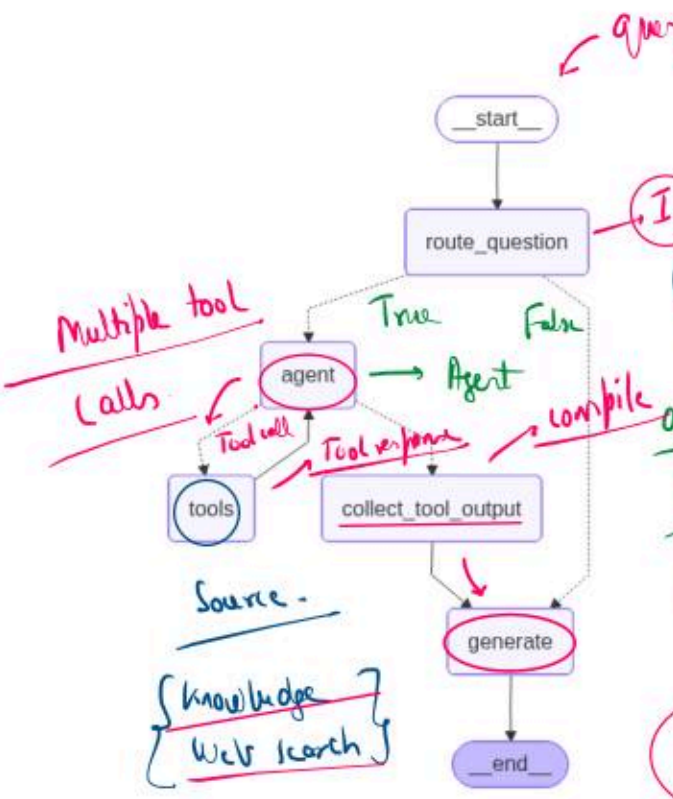
iterative.



langgraph

1 - Retrieval Decision





IF → Should we retrieve ?

WHERE → Source of Retrieval.

dynamic + smart } Query

Source of Retrieval

agent → Agent

agent dynamic

Agent + Tools → Perform Actions Python functions.

Agent → Source of retrieval.

① Chroma DB → Retriever → Knowledge base

Query → Pawan info X

② Web search retriever → Web search

Query → Current weather of Mumbai → LLM internal knowledge X

How → Where to do retrieval.

Retrievers - → dynamic

Python functions → parameters → Value

k →

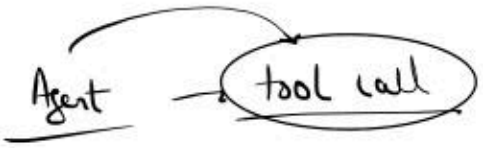
Reasoning / Logic → Tool call operation

Reasoning } Cognitive → Tool call operation
Planning }

WHERE

Web search → ① query → optimize. HOW

② results → agent → 5 1



System message → Defines the persona, role

Human message → query

tool call → function

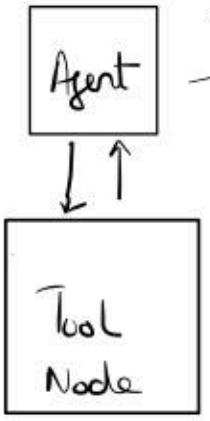
output → return

AI message → tool call

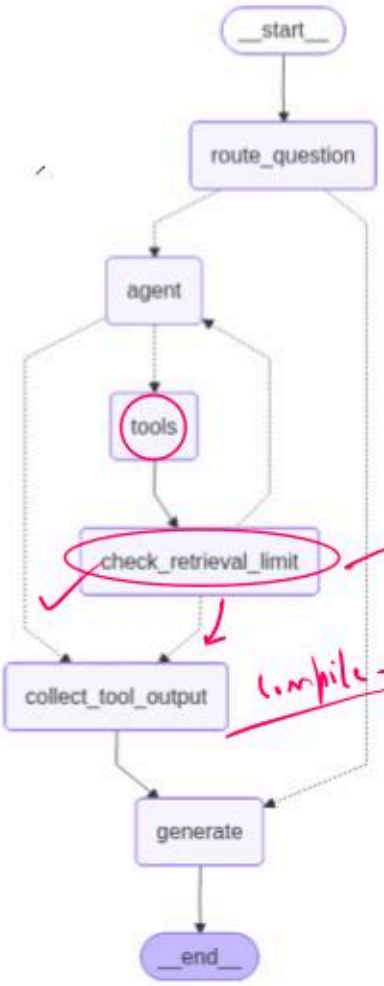
① tool name ② parameter values

Tool message → Tool call output

AI message

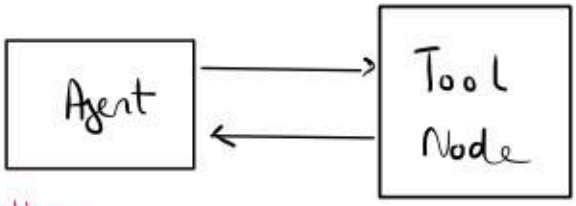


Tool output → tools define.



Query → complex

Agent → Retrieval loop.



tool calls.

Complex → Multiple tool calls

compile.

Web search
Retriever
} → Parallel.

Tool call → AI Message

Agent → Multiple tool calls.

Loop

Tokens

3 Retriever →

Retry logic → Tool Node

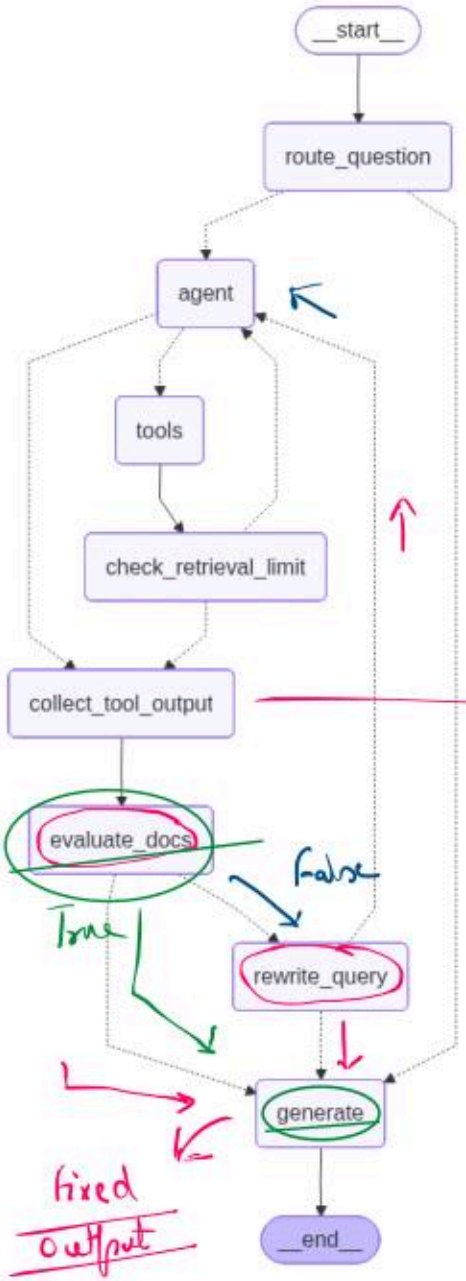
Break the loop

Fallback mechanism

tool calls → Break out of the loop

Maximum limit

4 - Relevancy Check and Query Rewriting



Relevancy check

Query rewriting

evaluate docs

compiled context

retrieved docs → compile → relevancy check

for loop

D_1, D_2, D_3, D_4, D_5 → Relevancy check

query

relevant_docs: D_3, D_5

relevant docs = [ ]

query → rewrite → rephrase

loop

Breaking condition

(2) Tasks.

(1) Relevancy check → Relevant docs → context

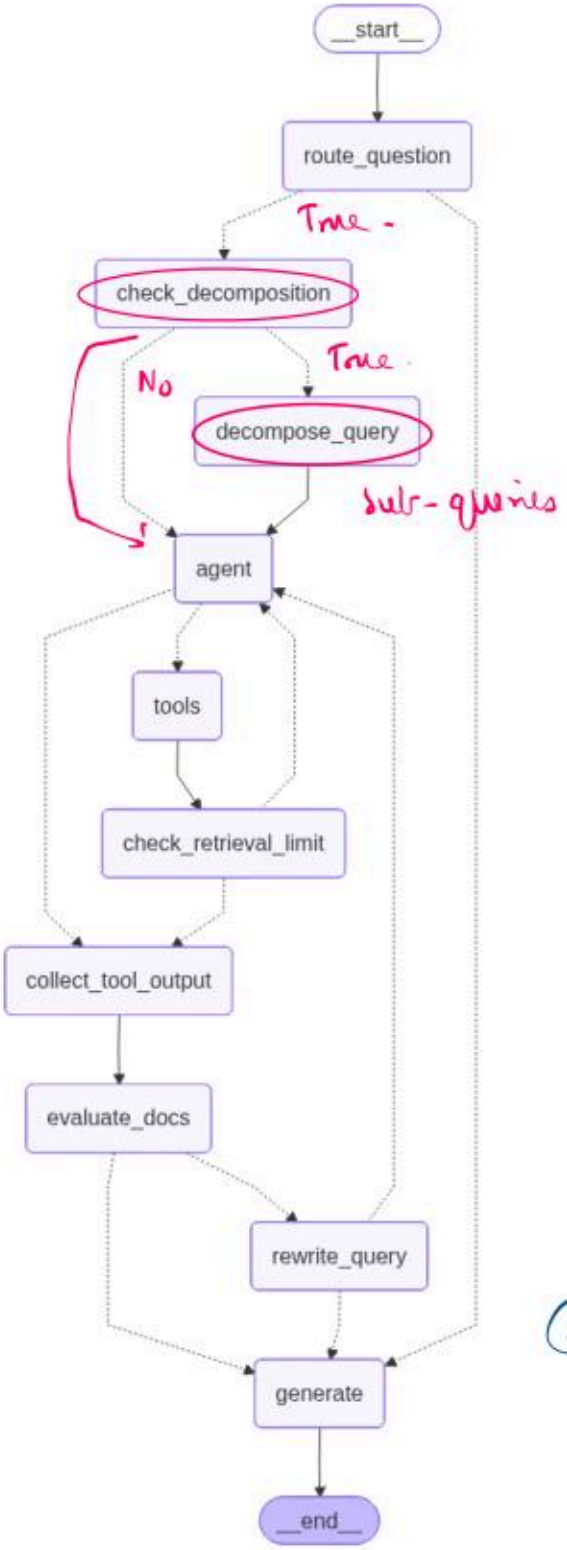
Noisy discard.

↓

Relevant docs → context

② Query → Rewrite / Rephrase.

Relevancy check.



Query → Complex

↓

Multiple Retrieval steps

Complex query

↓

Breakdown into

Simpler sub-queries → Retrieval

WHERE

HOW

Check → Query original

Complex → Yes / No

Breakdown into simple sub-queries

Use to do retrieval.

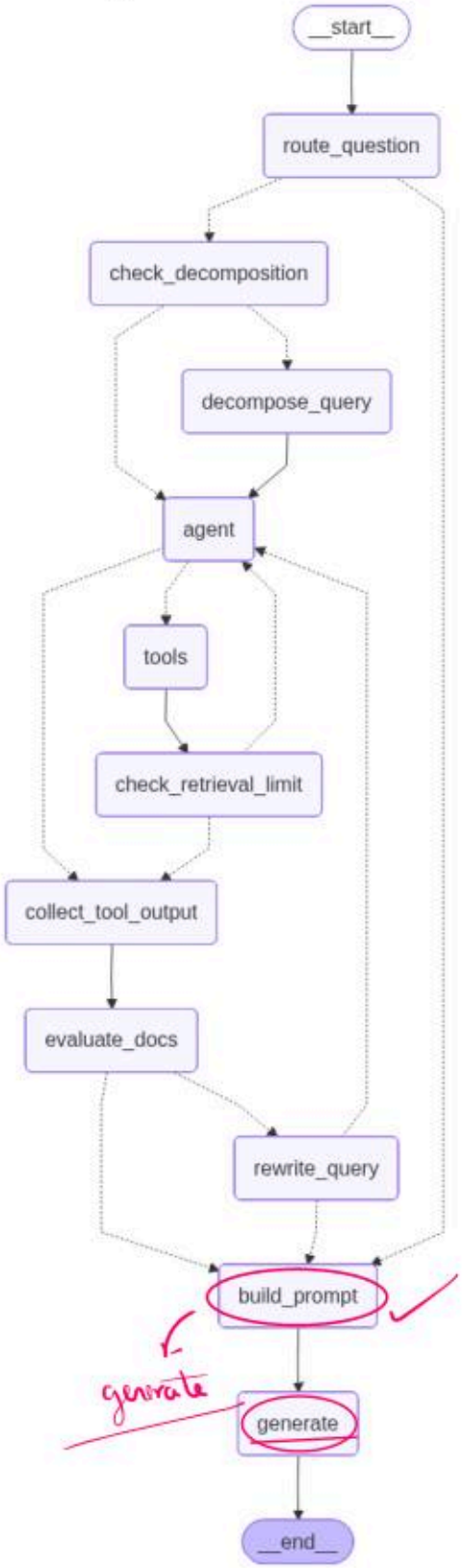
(2) Steps

1) To check whether input query is complex ✓

2) Decompose the query.

6 - Dynamic Generation Prompt

Agentic RAG



Augmentation — Template

Context + Query → Prompt → fixed.

Prompt → LLM → generation. → Response.

Dynamic

Prompt → Retrieval Sources.

Decompose.

Better response → context information.

Context → Retrieved docs.

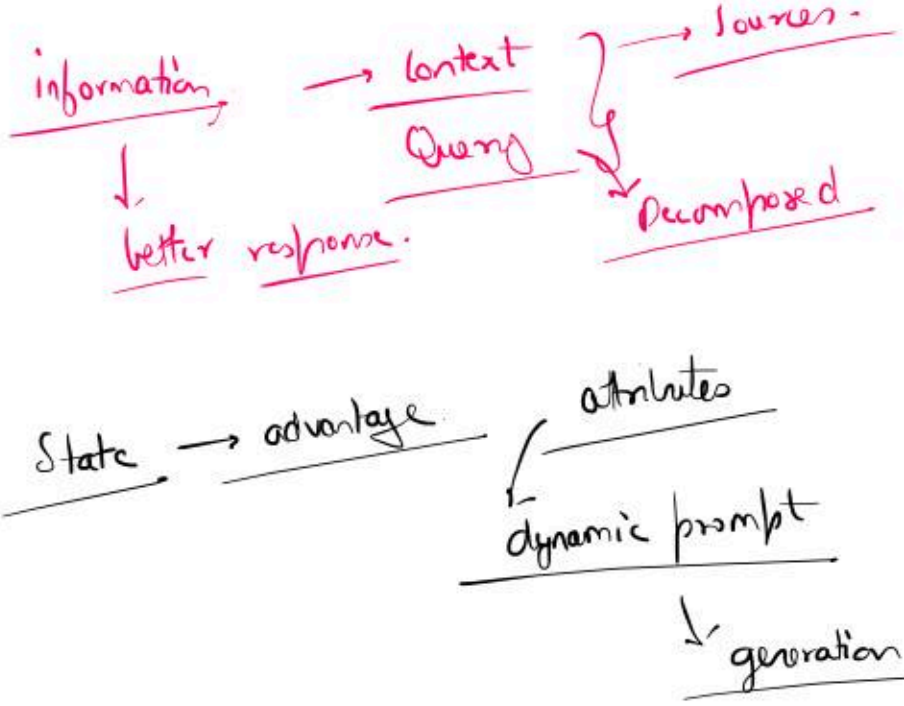
Query → Rewritten / decomposed.

Prompt → Context

VS — Web search

information — Sources information

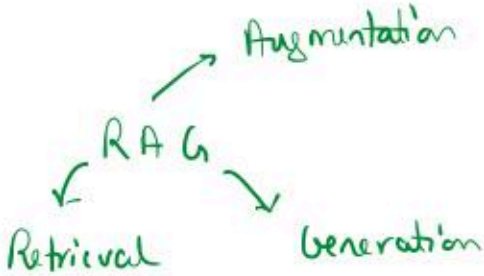
generation → letter


information
context
sources.
Query
Decomposed
better response.
State
advantage
attributes
dynamic prompt
generation.

Graph RAG

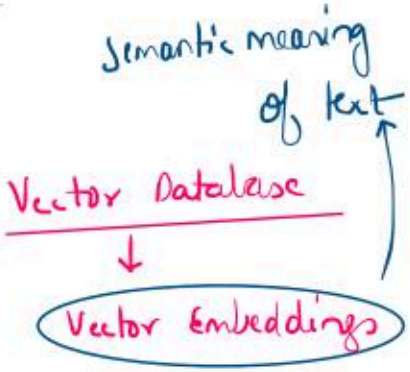
1. Traditional RAG V/s Graph RAG
2. Graphs → Concept
3. Ingestion Pipeline
4. Retrieval Pipeline.
5. Limitations.

Traditional RAG



2 steps

1. Data ingestion



a) Knowledge source → Load, Parse, text extract
   - Document Loaders
   - ↓ Unified format

b) Split the documents into chunks.
   - → Text Splitters.
   - ↓ accurate embeddings

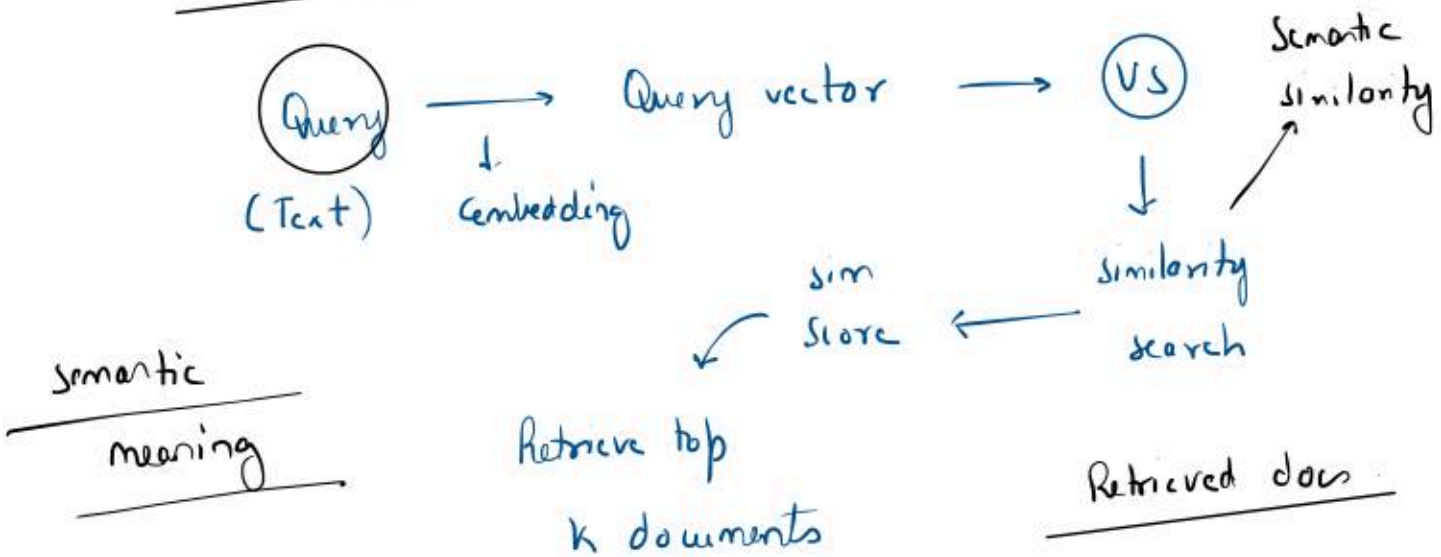
c) Embedding model → Vector Embeddings

c) Embedding model → Vector embeddings

↘ semantic meaning → numbers

d) Vector Databases → Storage + Indexing

② Retrieval pipeline



Augmentation

↓

context + Query → LLM → Grounded Response

① Foundation → Vector Database

↓

Text embeddings.

② Retrieval → similarity search → semantic meaning of text

Query → Relationships, Multi Hop questions

Eg Which company of Elon Musk is HQ in Austin? Location.

Relationship

Chunks → embeddings → VS

Graph RAG

Chunk → individual entity

↓

embed → embeddings

Query + doc - embedding

Scores

+ ↑ score

Query

Independent of each other

Graph RAG → Graphs

DSA

Advanced DS → Relationships

graph

Vector Databases → Vector Embeddings

Relationship ×

Graph → Key Property

Nodes → entities → Data

↓

Vector embeddings -

completely individual

Edges → Connecting points

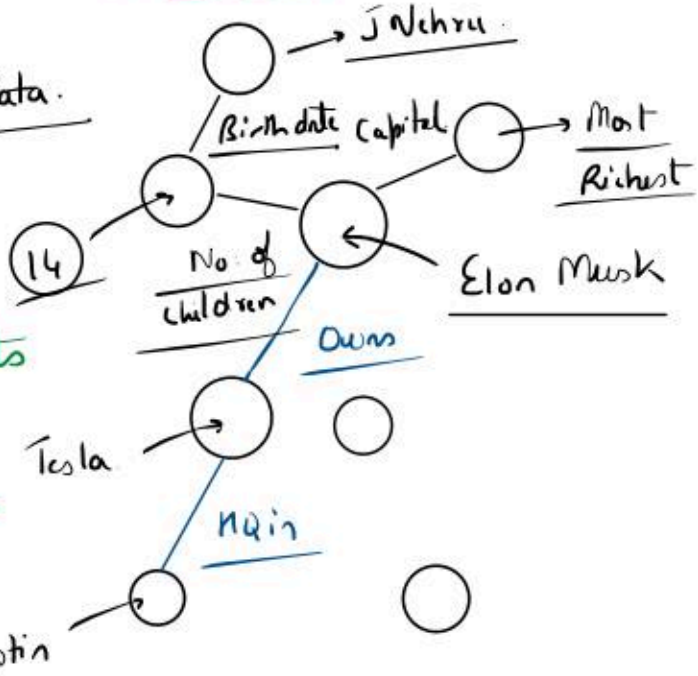
Relationships

2 Key components

① Nodes → Entities

② Edges → Relationships.

Data.



Graph RAG → Relationships b/w entities

① Nodes ⟶ Data

② Edges ⟶ Connection Points (Relationships)

Graph RAG

① Nodes → Data

② Edges → Connections

No.

Python Objects

Graph

Data Structure → Data Store container

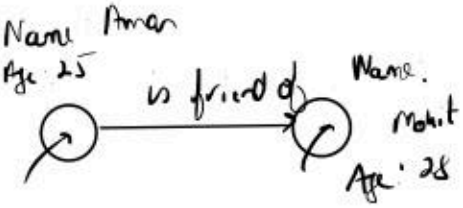
Order

Components

↓

Nodes

Edges } → Python Objects



class Node:

id:

data:

datatype:

class Edges:

relationship:

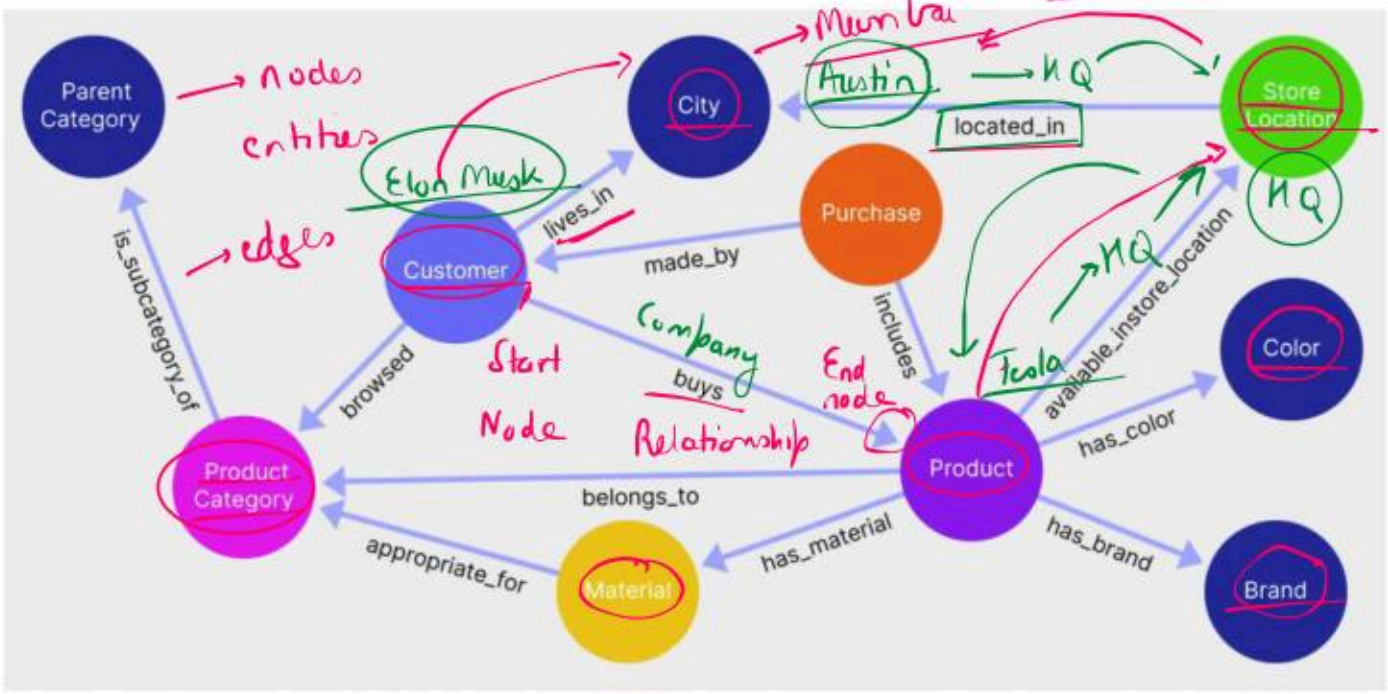
start_node:

end_node:

Graph → Collection of nodes → entities / Data (Name, Age)

Collection of edges (Relationships)

Knowledge Graph



Nodes → separate data entities

Edges → Relationships.

Hops → Traverse the graph } → Navigate

↓ Graph traversal.

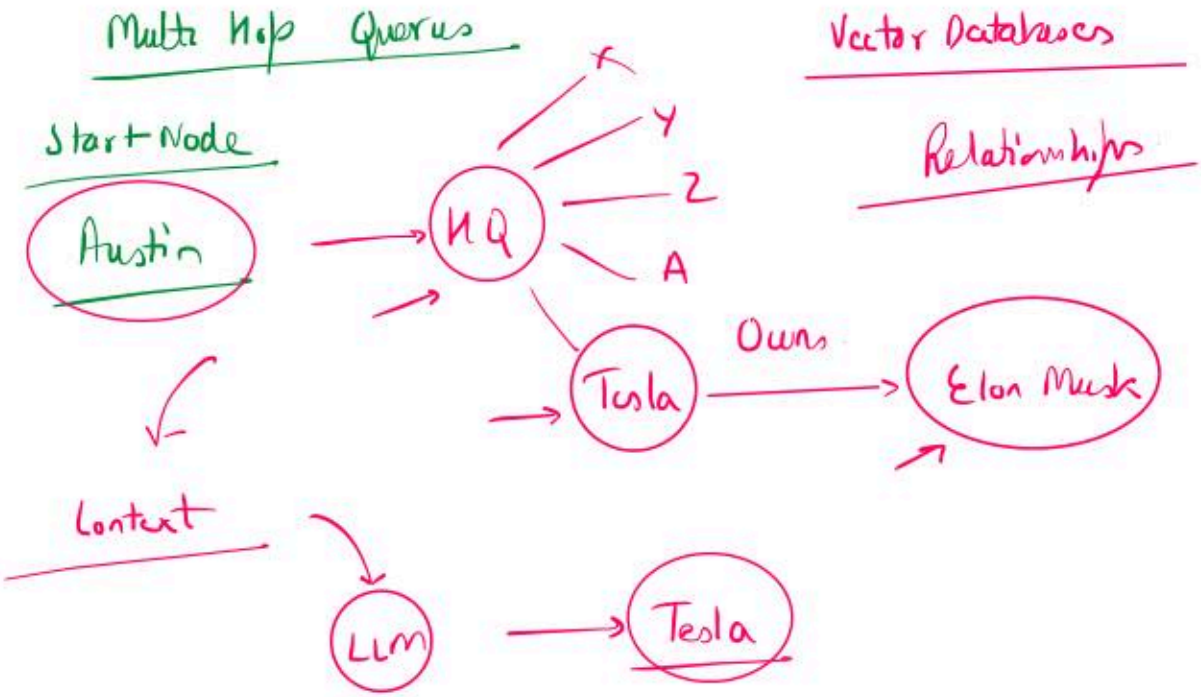
Steps → Hops

Graph RAG → Traverse Hop

✓ Eg Which company of Elon Musk is HQ in Austin?

Multi Hop Queries

Vector Databases


Multi Hop Queries
Vector Databases
Start Node
Relationships
Austin
HQ
X
Y
Z
A
Tesla
Owns
Elon Musk
Context
LLM
Tesla

Graphs (Relationship) Multi Hop Queries

Chunks → Vector Database

① Node → Data.
Entity

Graph → Graph Database.

② Edges → Connections / relations

Vector Database

① Persist your embeddings

Graph database → Data Persist

① Entities ② Connections

② Indexing

② Data → Retrieval quick

Elon Musk → Entity Data text

Node

↓

Data agnostic

```
Node: Elon Musk
├─ name: "Elon Musk"          ← text property
├─ type: "Person"             ← entity type
├─ source: "chunk_42"         ← which chunk it came from
└─ embedding: [0.23, 0.81, -0.14, ...]  ← vector stored ON THE SAME NODE
```

Cypher query language Readable

Data → Entity Node → text

Nodes / edges.

separate

Nodes

```
MATCH (a:Actor)-[:ACTED_IN]->(m:Movie)
WHERE m.name = 'Top Gun'
RETURN a.name
```

embedding model

Graph Database

Persist our data → Entities → Neo 4J

→ Connections

Indexing → Embeddings

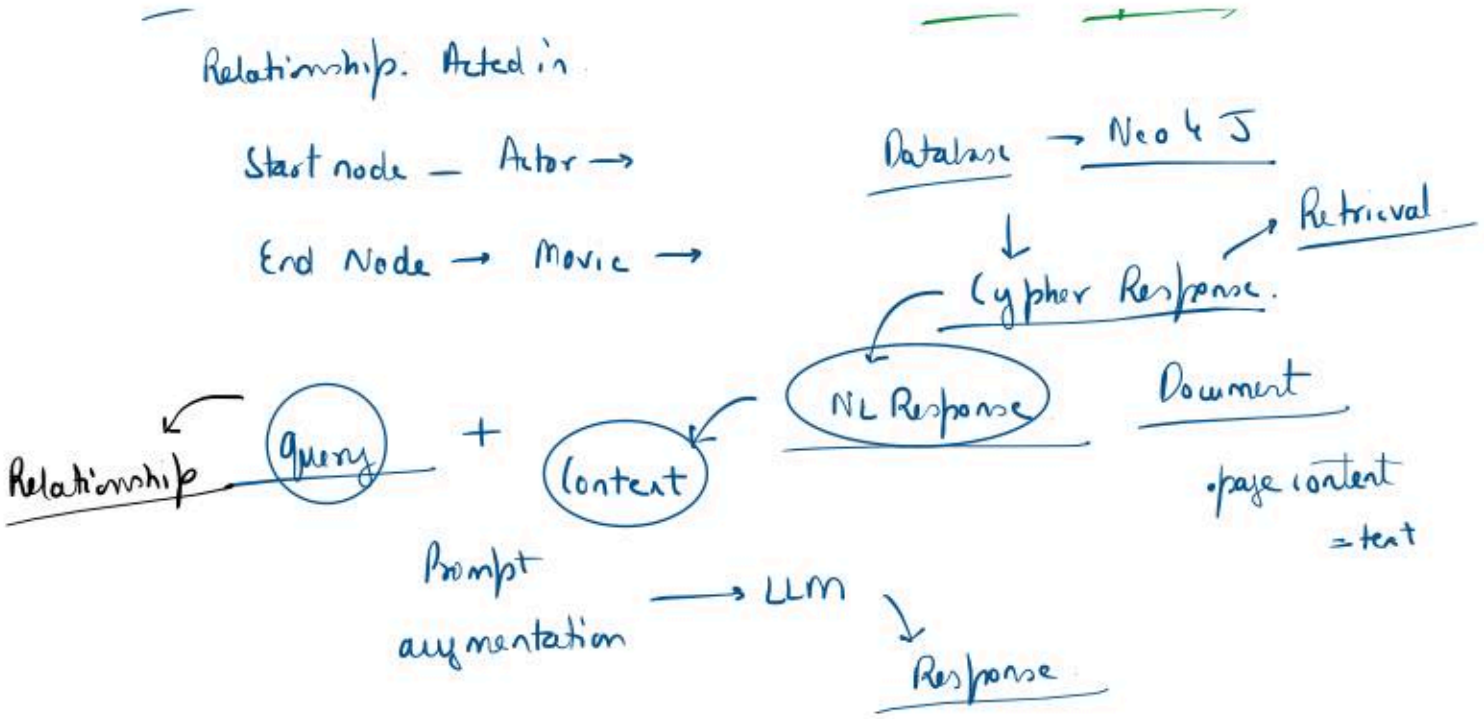
(ACTOR: Tom Cruise) → type. → name.

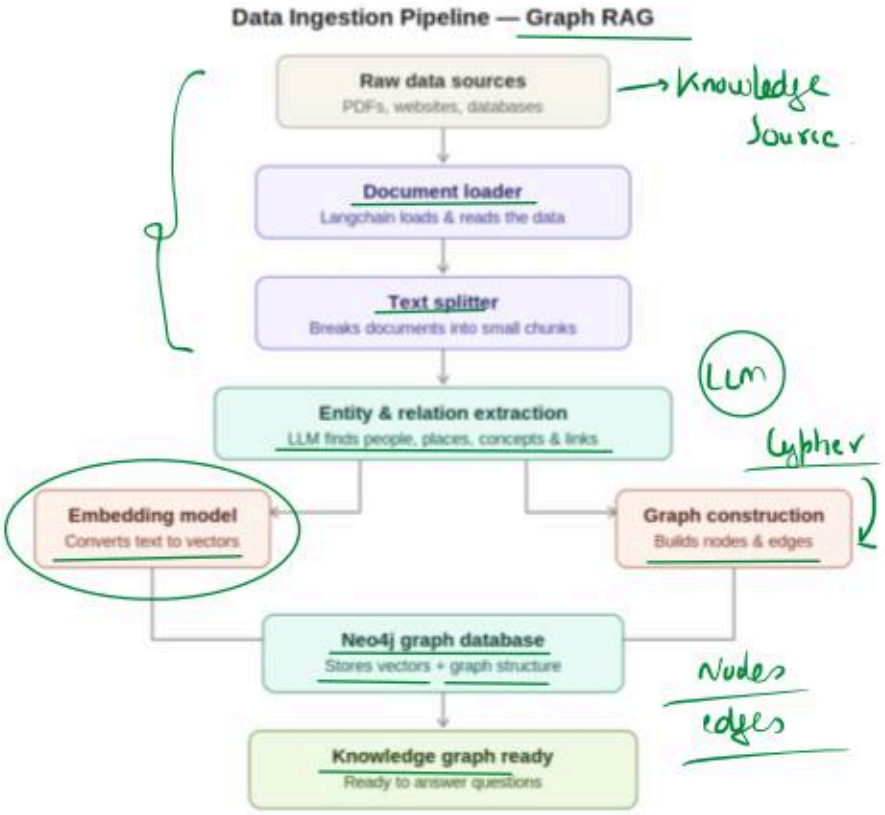
Acted In.

(MOVIE: Top Gun)

Edges

Relationship. Acted in


Relationship. Acted in
Start node — Actor →
End Node → Movie →
Database → Neo 4 J
Cypher Response.
Retrieval
NL Response
Document
page content
= text
Relationship
query
+
Context
Prompt
augmentation
→ LLM
Response



→ Knowledge Source

LLM

Cypher

Nodes
edges

Raw Documents
↓
Knowledge graph.
↙ ↘
nodes edges
(text, embedding vectors)

Neo 4J

✓ Vector Database

① knowledge source → Raw documents → Multiple file types: pdf, web search, doc, csv

Document loaders → Unified format

② Split into chunks → text splitters.

③ embedding model → embedding vector → Semantic meaning

④ Vector Database → Embedding vectors → store.

Graph Databases.

Graph Databases.

① Knowledge Source → Raw documents → text
Document loaders

② Split docs into chunks → Text splitters → text

③ Graph databases → entities (Nodes) — Embeddings X
↓ Relationships (edges)

Chunks → entities + Relationships (Extract)

NER → Named entity recognition. → NER model X
text → entities (type)

LLMs → text semantic meaning

Prompts — specific

① All the entities (Nodes)

② All the relationships b/w the entities (Edges) →

Graph → data to be added ↓ extract

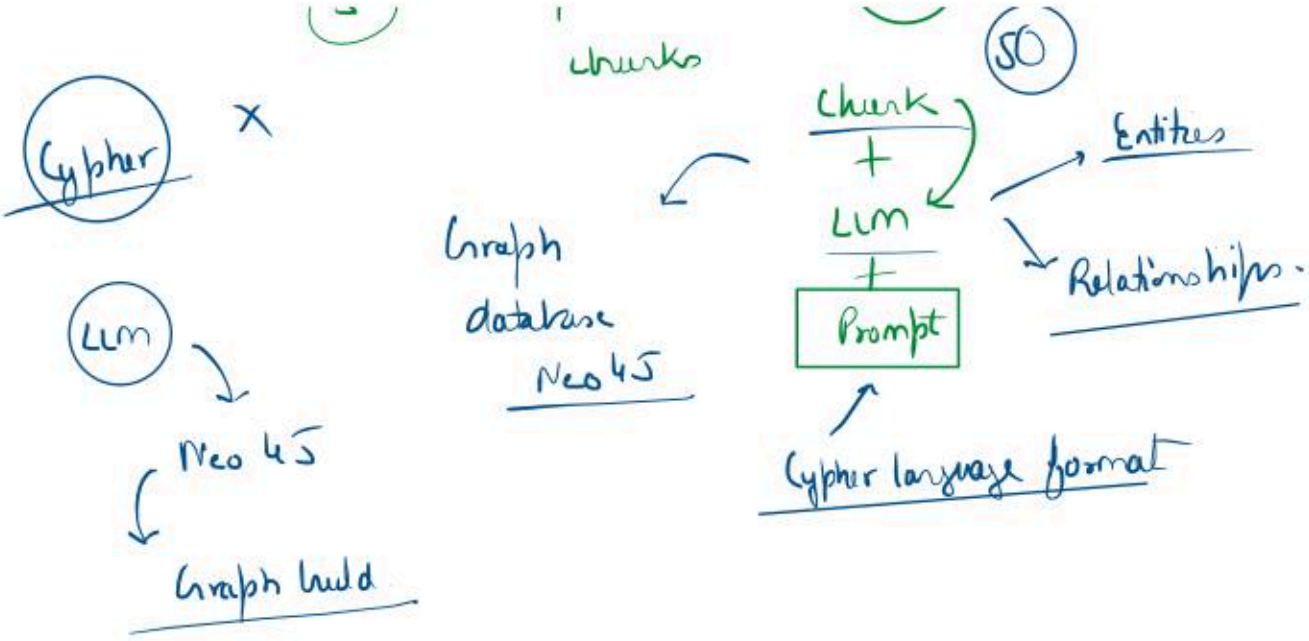
LLM + Prompt

LLM → Semantic Meaning
Context aware

Pdf document → 5 pages

5 → split into → 50
chunks

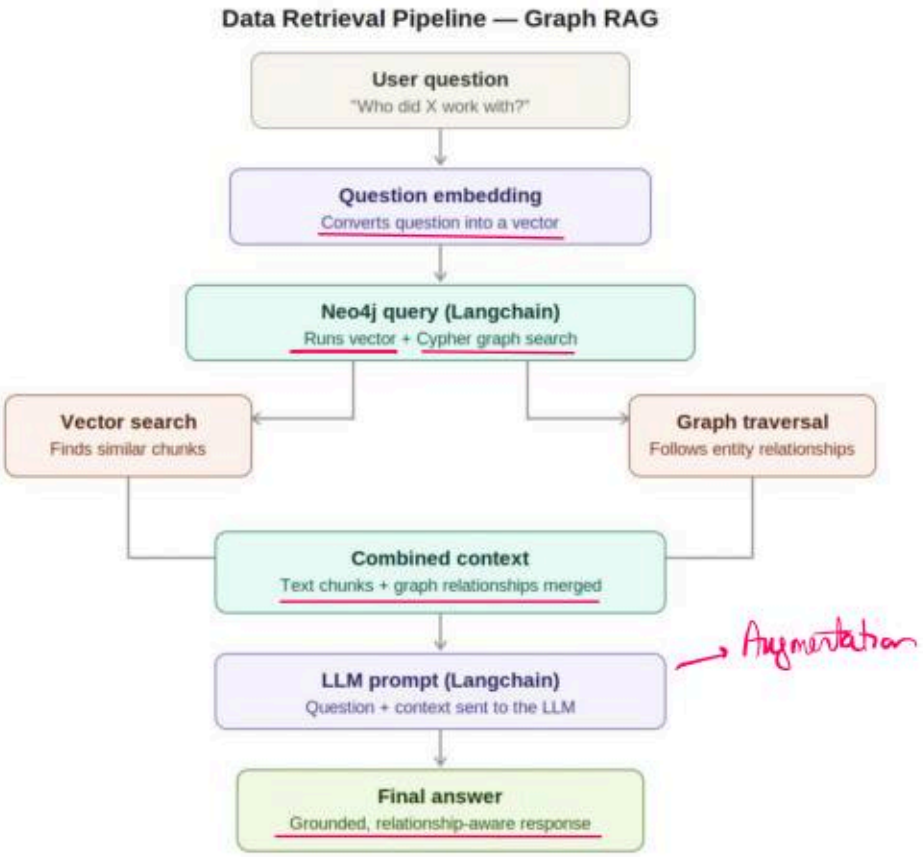
50

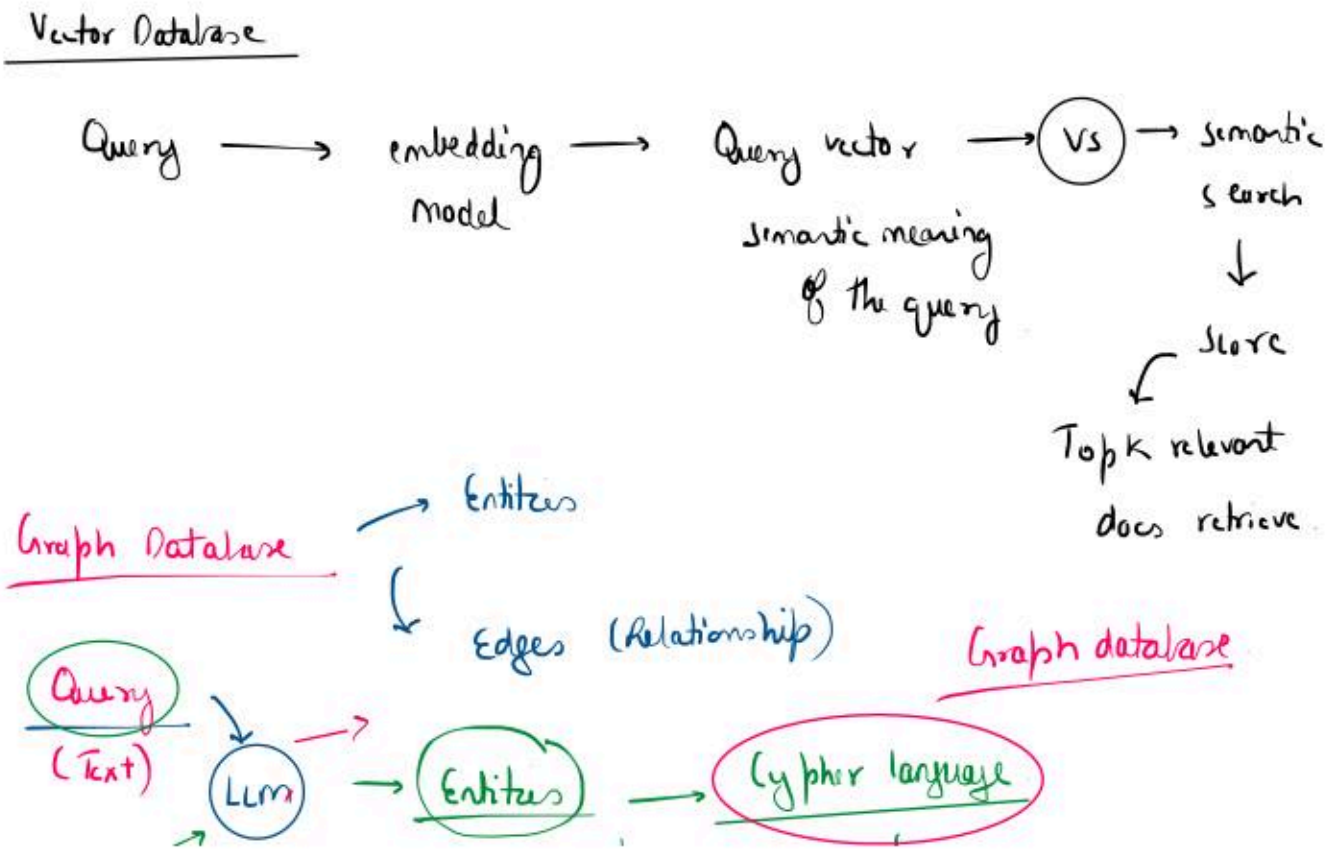


embedding model → entities (nodes)

↓

embedding vector


Data Retrieval Pipeline — Graph RAG
User question
"Who did X work with?"
Question embedding
Converts question into a vector
Neo4j query (Langchain)
Runs vector + Cypher graph search
Vector search
Finds similar chunks
Graph traversal
Follows entity relationships
Combined context
Text chunks + graph relationships merged
LLM prompt (Langchain)
Question + context sent to the LLM
Augmentation
Final answer
Grounded, relationship-aware response


Vector Database
Query
embedding model
Query vector
Semantic meaning of the query
VS
Semantic search
Store
Top k relevant docs retrieve
Graph Database
Entities
Edges (Relationship)
Graph database
Query (Text)
LLM
Entities
Cypher language

(Text) → LLM → Entities → Cypher language

Graph Schema → LLM

Relationships

graph database → Response → Cypher

Eg Which company of Elon Musk is HQ in Austin?

```
Nodes available:
  - Person (properties: name, description)
  - Organization (properties: name, description)
  - Location (properties: name)

Relationships available:
  - FOUNDED (Person → Organization)
  - HEADQUARTERED_IN (Organization → Location)
  - BORN_IN (Person → Location)
```

Graph schema

Neo 4 J

Target node

2 Hops. → NL context

```
[Elon Musk] --FOUNDED--> [Tesla]
[Elon Musk] --FOUNDED--> [SpaceX]
[Elon Musk] --FOUNDED--> [xAI]
[Tesla]     --LOCATED_IN--> [Austin]
[SpaceX]    --LOCATED_IN--> [Hawthorne]
```

1st Hop

2nd Hop

Coverage?

Point of origin

Embedding vectors. → Text

Context

```
Elon Musk founded Tesla. Tesla is located in Austin.
Elon Musk founded SpaceX. SpaceX is located in Hawthorne.
Elon Musk founded xAI.
```

Query → LLM → Entity Relationships

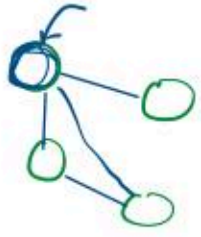
Query → LLM → Tesla

Embedding vector

```
User Query (text)
      ↓
Entity Extraction   → "Elon Musk"
      ↓
Semantic Search     → Find [Elon Musk] node in Neo4j
      ↓
Graph Traversal     → Follow edges outward
      ↓
Context Assembly    → Collect nodes + relationships
      ↓
LLM                 → Generate final answer
```

Elon Musk / Austin → Embedding model.

Embedding vector



Graph databases.

Graph databases.



① Semantic Search → Start node

② Graph traversal → Context → target

node → embedding vector } Semantic search → Start node

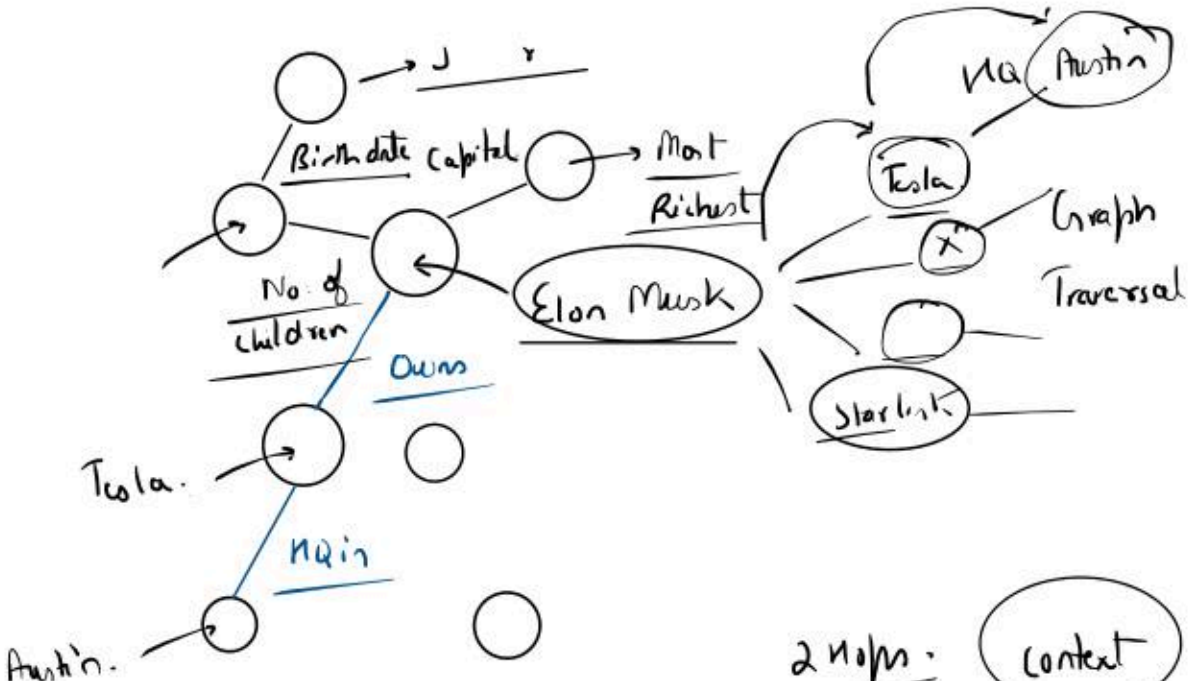
Query → LLM → Entity → Elon Musk

Entity → embedding model → Embedding vector

Graph database → Nodes

Traverse

Most similar embedding vector ← Semantic search ← Graph database (100)



2 Hops. → Context → NIL Context

T-hit → Text (Query)

Input → Text (Query)

Response → Text

NL Context

Graph database

Nodes

Edges

Traversal

Context → Coverage ↑

Traversal.

BFS

Elon Musk

Tesla

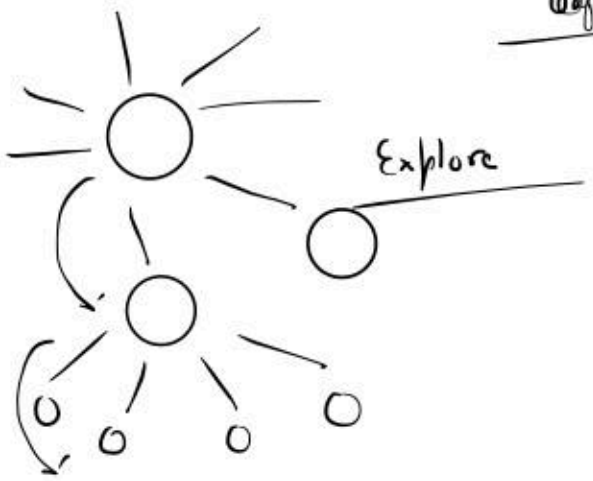
Neighborhood

End node

Leaf node.

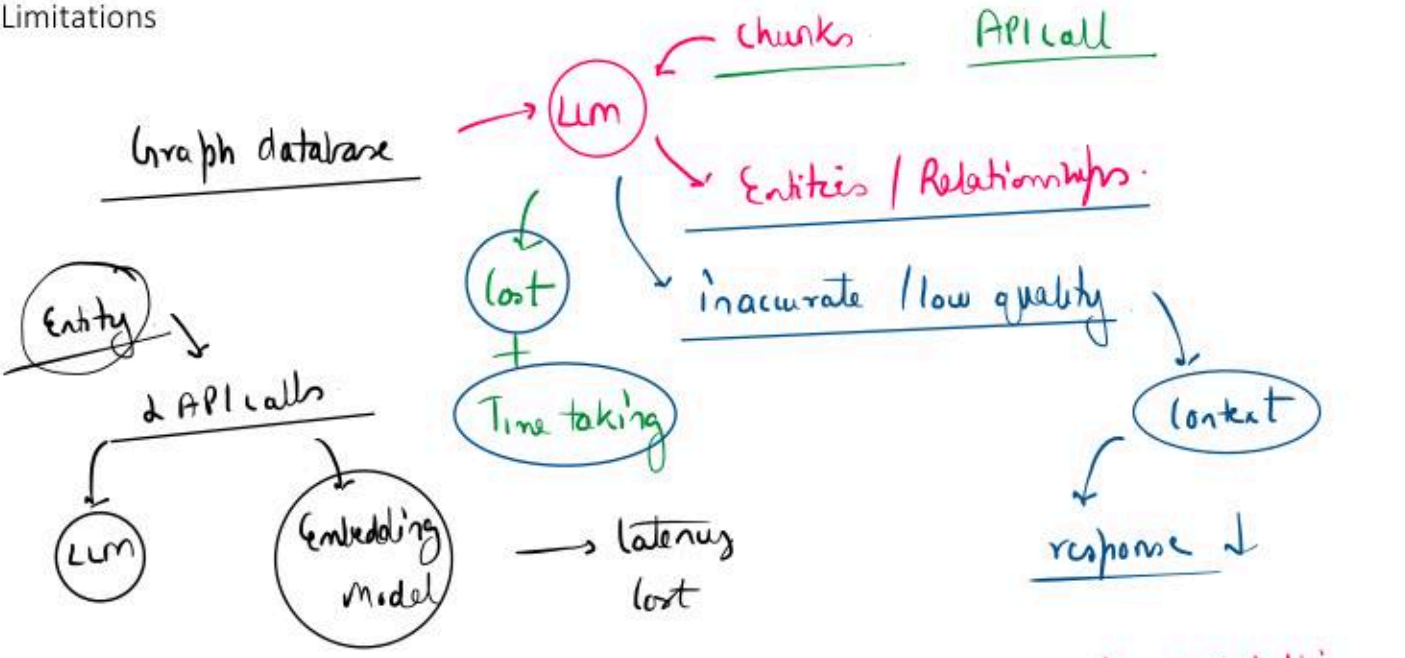
① Depth first search (DFS)

② Breadth first search (BFS)



Context Coverage ↑

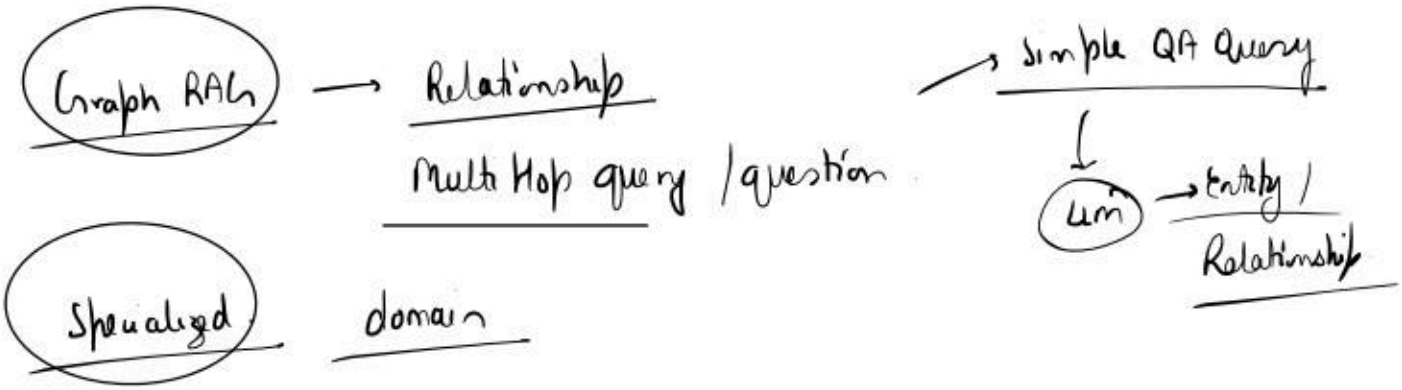
Limitations



Use Graph RAG when	Use Regular RAG when
Data has many interconnected entities	Simple Q&A over small documents
Questions require multi-hop reasoning	Semantic similarity is sufficient
Relationships across documents matter	Fast, cheap ingestion is a priority

LLM + Embedding

Embedding Model



RAGAS → RAG Assessment

↓

RAG systems → Evaluate → score

accuracy

↓

metric

Accuracy ↓

① Ingestion Phase

a) knowledge source → Raw Docs

✓ b) Chunking → Experimentation.

Input

Strategy

Parameters

chunk overlap

chunk size

✓ c) Embeddings → Embedding model.

Provider

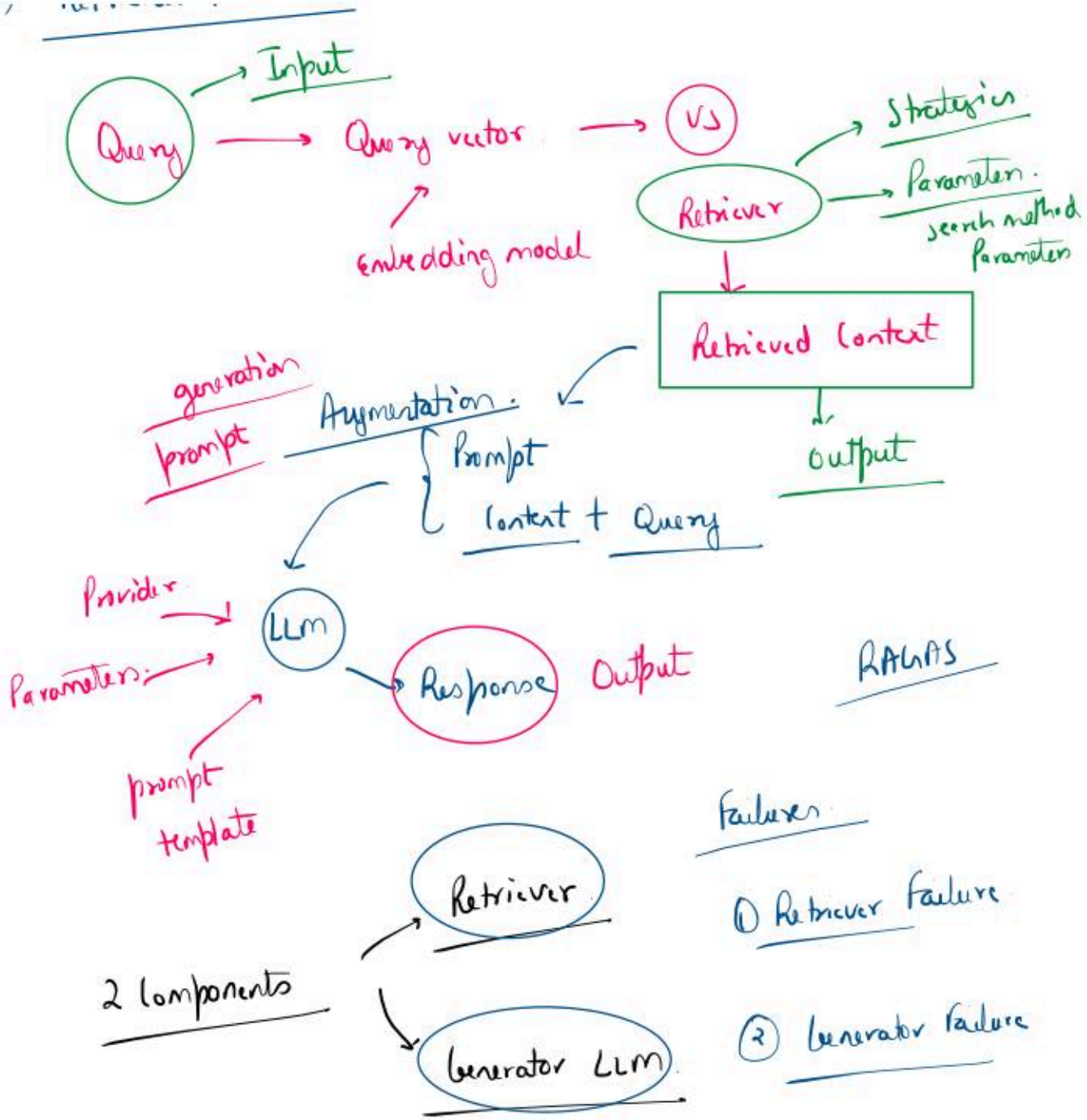
Parameters.

① output dimensions

d) Vector Database → Provider

Indexing

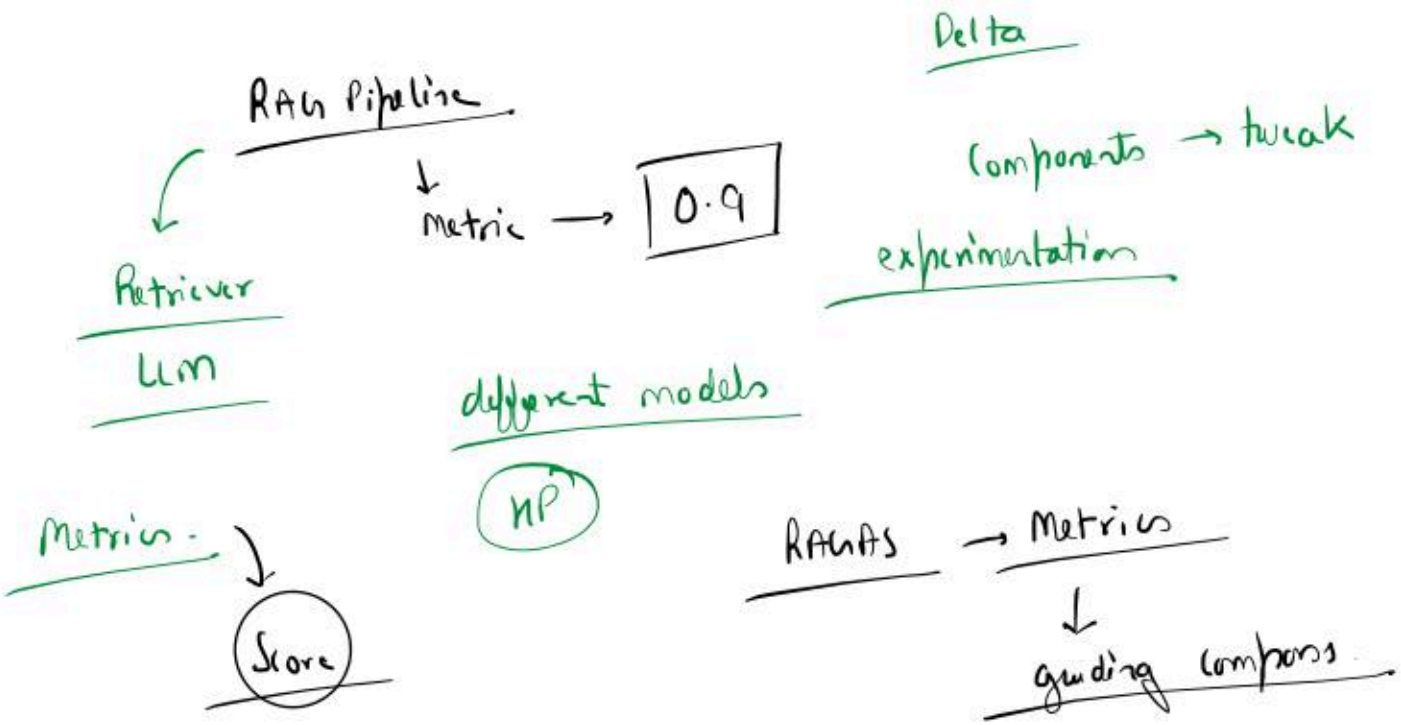
2) Retrieval Phase

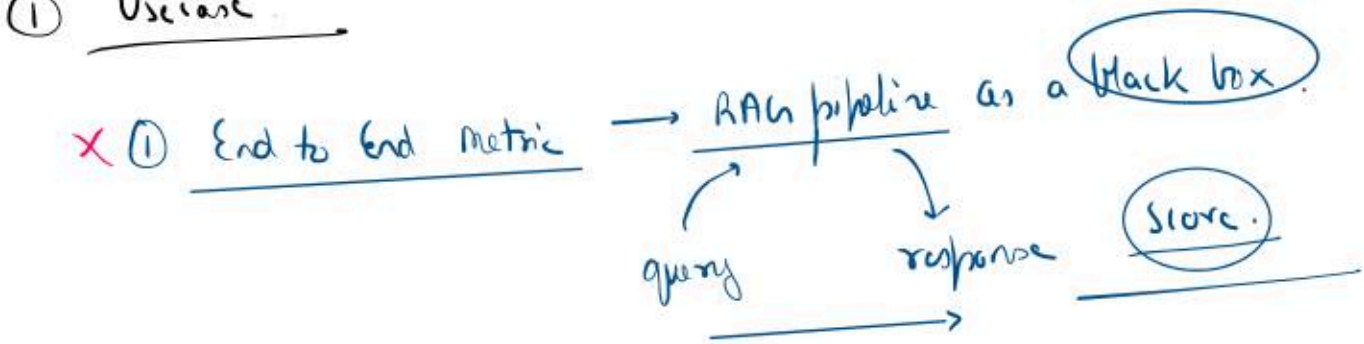

Query
Input
Query vector
VJ
Strategies
Retriever
Parameter.
search method
Parameters
Embedding model
Retrieved Context
generation prompt
Augmentation.
Prompt
Output
Context + Query
Provider
LLM
Parameters:
Response
Output
RAGAS
prompt template
Failures
Retriever
1) Retriever Failure
2 components
Generator LLM
2) Generator Failure

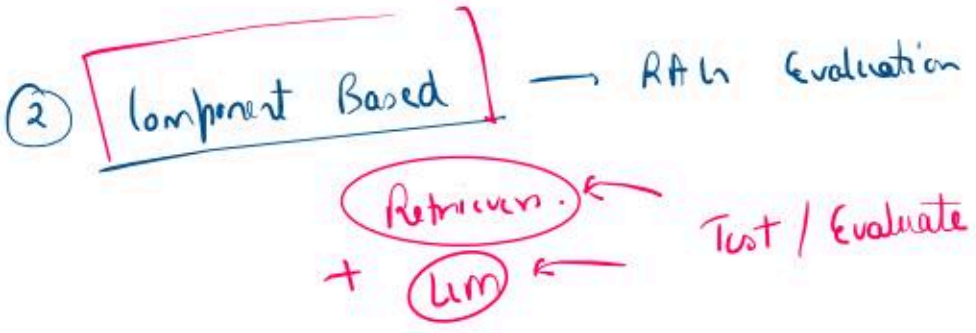
Metrics:


ML lifecycle
evaluation criteria
metric
Train
Evaluate
Regression
Classification
Precision
Recall
F1 score
Accuracy
R2 score
MAE
RMSE
RAG pipeline


(1) Retriever
(2) LLM


Delta
RAG Pipeline
metric → 0.9
Components → tweak
experimentation
Retriever
LLM
different models
HP
Metrics
Store
RAGAS → Metrics
guiding components


① Usecase
× ① End to End Metric → RAG pipeline as a black box
query
response
Store.


② Component Based → RAG Evaluation
Retriever.
+ LLM
Test / Evaluate


③ Business Metrics → ×


① Retriever only
③ Retriever + LLM

① Retriever only

② LLM only

③ Retriever + LLM

Context → Response

Output type

① Discrete output based. → pass, fail — category

② Numerical output based → int, float — range

↓

Statistical analysis, operations

③ Ranked output based → Ranked list

↓

multiple elements → Ranked

RAGAS → Agentic systems

Turns

① Single turn. → Input → Processing → Output — end

RAG Pipeline → Query → Response

② Multi turn

↓

Agentic systems. Loop. conversational

Agentic systems:

ReAct loop

RAG Pipeline → component Based

RAG Pipeline → Single turn metrics

RAG pipeline.

quality matters over quantity

LLMs

① LLM Based Metrics

② Non LLM Based Metrics

→ Accuracy (LLM)

① LLM Based

② Non LLM Based

LLM as a Judge

RAG pipeline.

Response → Capital of India is Mumbai

① Retriever → Retrieved docs

② LLM → Response.

Grounded Response → Capital of India is New Delhi

Answer Key

Retriever → Correct Retrieved docs

Comparison

LLM → Correct Response

LLM

Semantic Similarity

RAG Pipeline

Keyword matching

Semantic Meaning

LLM

LLM Based Metrics

Response

LLM → Judge → Score.

Grounded Response

NLU → Similarity

Comparison

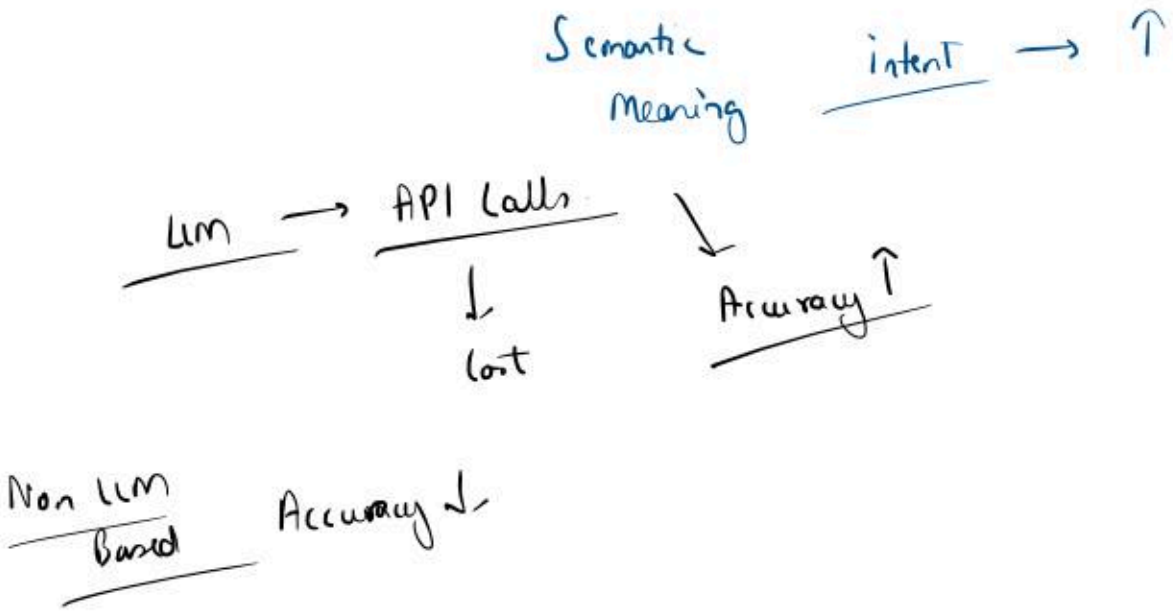
Response

Context } Query

LLM

Comparison

Semantic intent → ↑


Semantic
Meaning
intent
LLM
API Calls
Cost
Accuracy
Non LLM
Based
Accuracy

1. sample
2. user_input
3. retrieved_contexts
4. response
5. reference
6. reference_contexts
7. Evaluation Dataset
8. Experiments

① sample → 1 Row, test query → RAG pipeline

② user_input → query

③ retrieved_context → [str]

④ response → RAG Pipeline → output

⑤ reference → Answer Key | Grounded Response

⑥ reference_contents → retriver. | Grounded context

⑦ Evaluation Dataset → RAG pipeline evaluate

train data
↙ ↘
train set | val set

train set (val set)

(8) [Experiments] → RAG pipeline.

[retriever] → parameters

[LLM] → parameters

retriever - sim search

K : 3 → (5)

LLM → temp = 0.7

What do these metrics target

① Retriever

② LLM

a) Retrieved context → Relevant (input)

b) Retrieved context → fact + noise.

LLM

a) Response → grounded → hallucinations.

b) Response → query → Useful. to answer the query

c) Response → Robust

a) Context Recall

b) Context Precision

c) Noise sensitivity

d) Response Relevance

e) Faithfulness

① What are these metrics used for?

② Step by step procedure to calculate

③ formula

④ Results → score → interpret

⑤ Components

Component based

① Retriever.

② LLM

Retriever

Query → AI Agents

5 chunks

Context Recall evaluates whether the retriever **fetched all the information needed** to answer the question correctly. It checks for completeness — did the retriever miss anything important?

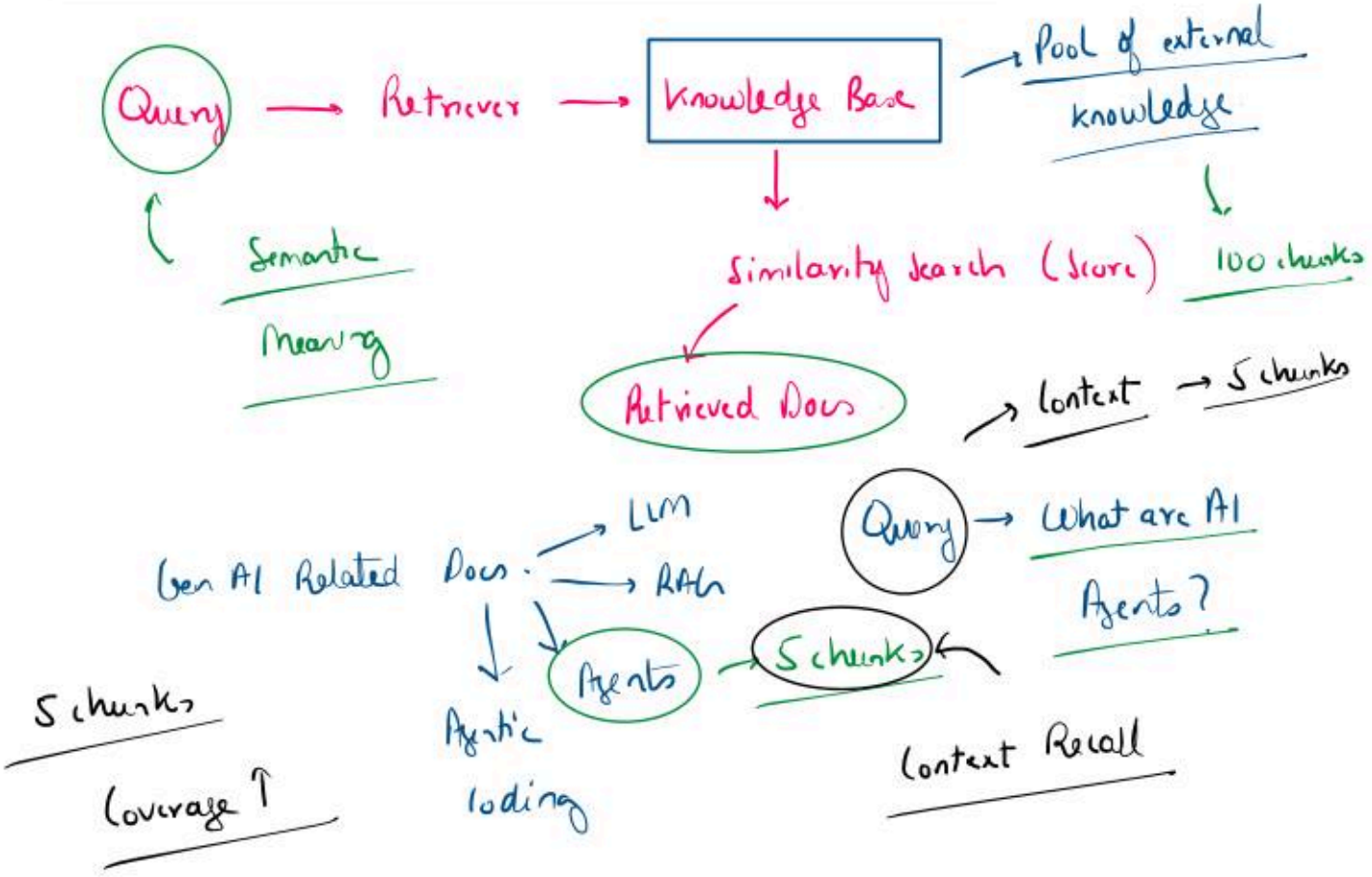
Context Recall measures how many of the relevant documents (or pieces of information) were successfully retrieved. It focuses on not missing important results.

count

query → context

$$\text{Context Recall} = \frac{\text{Number of claims in reference attributed to retrieved context}}{\text{Total number of claims in reference}}$$

2 / 4 = 0.5



Your question is: *"Tell me about the Eiffel Tower."* → Query

Reference answer (ground truth): *"The Eiffel Tower is located in Paris. It was built in 1889. It is 330 metres tall. It was designed by Gustave Eiffel."* → Perfect response

Step 1 — Break the reference into individual claims:

Claim #	Claim	Semantic meaning
C1 ✓	✓ The Eiffel Tower is located in Paris	
C2 ✓	It was built in 1889	
C3	It is 330 metres tall ✗	
C4	It was designed by Gustave Eiffel ✗	

LLM

API call

- Chunk 1: *"The Eiffel Tower stands in the heart of Paris, France."* ✓
- Chunk 2: *"Construction of the Eiffel Tower was completed in 1889 for the World's Fair."*
- Chunk 3: *"France is home to many famous landmarks including the Louvre museum."*

Retrieved context

LLM

LLM as a Judge → LLM based techniques.

Recall = 2/4 = 0.5

① Query → User → AI Agents

② Grounded Response → Generation Response → Perfect Response

Grounded Response → Reference → Facts list down Answer the Query

a) LLM to extract all the imp facts from the grounded response. → 10 facts → AI Agents

b) Compare the facts extracted with my retrieved context → retriever

60% → 6 facts → Recall = 6/10 = 0.6

40% ✗

1 → 0 / 0 → 1 (FP) ✗

ML → Recall → TP / (TP + FN) → All positive rows. / Actual Positives.

Coverage

Negative

K: 10

5 chunks

Recall ↑

5 chunks AI Agents

3 chunks LLMs ✗

2 chunks RAGs ✗

3 chunks AI Bent

5 chunks LLM

2 chunks RAG

Recall ↓

Query → Related chunks.

[0 - 1]

Recall ↓

What a low score tells you:

- Your retriever is missing important chunks from the vector store
- The LLM will be forced to either hallucinate the missing facts or give an incomplete answer — through no fault of its own
- Possible causes: chunk size too small, top-k too low, embedding model not capturing semantic meaning well enough

Query

Retriever

What a high score tells you:

- Your retriever is successfully fetching all the information needed to construct a complete answer
- The LLM has everything it needs — whether it uses it well is then measured by the generation metrics

Coverage ↑

Context → Coverage ↑

Retriever

Context Precision

Context Precision evaluates whether the **relevant chunks are ranked higher** in the retrieved context list. It is not just about *whether* the right chunks were retrieved — it is about *where* they appear in the list.

Context Precision is a metric that evaluates the retriever's ability to rank relevant chunks higher than irrelevant ones for a given query in the retrieved context. Specifically, it assesses the degree to which relevant chunks in the retrieved context are placed at the top of the ranking.

It is calculated as the mean of the precision@k for each chunk in the context. Precision@k is the ratio of the number of relevant chunks at rank k to the total number of chunks at rank k.

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K} (\text{Precision@k} \times v_k)}{\text{Total number of relevant chunks in top K}}$$

Where:

$$\text{Precision@k} = \frac{\text{true positives@k}}{\text{true positives@k} + \text{false positives@k}}$$

V_K = Boolean value

Relevancy factor

Query + C1 → LLM

Retriever → K = 5

Query → "What is the capital of France?"

Query + C1 → LLM

Position (k)	Chunk	Relevant? (v_k)
1	"Paris is the capital of France."	✅ 1 True
2	"France is famous for its cuisine."	❌ 0 Irrelevant
3	"The Eiffel Tower is in Paris."	✅ 1
4	"France joined the EU in 1993."	❌ 0
5	"Paris has a population of 2 million."	✅ 1

Position (k)	Chunk	Relevant? (v_k)
1	"Paris is the capital of France."	✅ 1
2	"The Eiffel Tower is in Paris."	✅ 1
3	"Paris has a population of 2 million."	✅ 1
4	"France is famous for its cuisine."	❌ 0
5	"France joined the EU in 1993."	❌ 0

What a low score tells you:

- Your retriever is fetching relevant chunks but burying them behind irrelevant ones
- The LLM ends up seeing noise before signal — which can hurt the quality of the generated answer
- You may need to tune your similarity search, re-ranking strategy, or embedding model

What a high score tells you:

- Your retriever is correctly prioritizing the most relevant chunks at the top
- The LLM gets the best context first, giving it the best chance to generate a good answer

Similarity search

ReRanker

Precision@K → for each of the K value

K = 5 → 5 times

$K = 5$ — 5 times

$$\text{Precision@k} \times v_k$$

1 upto that K

$$\frac{TP}{TP + FP}$$

$v_k \rightarrow$ Boolean value

1 / 0

Ranked at higher position + Relevant

✓ Precision @ 1 = $\frac{1}{1} \times 1$

$= 1$

✓ × Precision @ 2 = ✓ $\frac{1}{2} \times v_k = 0.5 \times 0 = 0$

Precision @ 3 = $\frac{2}{3} \times 1 = 0.67$

Precision @ 4 = 0

Precision @ 5 = $\frac{3}{5} = 0.6$

Context Precision = $\frac{✓ 1 + ✓ 0.67 + ✓ 0.6}{3}$

$= \frac{2.27}{3}$ 2.27/3=0.7567

top 5

2 position

4 position

Context Precision @ 5 = 0.756 → 1

2 position
4 position

Context Precision @ 5 = 0.756

Relevant + High Ranking → Top positions | Relevant |

Precision @ 1 = $\frac{1}{1} \times 1 = 1$

Precision @ 2 = 1

Precision @ 3 = 1

= Context Precision @ 5 = $\frac{3}{3}$

→ = 1

top positions

Retriever

Precision, Recall } Retriever

5 Metrics

Robustness of LLM

Noise Sensitivity evaluates **how much the LLM is misled by irrelevant chunks** in the retrieved context. It measures the robustness of your LLM — does it stay faithful to the relevant chunks, or does it get distracted by the noise and start making incorrect claims?

$$\text{Noise Sensitivity} = \frac{\text{Number of incorrect claims in response attributed to noise}}{\text{Total number of claims in response}}$$

(2) No. of noisy claims Response. 0.5

(4) LLM → Noisy info

Noise

Query

Your question is: *"When was the Eiffel Tower built and how tall is it?"*

Reference answer (ground truth): *"The Eiffel Tower was built in 1889 and is 330 metres tall."*

Facts extracts

Built in 1889

350 m tall

Step 1 — Your retriever fetches these chunks:

Retrieved chunks

Chunk	Type
Chunk 1: "The Eiffel Tower was completed in 1889." ✓	✅ Relevant
Chunk 2: "The Eiffel Tower stands 330 metres tall." ✓	✅ Relevant
Chunk 3: "The Louvre Museum attracts 9 million visitors per year." ✗	❌ Noise
Chunk 4: "The Arc de Triomphe is 50 metres tall." ✗	❌ Noise

Useful

Noisy

Response generated by LLM

"The Eiffel Tower was built in 1889. It stands 330 metres tall. It attracts 9 million visitors per year. It is 50 metres tall."

① ✗ ② ✗ ③ ✓ 0.5 ✗

noisy

④ ✓

Noisy

Noisy

Reference / Grounded Response → without noise.

"The Eiffel Tower was built in 1889 and stands 330 metres tall."

$$NS = \frac{2}{4} = 0.5$$

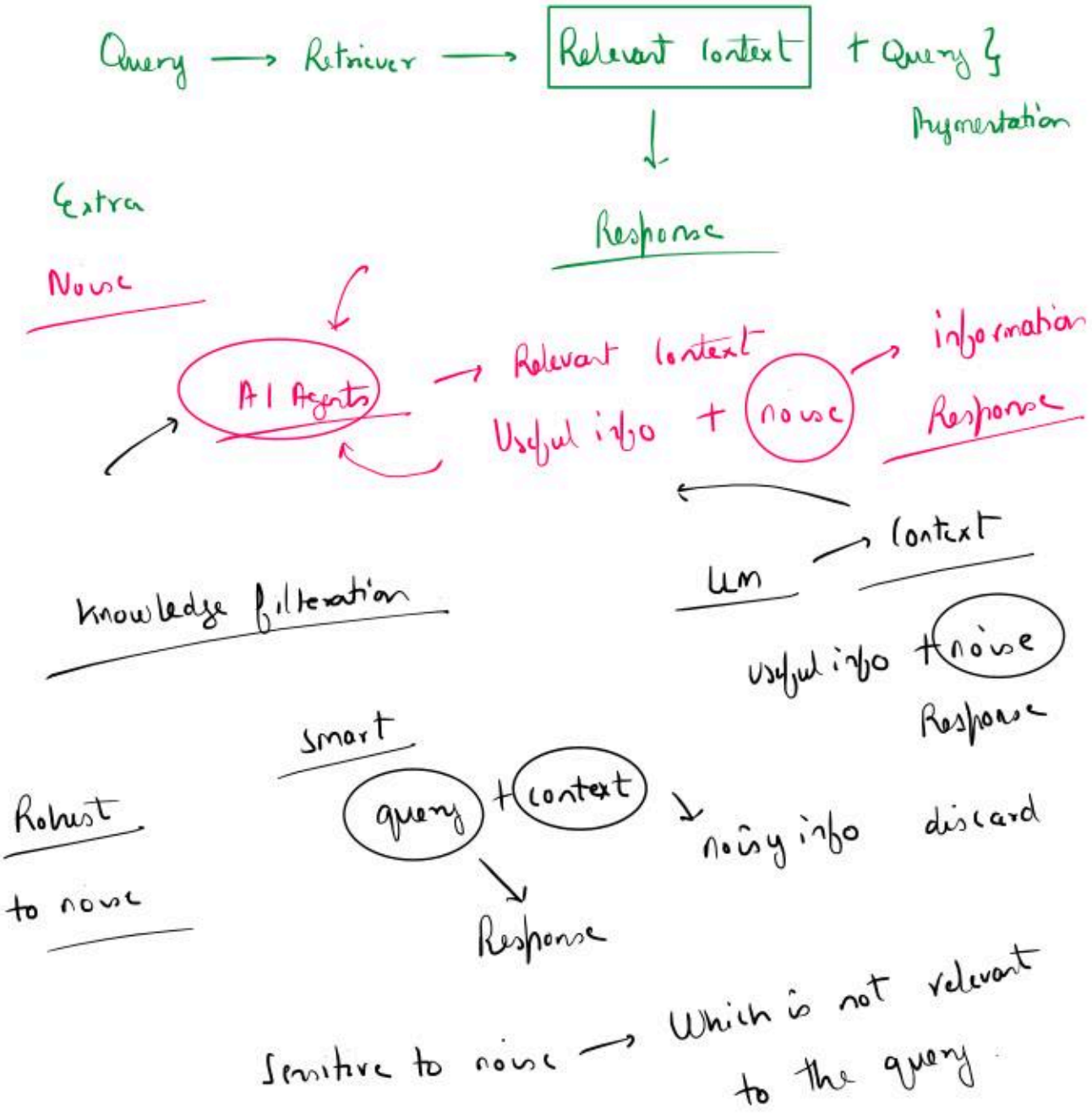
① Query

② Grounded Response / Reference → Actual Answer → Facts those

are useful facts only.
↓
without noise

③ Response → Generated by LLM

④ Retrieved context → Chunks.



Steps

1. Grounded Response → LLM → Facts / Claims → extract imp details
2. Retrieved Docs → Match → Useful info
   Not Matching → Noisy info
3. Response → Facts → count → noisy facts

What a high score tells you: Bad.

- Either your retriever is bringing in too many irrelevant chunks, or
- Your LLM is not robust enough to ignore irrelevant context, or
- Both

Retriever ✗

What a low score (close to 0) tells you: Good.

- Your LLM is successfully distinguishing relevant from irrelevant chunks
- Even when the retriever brings in noise, the LLM is not misled by it

LLM → query, context, reference, response → intent

Keyword matching

NS ↑ = Robustness ↓

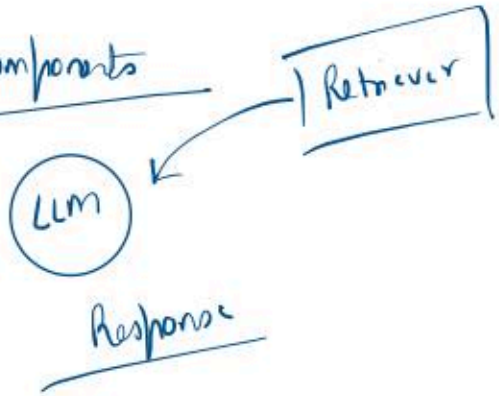
Lower is Better

0 → Noisy info ✗

1 → Noisy info ✓

OS = Half of response is made up of noisy facts.

Components



Response Relevance

Response → useful to answer the query → factual accuracy

Response Relevancy evaluates whether the **generated response actually addresses the user's question**. It checks if the answer is on-topic and directly relevant to what was asked — not whether it is factually correct.

Faithfulness

Mean:

$$\text{Response Relevancy} = \frac{1}{N}\sum_{i=1}^{N} \cos(e_{g_i}, e_o)$$

$\cos(e_g, e_o)$ e_g → embedding gen e_o → embedding original

Response

3 queries

N=3

$\cos(e_1, e_0) + \cos(e_2, e_0) + \cos(e_3, e_0)$

Query to Query comparison

Goal → Response is it useful or not → to answer the query

Reverse Eng

Response → LLM → Questions → query + context → facts

① Response → LLM

↓

Original query hypothetical 3 queries

② 3 queries → Embedding model → Embeddings

queries

original query

Similarity → Cosine similarity

3

Embeddings

1

Original query → vector embedding

Hypothetical queries → embeddings

Cosine similarity

Similarity score

Cosine similarity

HyDE

Response

User Input

Cosine similarity

① Response

Query → "What is the capital of France?"

LLM

3 scores

Response → "Paris is the capital of France. It is also the largest city in the country."



What a low score tells you:

- Your LLM is going off-topic or giving evasive answers
- Your prompt template may not be instructing the LLM clearly enough to stay focused on the question
- The LLM may be padding responses with unnecessary information

What a high score tells you:

- Your LLM is staying focused and directly addressing what the user asked
- The prompt template is working well in keeping the LLM on track

Factual Accuracy ✗

Faithfulness

Faithfulness

Facts in response → Grounded in retrieved context

Faithfulness evaluates whether **every claim made in the generated response can be traced back to the retrieved context**. It is the primary **hallucination detection** metric in RAGAS.

A response is considered faithful if the LLM only says things that are directly supported by the retrieved chunks — it should not bring in outside knowledge, make assumptions, or invent facts that are not present in the context.

↓ hallucinations

Context

LLM → Made up facts → Response → Retrieved context

$$\text{Faithfulness} = \frac{\text{Number of claims in response that are supported by retrieved context}}{\text{Total number of claims in response}}$$

→ True

↓ LLM → Extract

① Retrieved context → Retriever

② Response → LLM after reading the context → ③

Steps

① Response → facts extract → LLM

② S facts extract → Retrieved context → Binary label True / False

③ Count the number of True value.

↓ grounded

↓ hallucinating

Query "Tell me about Albert Einstein."

Retrieved context

Retrieved chunks:

- Chunk 1: *"Albert Einstein was born on 14 March 1879 in Ulm, Germany."*
- Chunk 2: *"Einstein developed the theory of relativity, one of the two pillars of modern physics."*
- Chunk 3: *"Einstein was awarded the Nobel Prize in Physics in 1921."* ×

3 chunks

Response

"Albert Einstein was born on 14 March 1879 in Germany. He developed the theory of relativity. He won the Nobel Prize in Physics in 1921. He was also known for his work on quantum mechanics. He had an IQ of 160."

LLM → Key facts

Response + Context + Query

Claim #	Claim
C1	Born on 14 March 1879 in Germany
C2	Developed the theory of relativity
C3	Won the Nobel Prize in Physics in 1921
C4	Known for work on quantum mechanics
C5	Had an IQ of 160

What a low score tells you:

- Your LLM is hallucinating — making claims beyond what the retrieved context supports
- Your prompt template may not be clearly instructing the LLM to only use the provided context
- The LLM may be over-relying on its own parametric knowledge instead of the retrieved chunks

What a high score tells you:

- Your LLM is staying strictly within the boundaries of the retrieved context
- Hallucination is under control